



AIDOS LAB
AI FOR DATA-ORIENTED SCIENCE



Learning on topological data: Graph neural networks, expressivity and higher-order message-passing

Lecture 3 — Geometry and Topology in Machine Learning

Lecture 3 — Learning on topological data

Outline

- Graphs are all around us...
- Towards learning on graphs: learning on sets
 - Permutation invariance and equivariance
- Learning on graphs: message-passing
- Expressivity: what tasks can message-passing solve?
- Extending message-passing to higher-order domains: topological neural networks

Lecture 3 — Learning on topological data

Resources and acknowledgements

First part of the talk inspired by Theoretical Foundations of Graph Neural Networks, by Petar Veličković
(<https://www.youtube.com/watch?v=uF53xsT7mjc>)

Also take a look at:

Bronstein, Michael M., Joan Bruna, Taco Cohen, and Petar Veličković. “Geometric Deep Learning: Grids, Groups, Graphs, Geodesics, and Gauges.” *arXiv Preprint arXiv:2104.13478*, 2021.

and at the page <https://geometricdeeplearning.com/>

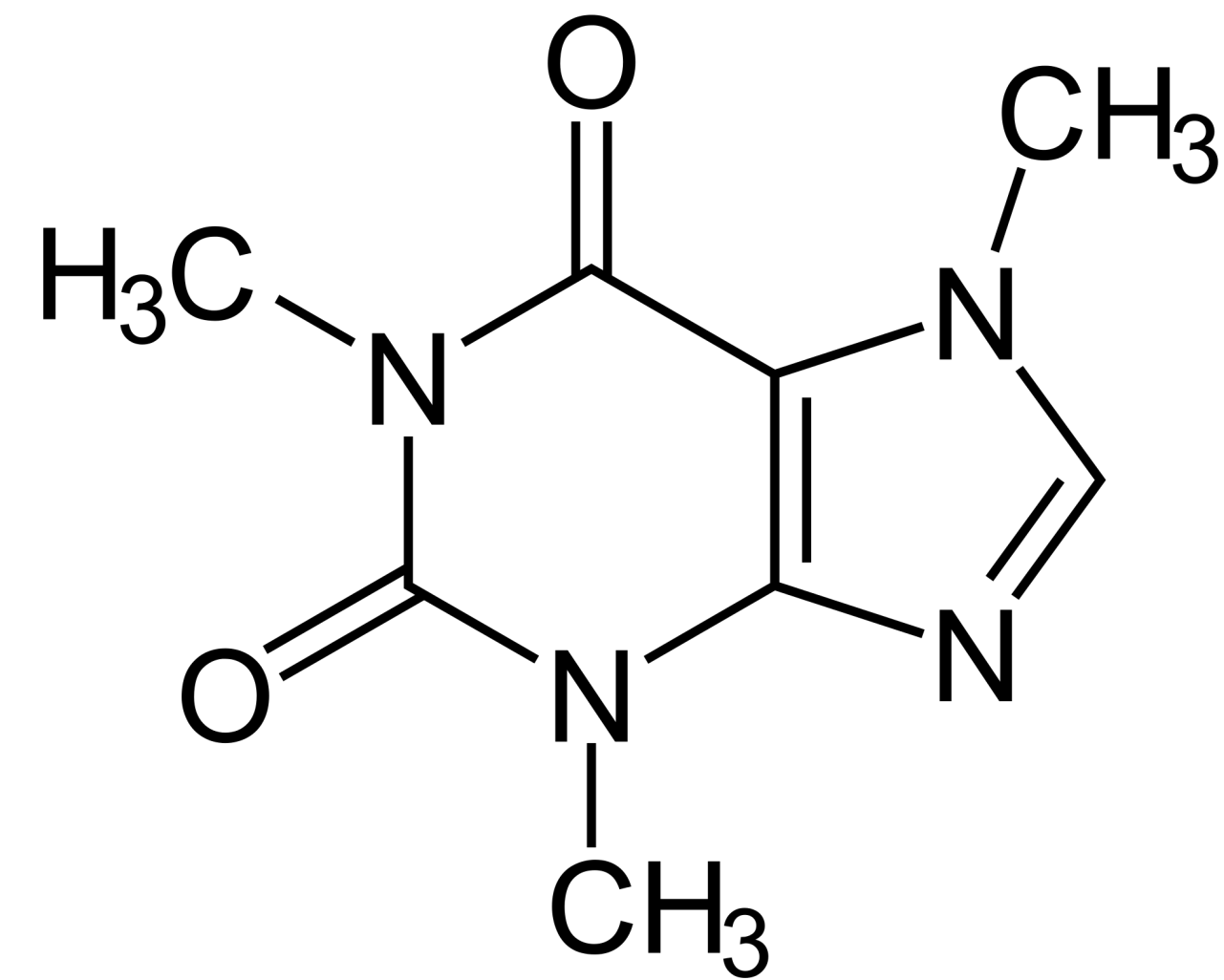
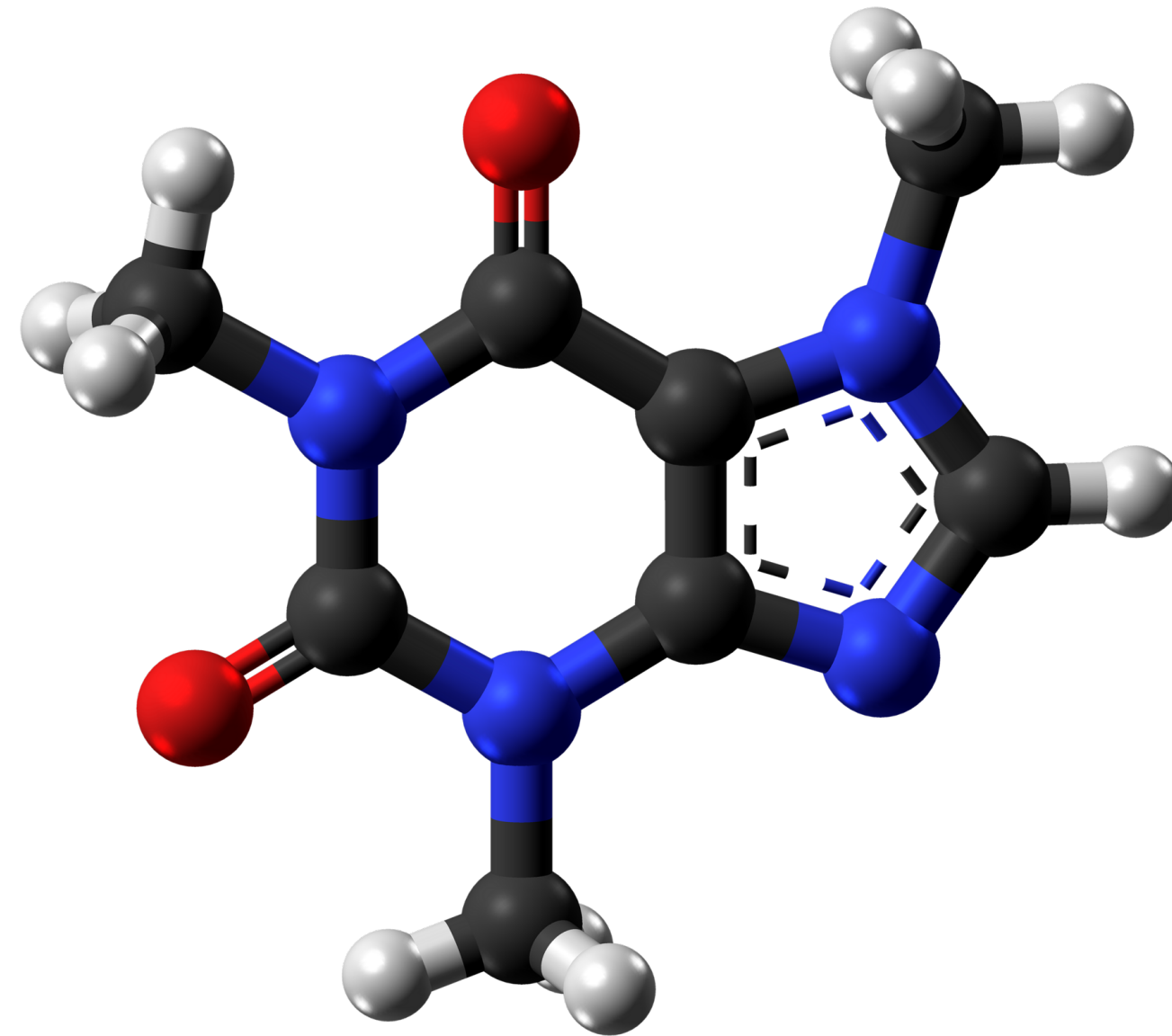
Motivation

Session 3 — Learning on Topological Data

Molecules are graphs

Caffeine molecule

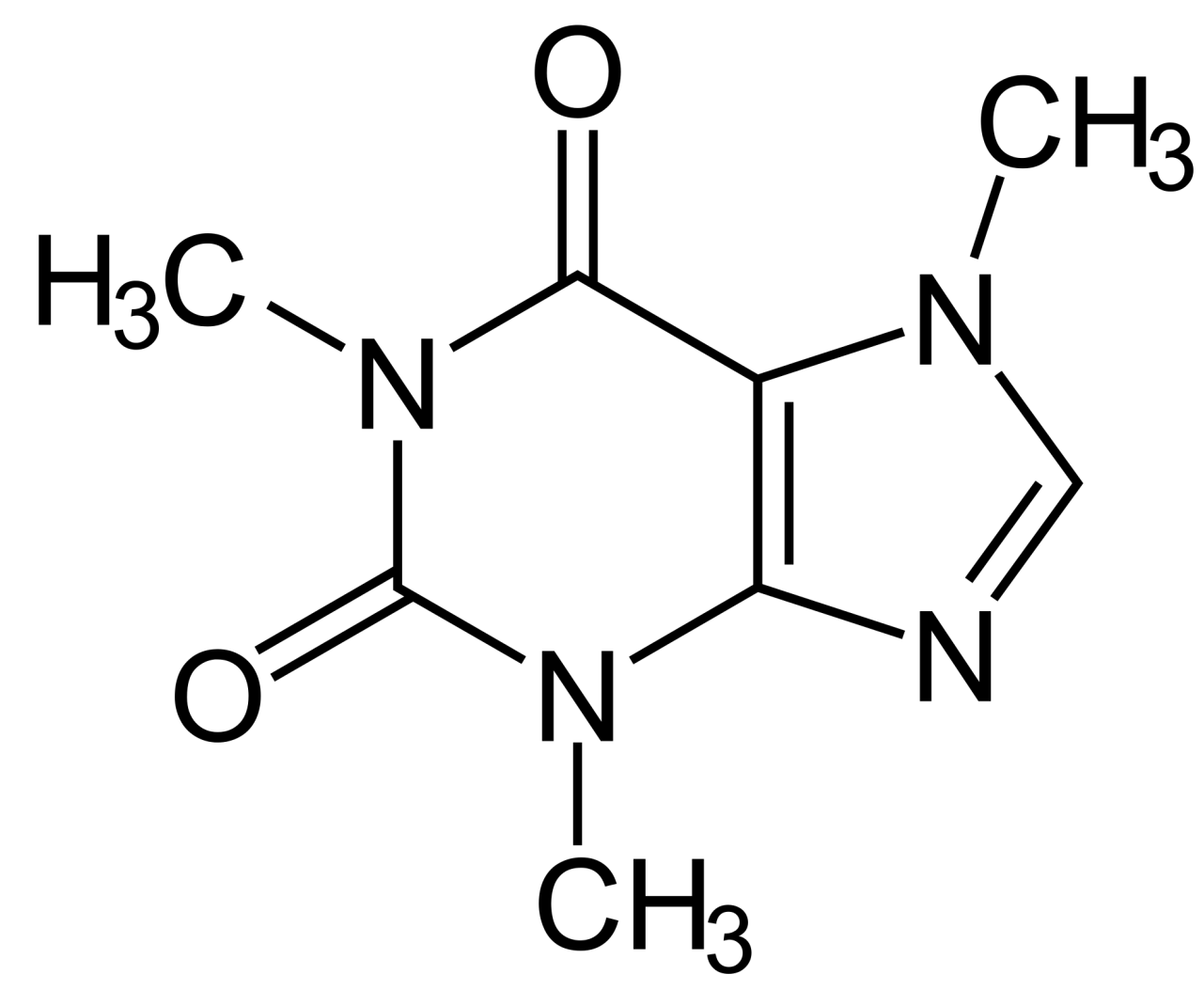
- Nodes given by atoms, edges by bonds
- Features as atom type, charge, bond type



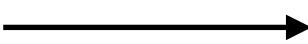
Caffeine molecule (C₈H₁₀N₄O₂)

Molecular classification tasks

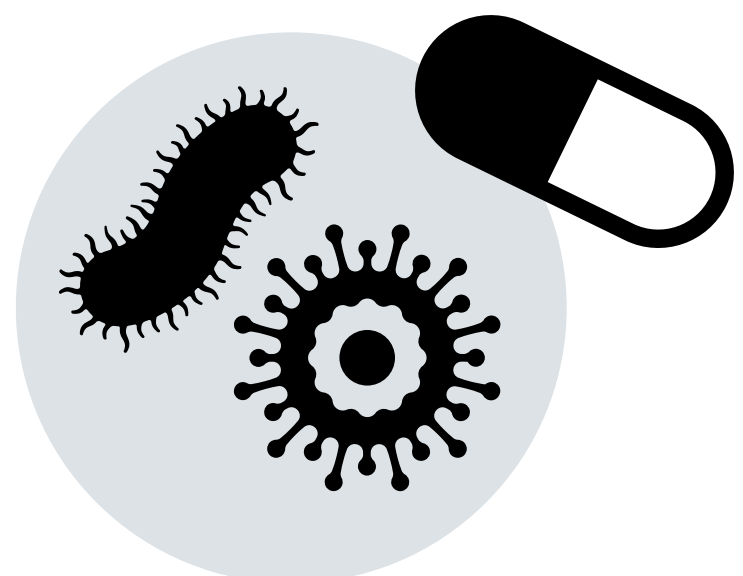
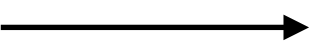
An example: drug discovery



Caffeine molecule (C₈H₁₀N₄O₂)



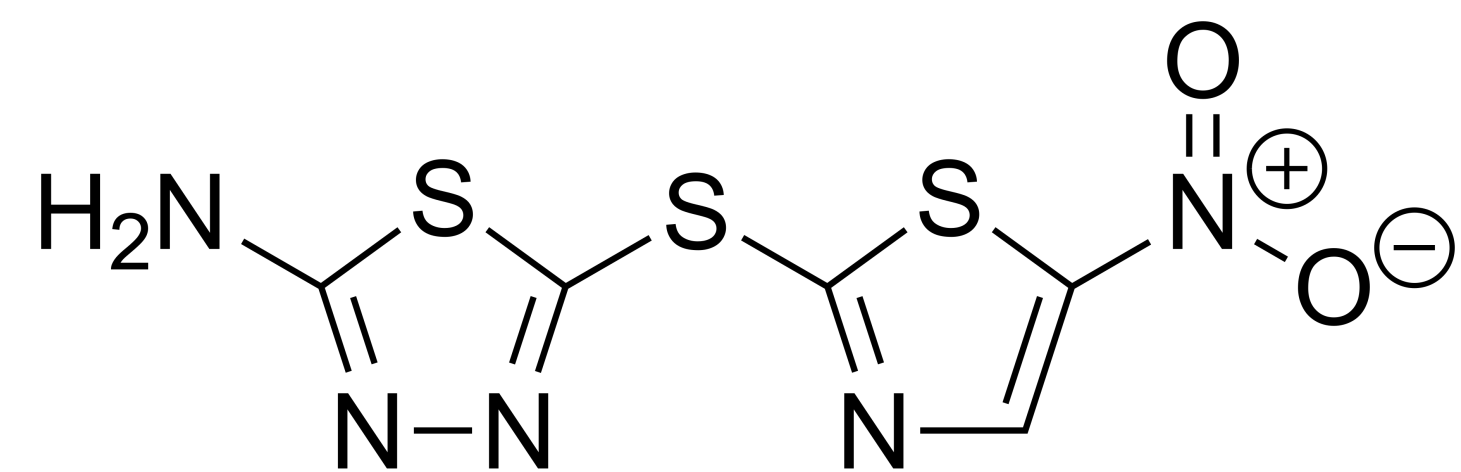
DL model



Does it inhibit E.coli?

Molecular classification tasks

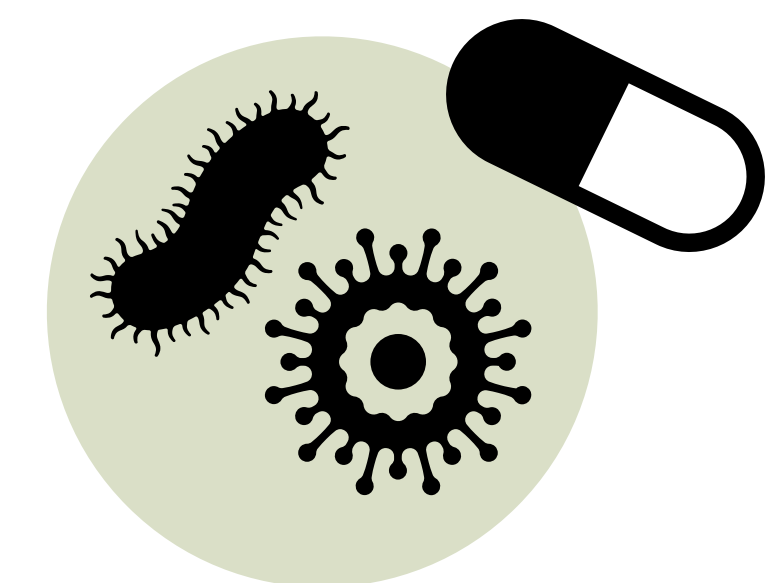
An example: drug discovery



Hallicin



DL model



Does it inhibit E.coli?

Stokes, J. M., et al. (2020). A Deep Learning Approach to Antibiotic Discovery. *Cell*, 180(4), 688-702.e13. <https://doi.org/10.1016/j.cell.2020.01.021>

Transportation networks are graphs

The motorway network in Madrid



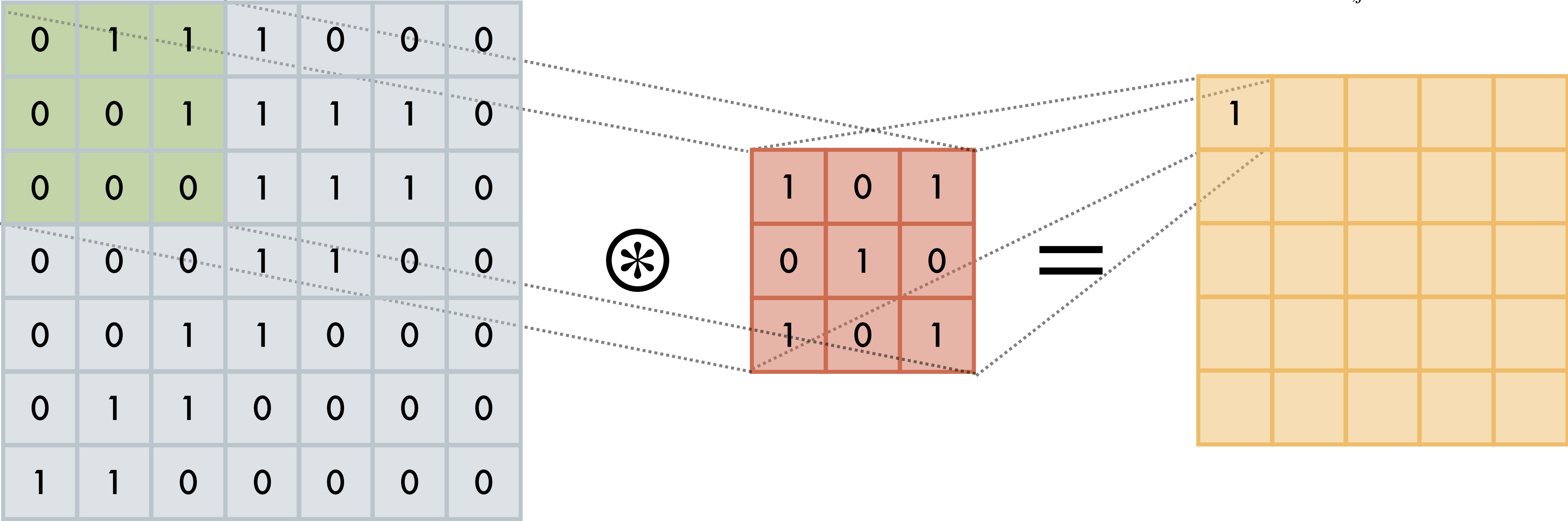
Learning on graphs: graph neural networks (GNNs)

Lecture 3 — Learning on Topological Data

Revisiting CNNs

Locality

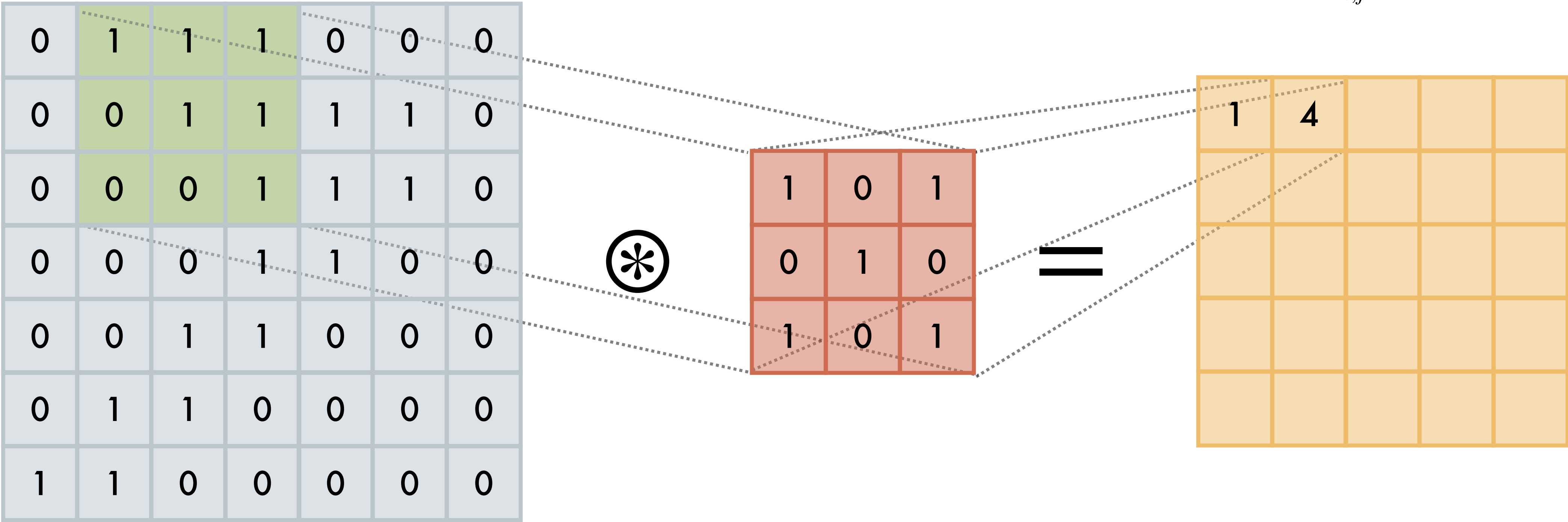
$$(I \circledast K)(x,y) = \sum_{i,j} K(i,j) I(x+i,y+j)$$



Revisiting CNNs

Locality

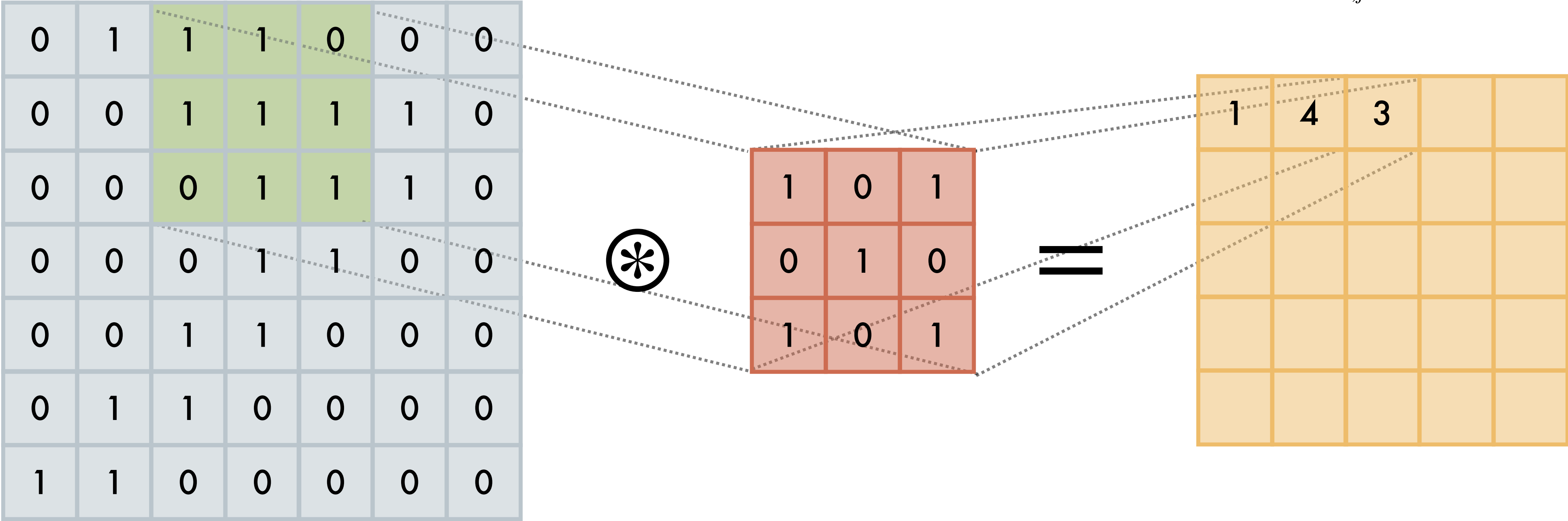
$$(I \circledast K)(x,y) = \sum_{i,j} K(i,j) I(x+i,y+j)$$



Revisiting CNNs

Locality

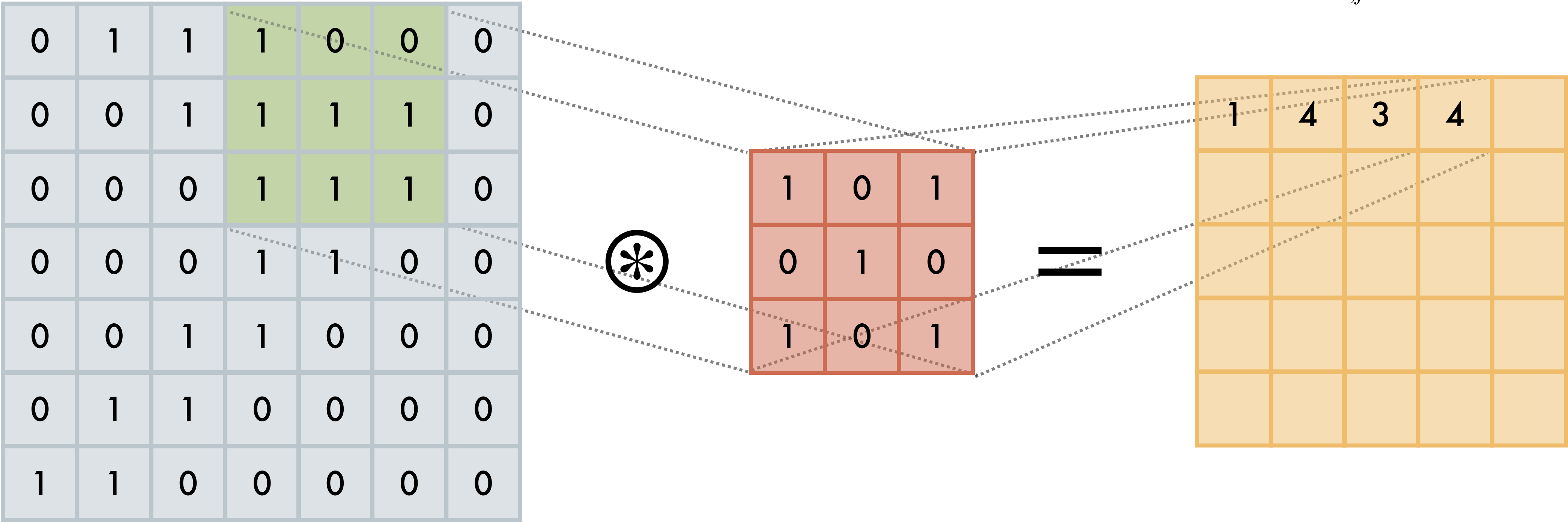
$$(I \circledast K)(x, y) = \sum_{i, j} K(i, j) I(x + i, y + j)$$



Revisiting CNNs

Locality

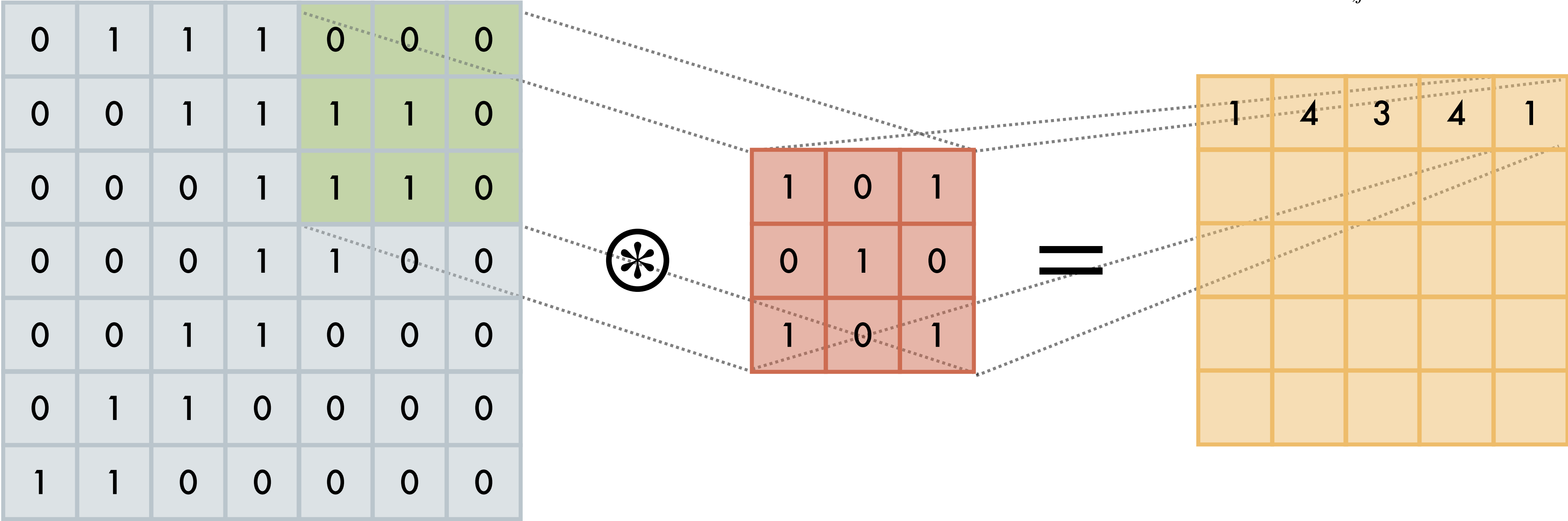
$$(I \circledast K)(x,y) = \sum_{i,j} K(i,j) I(x+i,y+j)$$



Revisiting CNNs

Locality

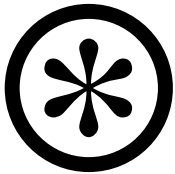
$$(I \circledast K)(x, y) = \sum_{i, j} K(i, j) I(x + i, y + j)$$



Revisiting CNNs

Locality

0	1	1	1	0	0	0
0	0	1	1	1	1	0
0	0	0	1	1	1	0
0	0	0	1	1	0	0
0	0	1	1	0	0	0
0	1	1	0	0	0	0
1	1	0	0	0	0	0



1	0	1
0	1	0
1	0	1

=

1	4	3	4	1
1	2	4	3	3
1	2	3	4	1
1	3	3	1	1
3	3	1	1	0

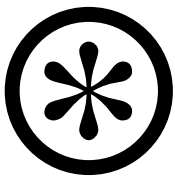
$$(I \circledast K)(x, y) = \sum_{i, j} K(i, j) \, I(x + i, y + j)$$

Revisiting CNNs

Translation equivariance

I

0	1	1	1	0	0	0
0	0	1	1	1	1	0
0	0	0	1	1	1	0
0	0	0	1	1	0	0
0	0	1	1	0	0	0
0	1	1	0	0	0	0
1	1	0	0	0	0	0



K

1	0	1
0	1	0
1	0	1

=

$I \circledast K$

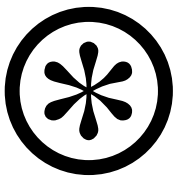
1	4	3	4	1
1	2	4	3	3
1	2	3	4	1
1	3	3	1	1
3	3	1	1	0

Revisiting CNNs

Translation equivariance

$T(I)$

1	1	1	0	0	0	0
0	1	1	1	1	0	0
0	0	1	1	1	0	0
0	0	1	1	0	0	0
0	1	1	0	0	0	0
0	1	0	0	0	0	1
1	0	0	0	0	0	1



K

1	0	1
0	1	0
1	0	1

=

$T(I) \circledast K = T(I \circledast K)$

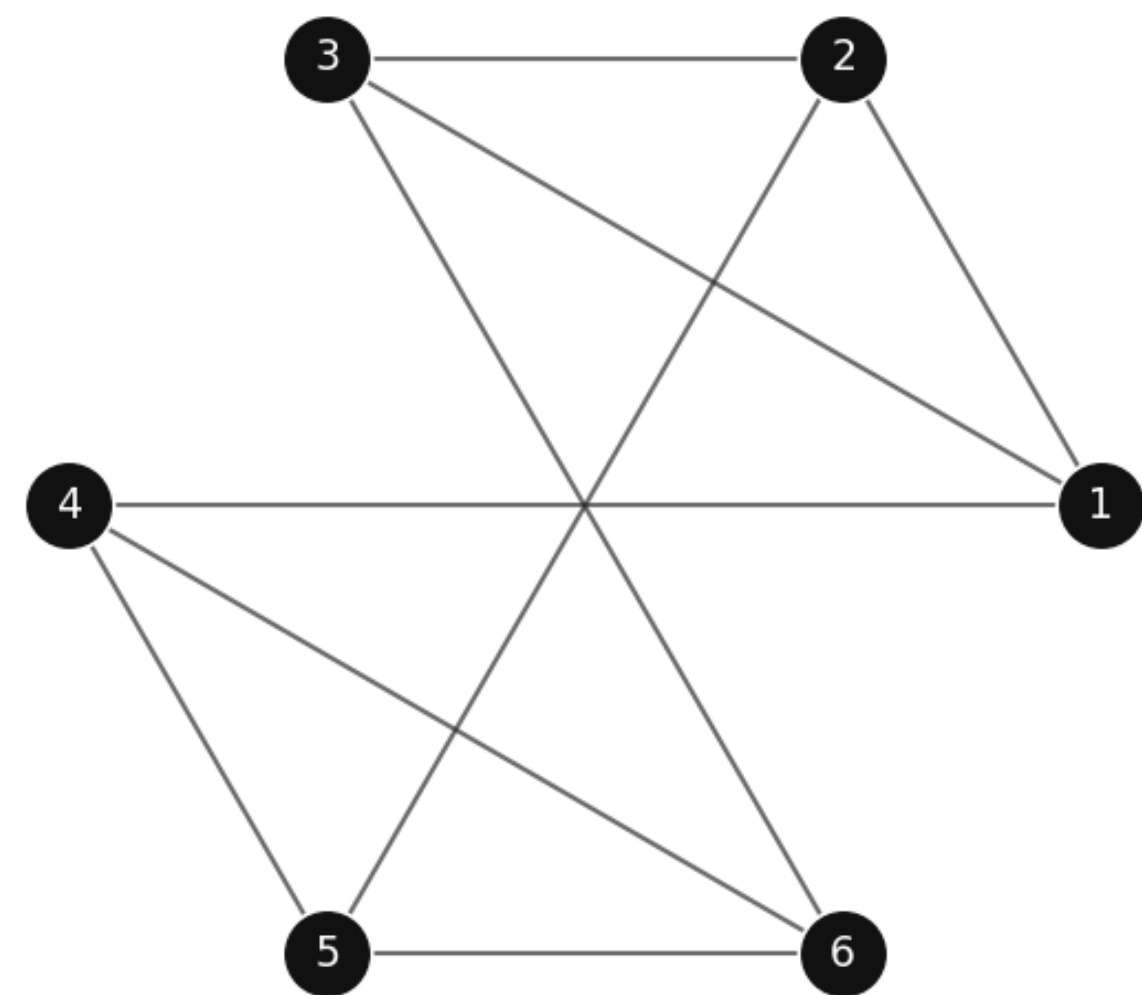
4	3	4	1	1
2	4	3	3	1
2	3	4	1	1
3	3	1	1	1
3	1	1	0	3

Symmetries and invariances

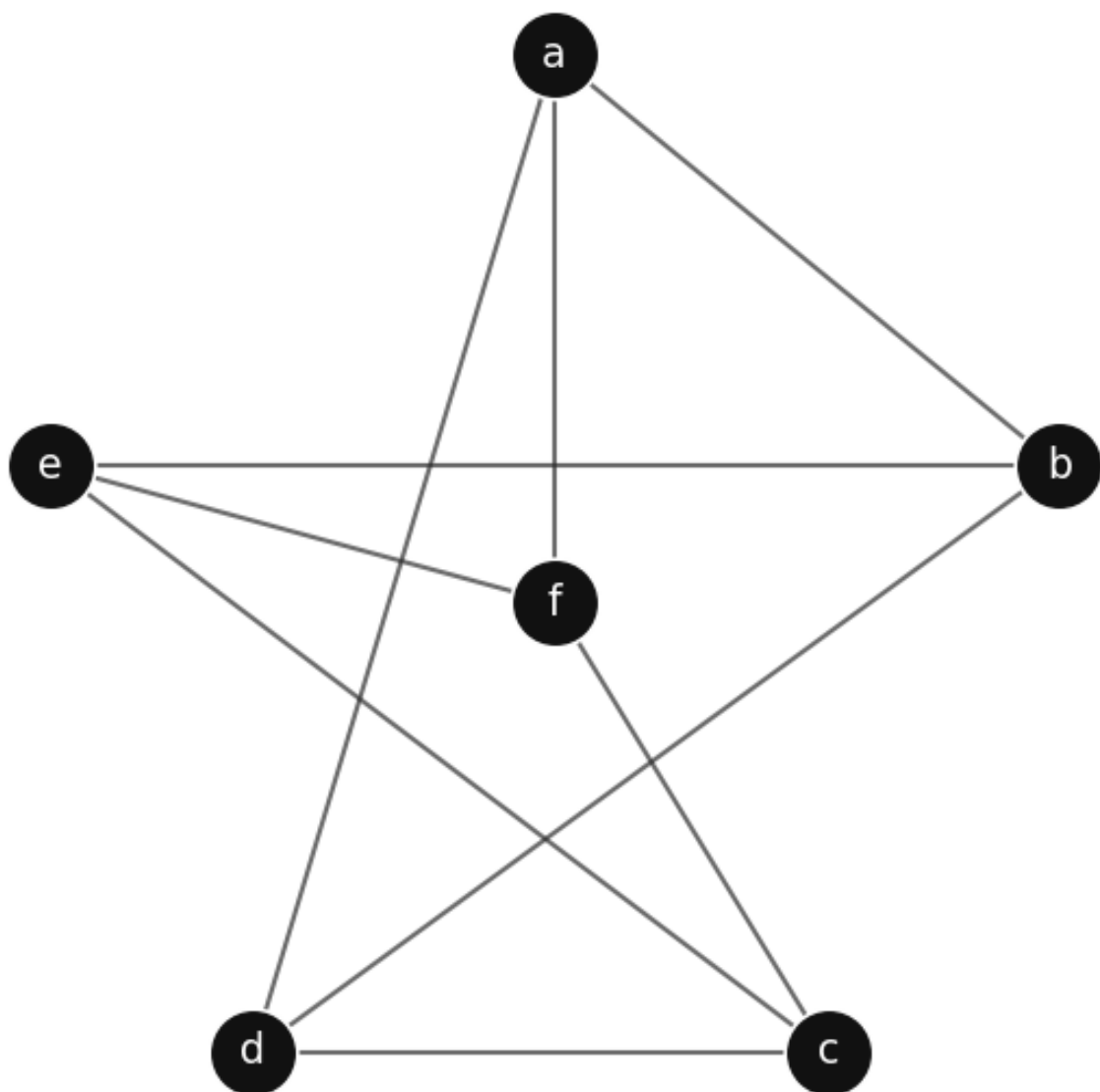
Isomorphism invariance in graphs

What are similar properties in graphs?

$G_1 = (V_1, E_1)$



$G_2 = (V_2, E_2)$



There is also a notion of *locality*: a node is connected only to some of the other nodes, its *neighbors*

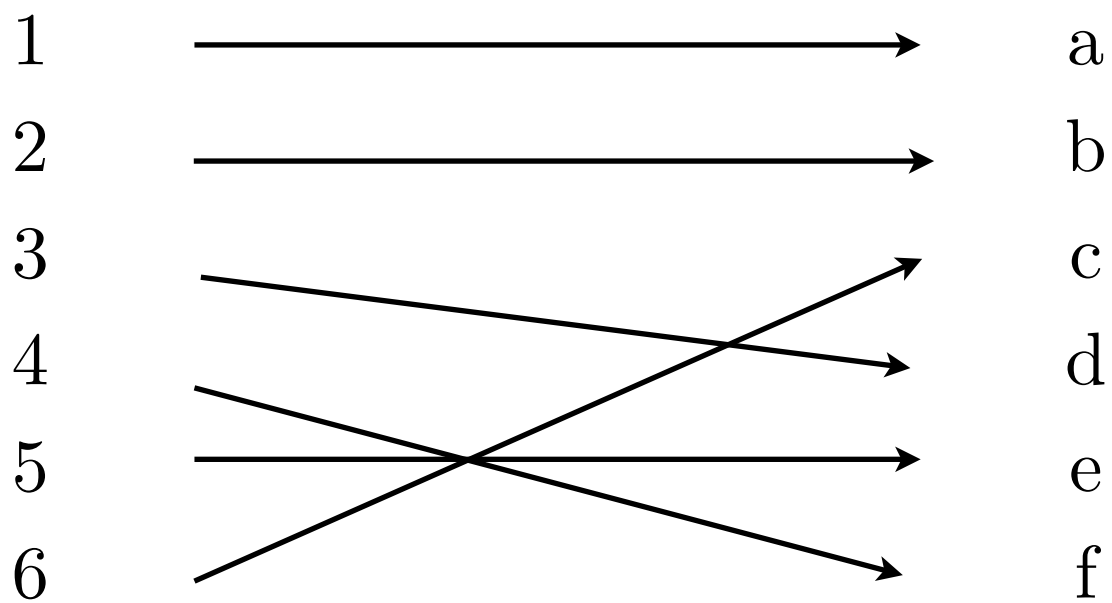
There is also an operation that only changes the labelling of the graph, but not the underlying structure

Def. Graph Isomorphism

Given two graphs $G_1 = (V_1, E_1)$ and $G_2 = (V_2, E_2)$, a graph isomorphism is a map between their vertices

$$\phi : V_1 \rightarrow V_2$$

such that $(i, j) \in E_1 \iff (\phi(i), \phi(j)) \in E_2$

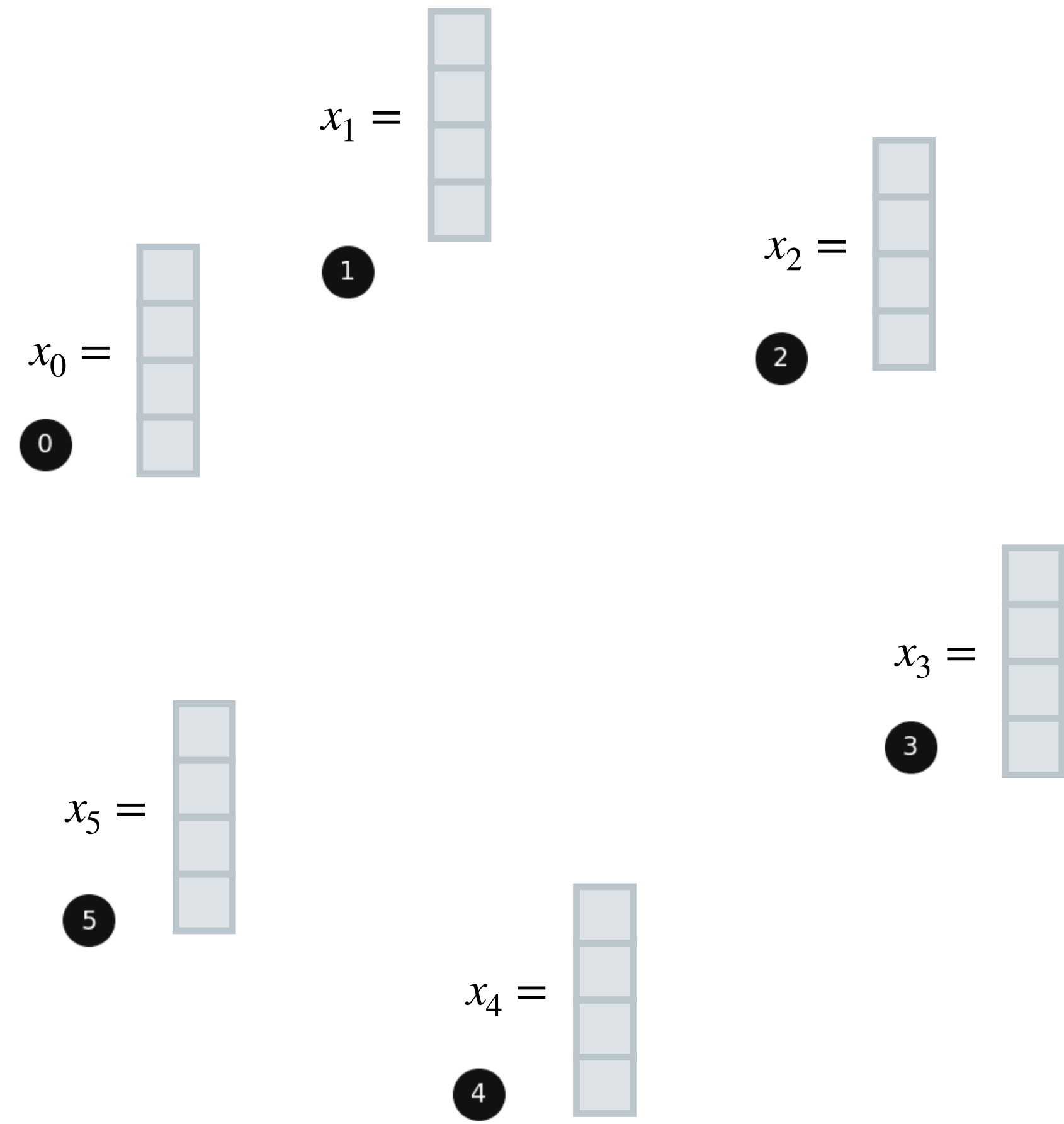


Learning on sets: permutation invariance

Lecture 3 — Learning on Topological Data

Symmetries and invariances

Permutations on nodes



- Start with a set of nodes V and no edges
- Features on the nodes: $x_i \in \mathbb{R}^k$ for $i \in V$
- Stack them: $X = (x_1, \dots, x_n)^\top \in \mathbb{R}^{n \times k}$
 - Already specifies an order in which you are visiting the nodes
 - We want our architectures to not depend on this choice

What transformations change the ordering of the nodes?

The permutation set

Some mathematical background

Def. Permutation

Let S_1 and S_2 be two sets of n elements. A permutation is a *bijective map* between them $P : S_1 \rightarrow S_2$

We call S_n to the set of permutations of sets of n elements

Def. Bijective map

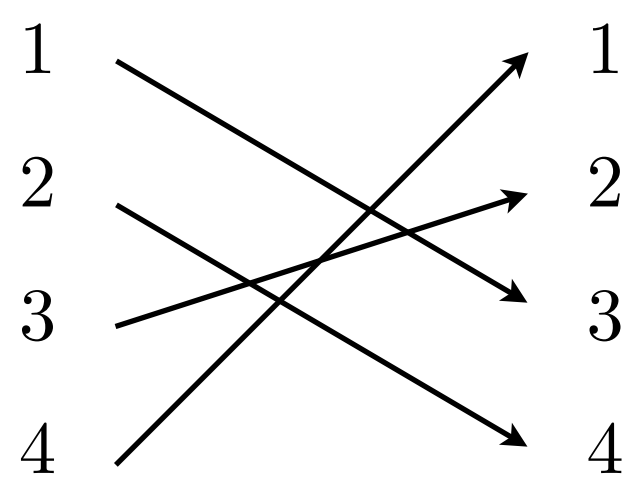
A bijective map between sets $f : S_1 \rightarrow S_2$ is a map that is both:

- *Injective*: $f(x) \neq f(y) \implies x \neq y$
- *Surjective*: $\forall y \in S_2, \exists x \in S_1 \text{ s.t. } f(x) = y$

Representations of permutations: $\sigma \in S_n$

$$\sigma = \begin{pmatrix} 1 & 2 & \dots & n \\ \sigma(1) & \sigma(2) & \dots & \sigma(n) \end{pmatrix}$$

Example:



$$\sigma = \begin{pmatrix} 1 & 2 & 3 & 4 \\ 3 & 4 & 2 & 1 \end{pmatrix} \in S_4$$

Permutation matrix: applied to another matrix $P_\sigma X$ permutes its rows according to σ

$$(P_\sigma)_{ij} = \begin{cases} 1 & \sigma(i) = j \\ 0 & \text{otherwise} \end{cases}$$

$$P_\sigma = \begin{pmatrix} 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 \end{pmatrix}, \quad X = \begin{pmatrix} x_1^\top \\ x_2^\top \\ x_3^\top \\ x_4^\top \end{pmatrix}, \quad P_\sigma X = \begin{pmatrix} x_3^\top \\ x_4^\top \\ x_2^\top \\ x_1^\top \end{pmatrix}$$

Permutation invariance and equivariance

We want: $f_\theta : \mathbb{R}^{n \times k} \rightarrow \mathbb{R}^p, \quad X \mapsto f_\theta(X)$

Graph level tasks:

f_θ is **permutation invariant** if for all permutation matrices P

$$f_\theta(PX) = f_\theta(X)$$

Example: DeepSets¹

$$f(X) = \varphi \left(\sum_{i \in V} \phi(\mathbf{x}_i) \right)$$

$$\phi : \mathbb{R}^k \rightarrow \mathbb{R}^m$$

$$\varphi : \mathbb{R}^m \rightarrow \mathbb{R}^p$$

Node level tasks:

f_θ is **permutation equivariant** if for all permutation matrices P

$$f_\theta(PX) = Pf_\theta(X)$$

You could change the sum to any permutation invariant aggregator $\oplus$ (sum, avg, max)

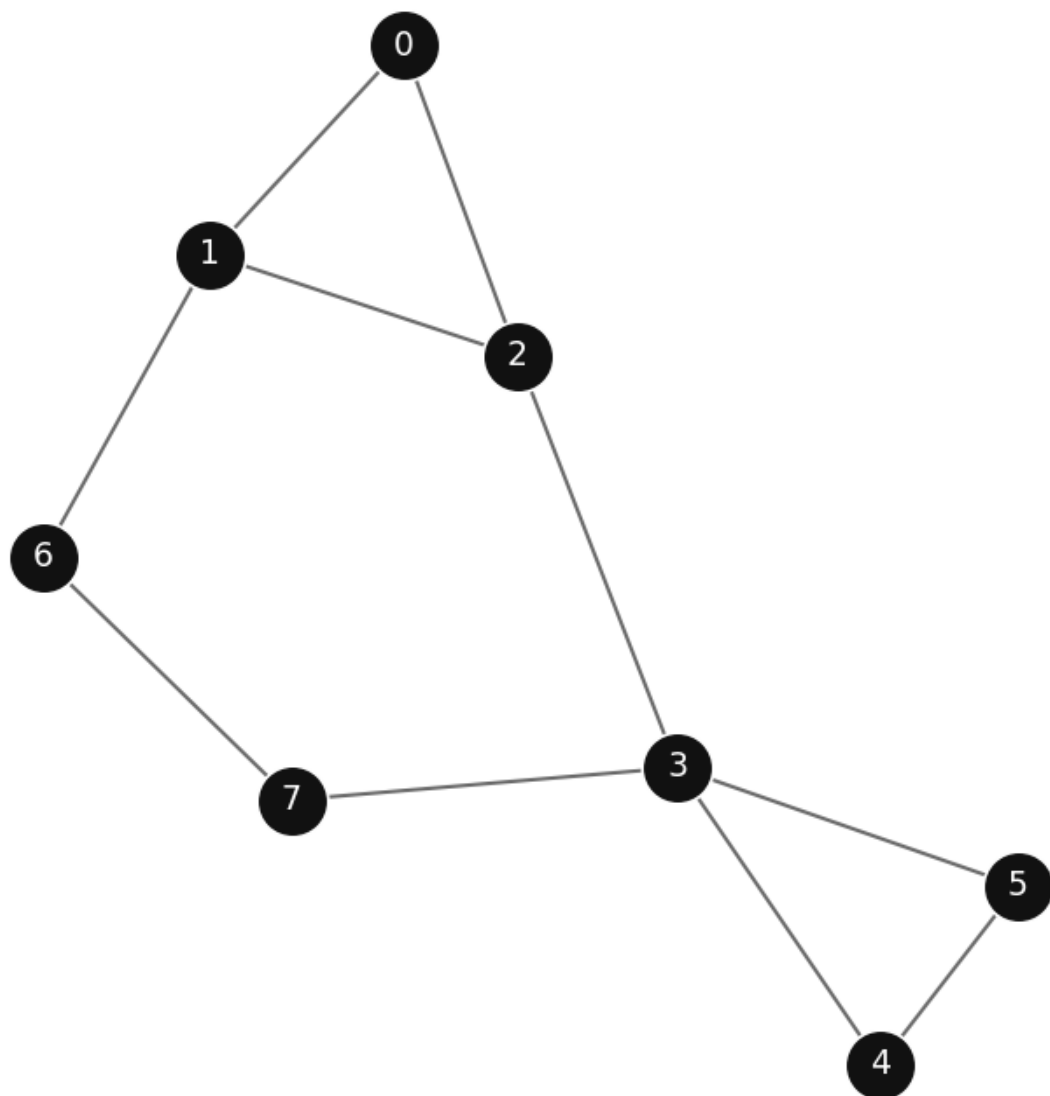
¹Zaheer, M., Kottur, S., Ravanbakhsh, S., Poczos, B., Salakhutdinov, R. R., & Smola, A. (2017). Deep Sets. *Advances in Neural Information Processing Systems*, 30.

Learning on graphs: message-passing

Lecture 3 — Learning on Topological Data I

Including edges into the model

Including the adjacency matrix in the model



$$A \in \mathbb{R}^{n \times n}, \quad A_{ij} = \begin{cases} 1, & \text{if } (i,j) \in E \\ 0, & \text{otherwise} \end{cases}$$

$$A = \begin{bmatrix} 0 & 1 & 1 & 0 & 0 & 0 & 0 & 0 \\ 1 & 0 & 1 & 0 & 0 & 0 & 1 & 0 \\ 1 & 1 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 1 & 1 & 0 & 1 \\ 0 & 0 & 0 & 1 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 & 0 & 0 & 1 & 0 \end{bmatrix}$$

Now we want to produce a function that also takes into account node features and edges:

$$f_{\theta}(X, A)$$

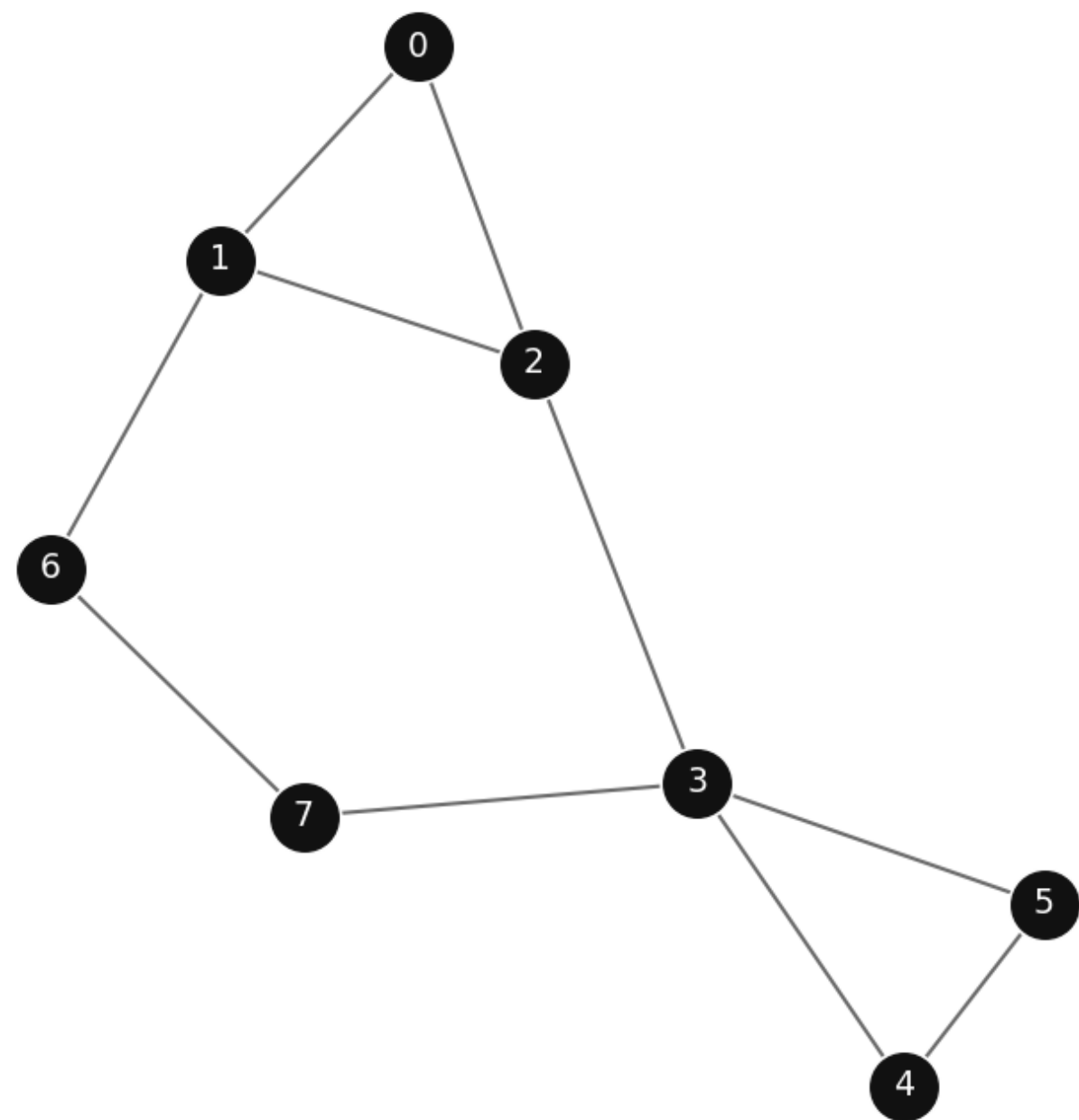
Note: we are not considering features over the edges, but this could be done

If we permute the nodes, the edges are permuted accordingly. This is reflected by transforming the adjacency matrix as $A \mapsto PAP^{\top}$

- **Permutation invariance:** $f_{\theta}(PX, PAP^{\top}) = f_{\theta}(X, A)$
- **Permutation equivariance:** $f_{\theta}(PX, PAP^{\top}) = P f_{\theta}(X, A)$

Locality on graphs

Neighborhoods



Neighborhood: $N_i = \{ j \in V \mid (i, j) \in E \} = \{ j \in V \mid A_{ij} = 1 \}$

Multiset of features in the neighborhood: $X_{N_i} = \{ \{ x_j : j \in N_i \} \}$

Local function: a function on the node features depending only on the features of the neighbors

$$g(x_i, X_{N_i})$$

Similar to the aggregation rule in a CNN from a kernel

A recipe for GNNs

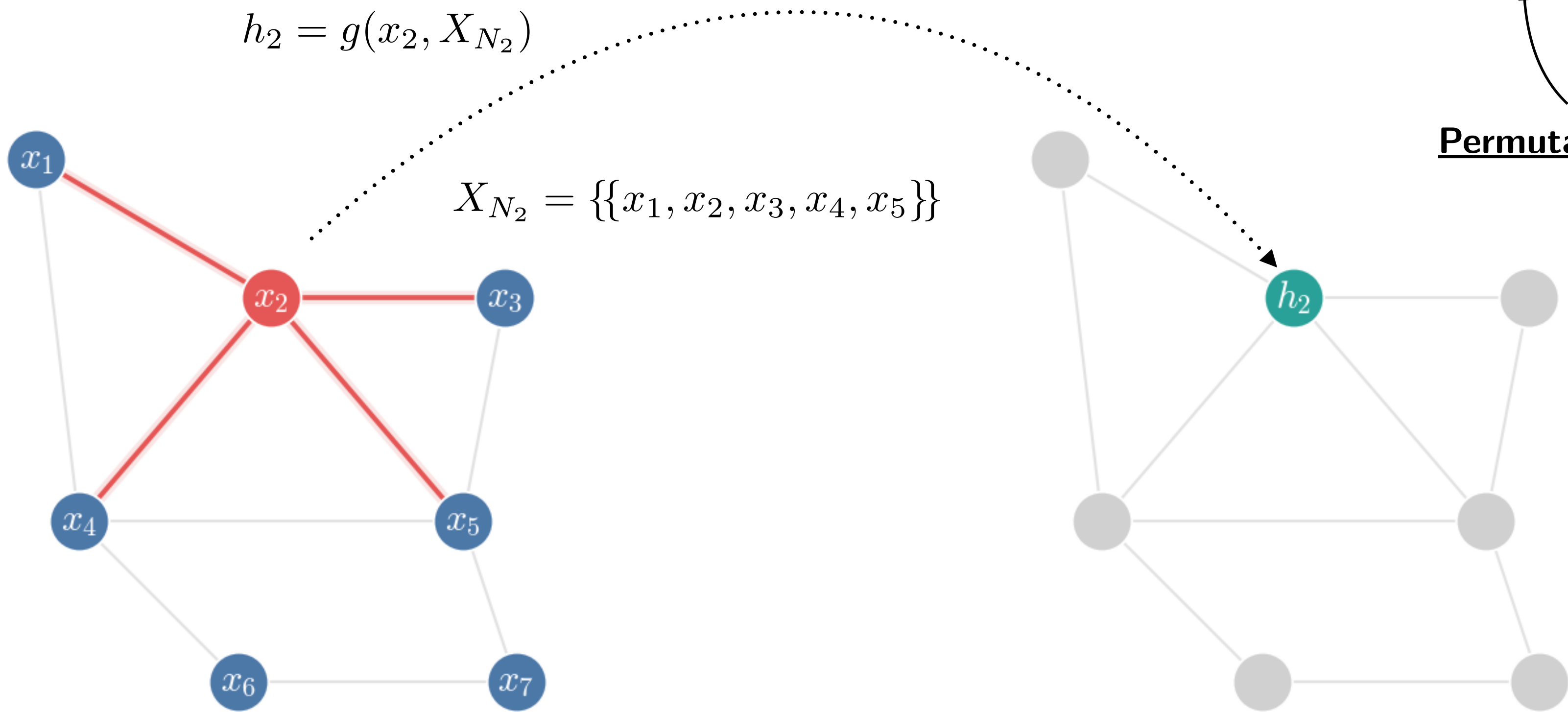
Putting everything together

A local function g is applied to all nodes and neighborhoods
 g should not depend on the order of vertices in X_{N_i}

Permutation equivariant

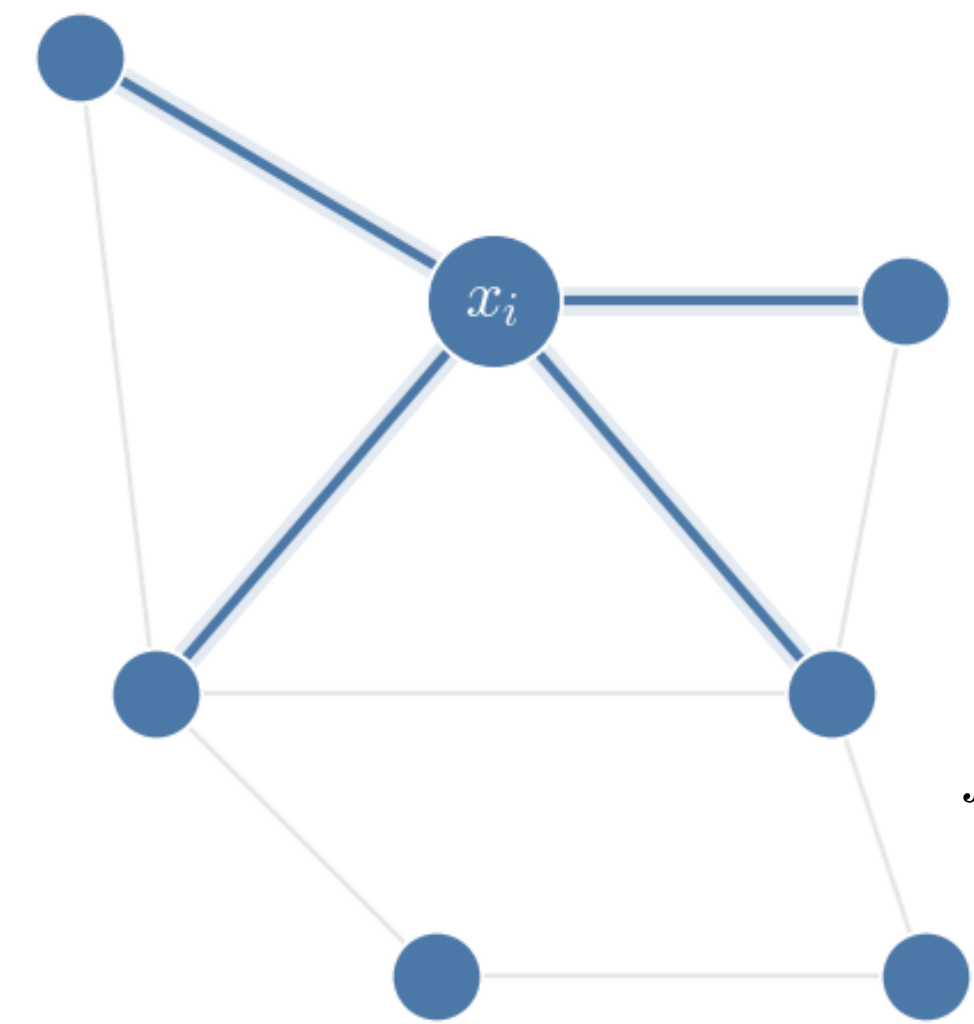
$$f(X, A) = \begin{bmatrix} g(x_1, X_{N_1}) \\ g(x_2, X_{N_2}) \\ \vdots \\ g(x_n, X_{N_n}) \end{bmatrix}$$

Permutation invariant over X_{N_i}



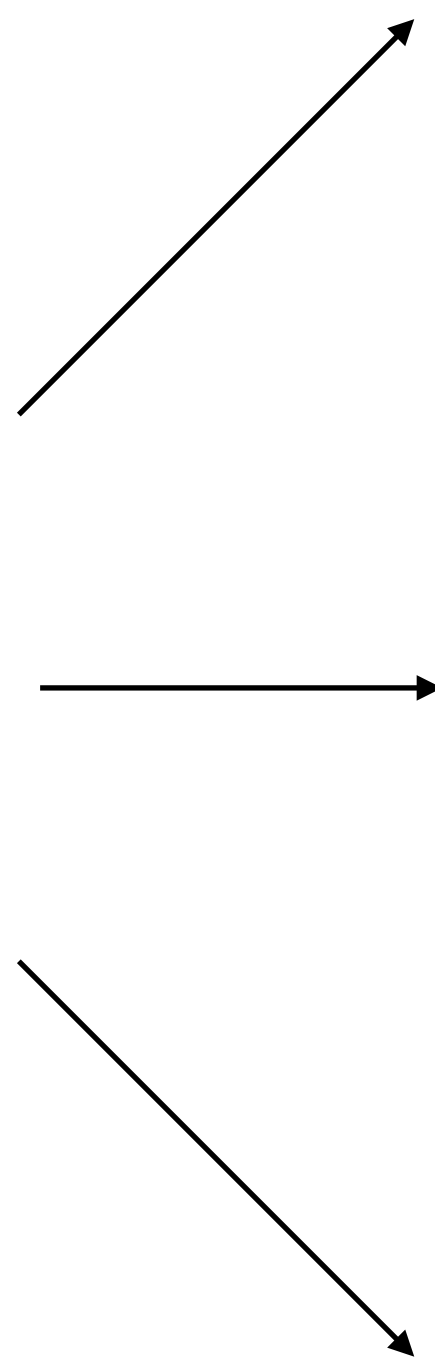
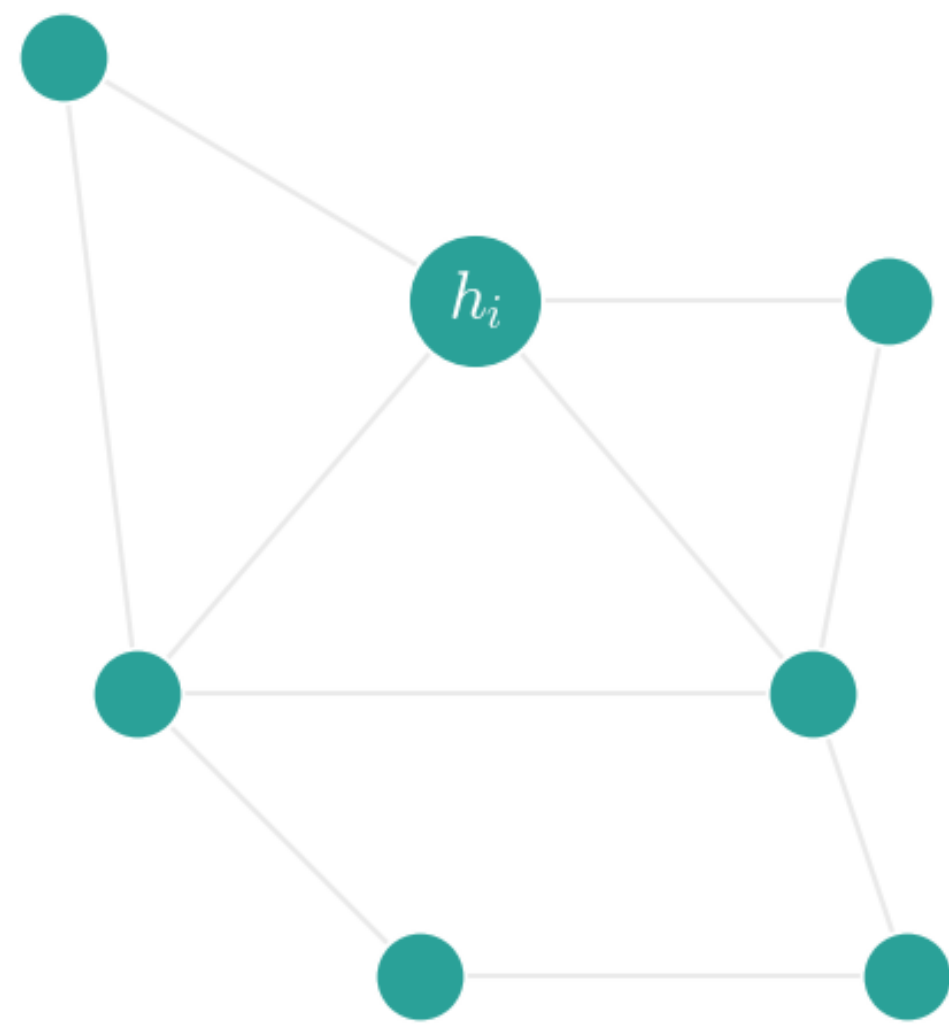
A recipe for GNNs

Solving different tasks



GNN layer
→

$$f(X, A) = \begin{bmatrix} g(x_1, X_{N_1}) \\ g(x_2, X_{N_2}) \\ \vdots \\ g(x_n, X_{N_n}) \end{bmatrix}$$



Node classification:

$$z_i = f(h_i)$$

Graph classification:

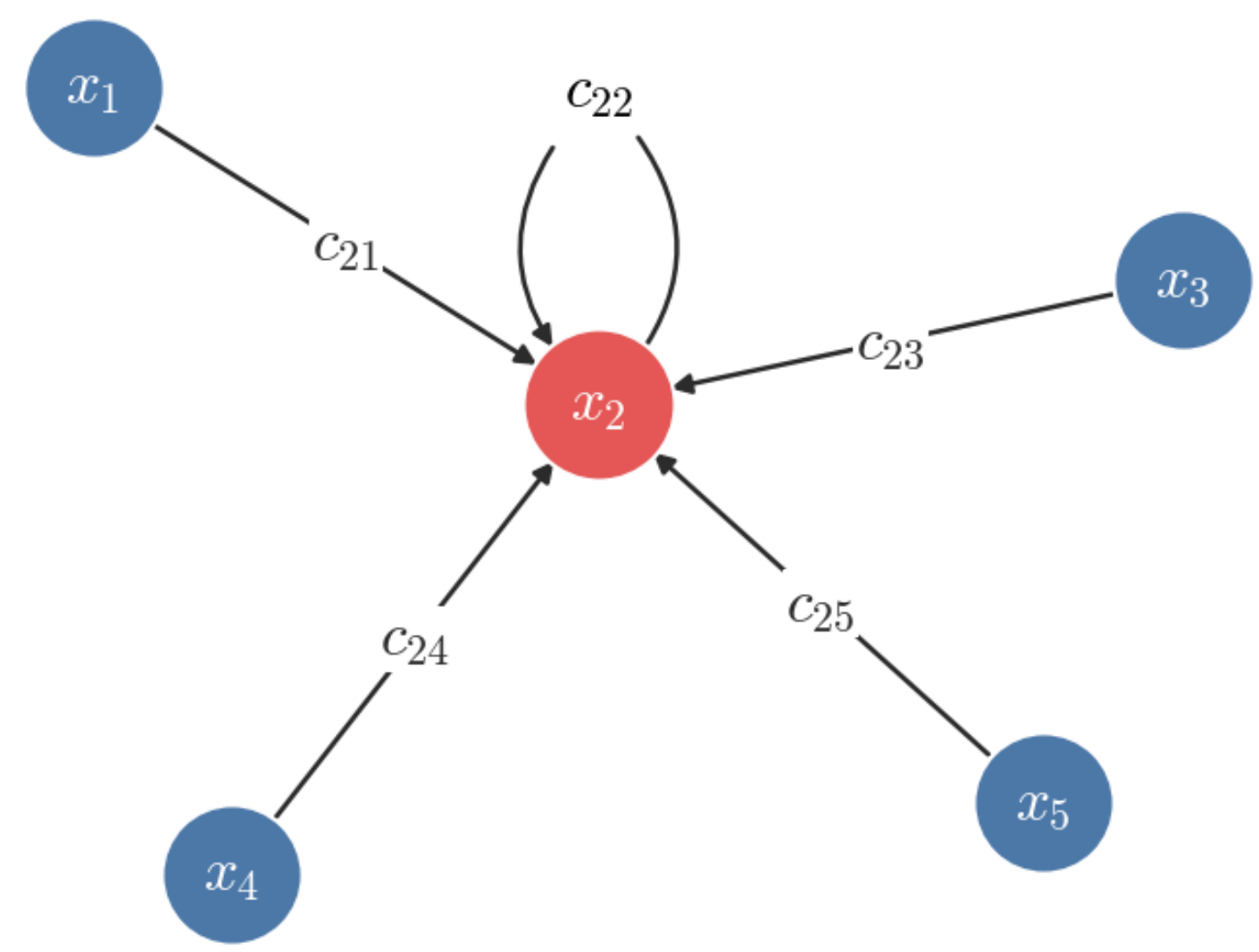
$$z_G = f\left(\bigoplus_{i \in V} h_i\right)$$

Link prediction:

$$z_{ij} = f(h_i, h_j, e_{ij})$$

Message-passing

Instantiating local functions



Convolutional

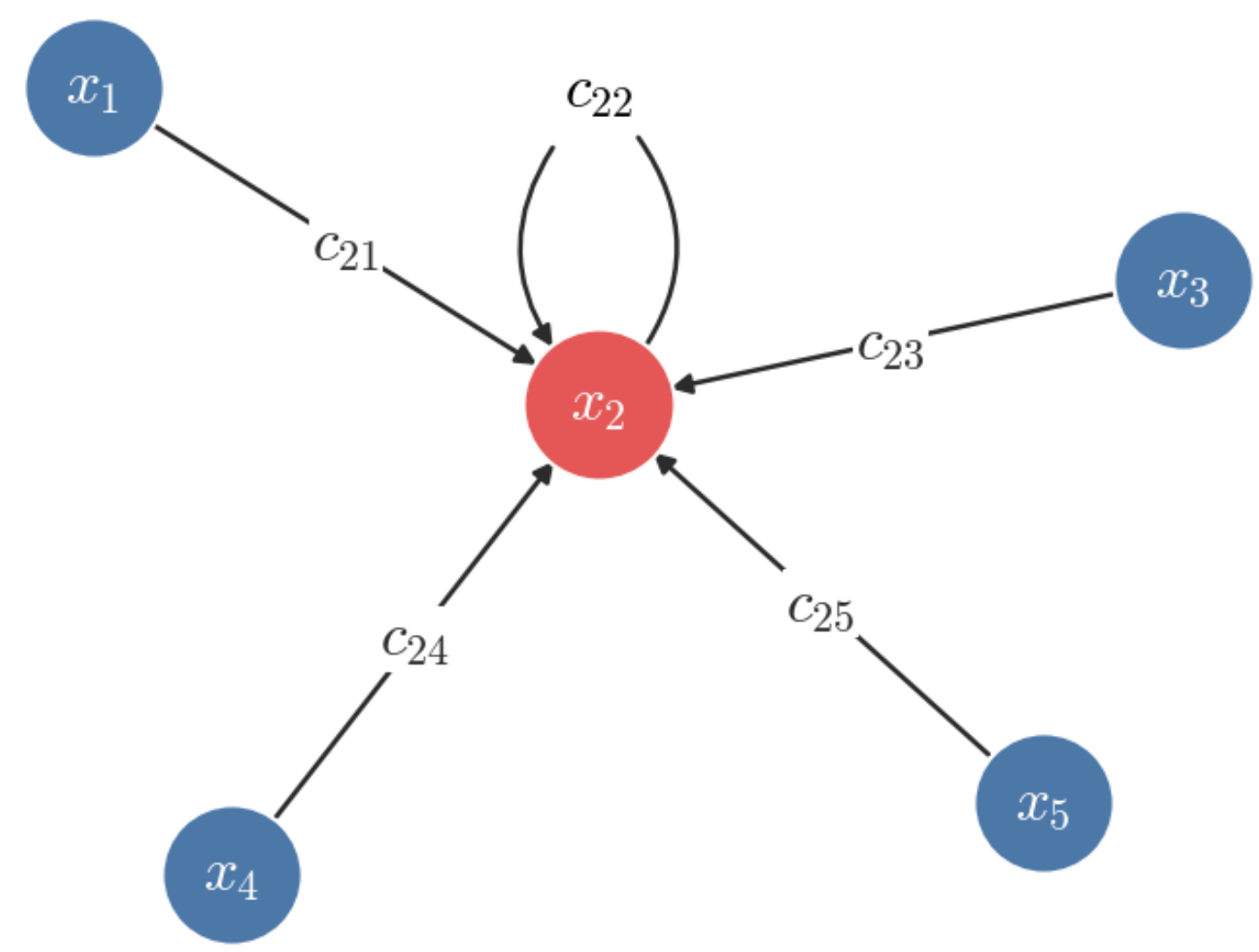
$$h_i = \phi \left(x_i, \bigoplus_{j \in \mathcal{N}_i} c_{ij} \psi(x_j) \right)$$

Defferrard, M., Bresson, X., and Vandergheynst, P. (2016). Convolutional Neural Networks on Graphs with Fast Localized Spectral Filtering. *Advances in Neural Information Processing Systems*, 29.

Kipf, Thomas N., and Max Welling. Semi-Supervised Classification with Graph Convolutional Networks. *International Conference on Learning Representations* (2017).

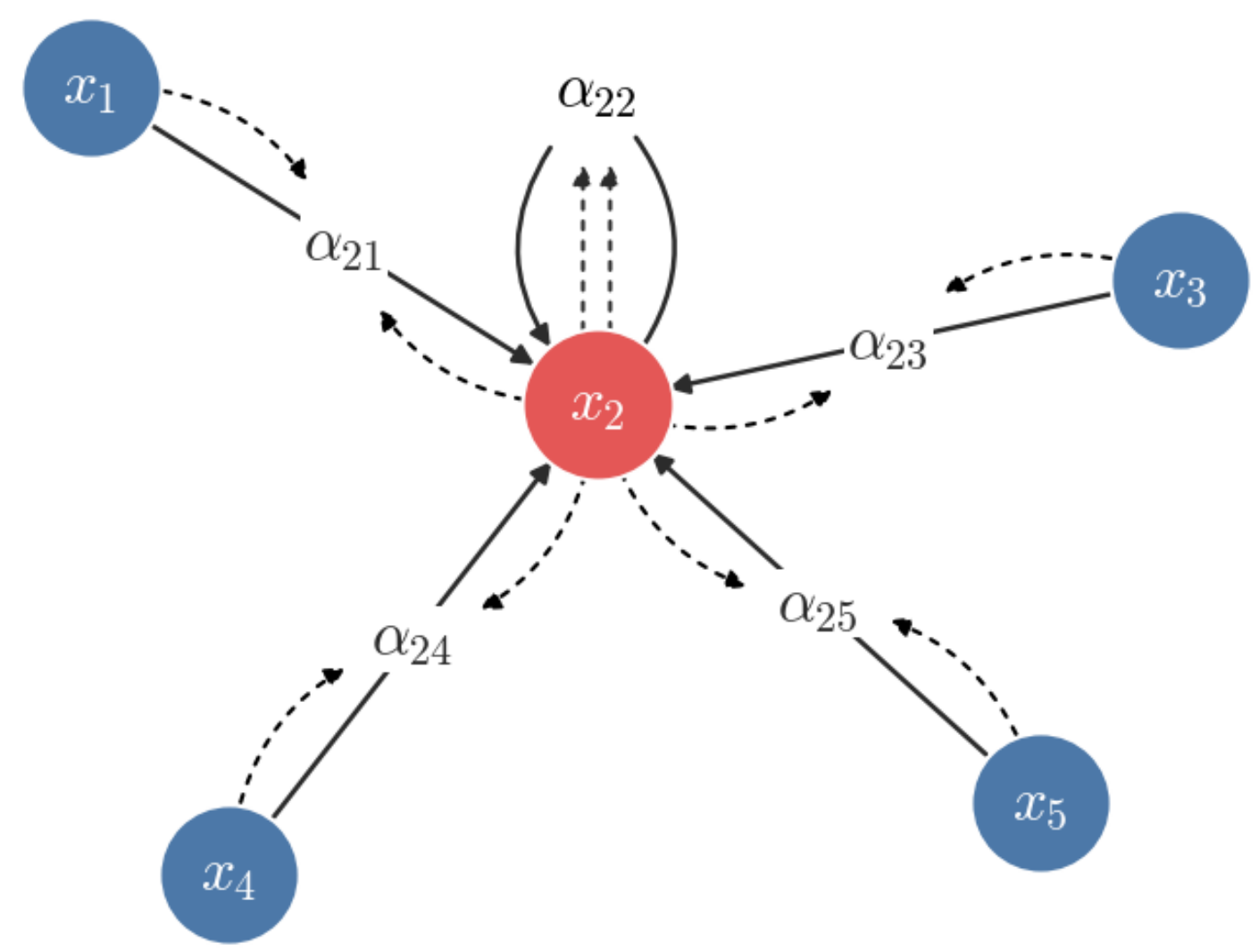
Message-passing

Instantiating local functions



Convolutional

$$h_i = \phi \left(x_i, \bigoplus_{j \in \mathcal{N}_i} c_{ij} \psi(x_j) \right)$$



Attentional

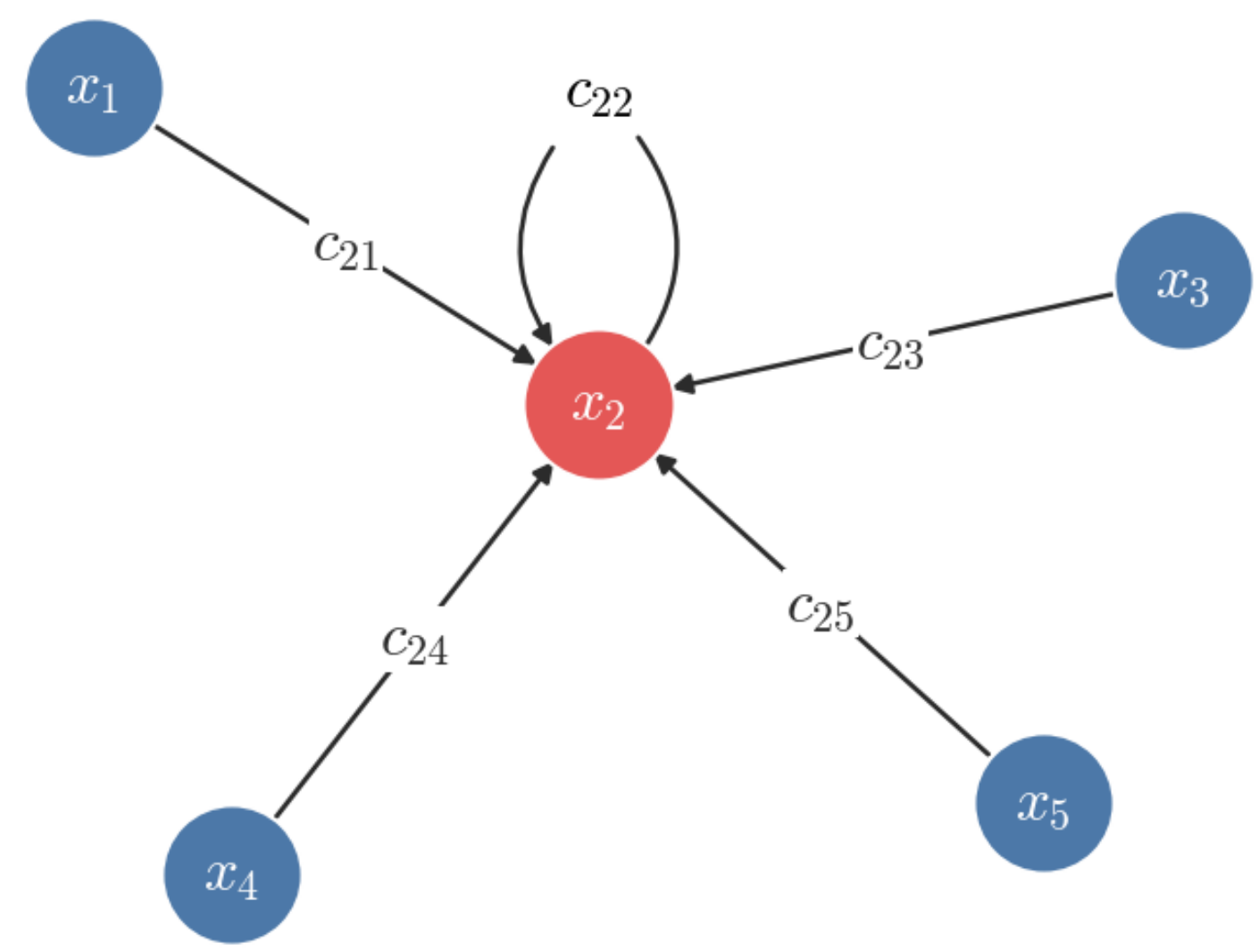
$$h_i = \phi \left(x_i, \bigoplus_{j \in \mathcal{N}_i} \alpha(x_i, x_j) \psi(x_j) \right)$$

Veličković, Petar, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Liò, and Yoshua Bengio. Graph Attention Networks. *International Conference on Learning Representations* (2018).

Monti, Federico, Davide Boscaini, Jonathan Masci, Emanuele Rodola, Jan Svoboda, and Michael M. Bronstein. Geometric Deep Learning on Graphs and Manifolds Using Mixture Model CNNs. *IEEE Conference on Computer Vision and Pattern Recognition* (2017), 5115–24.

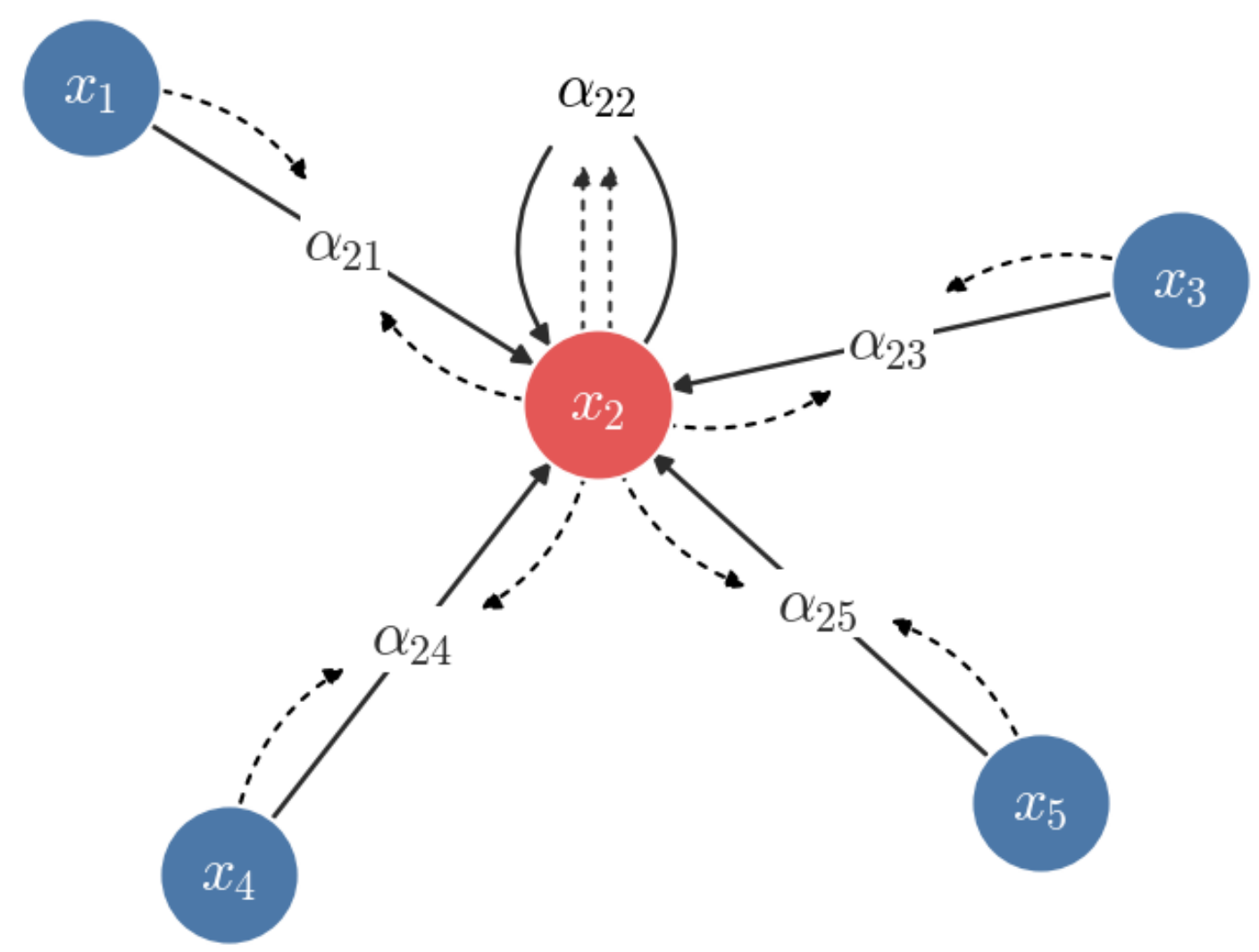
Message-passing

Instantiating local functions



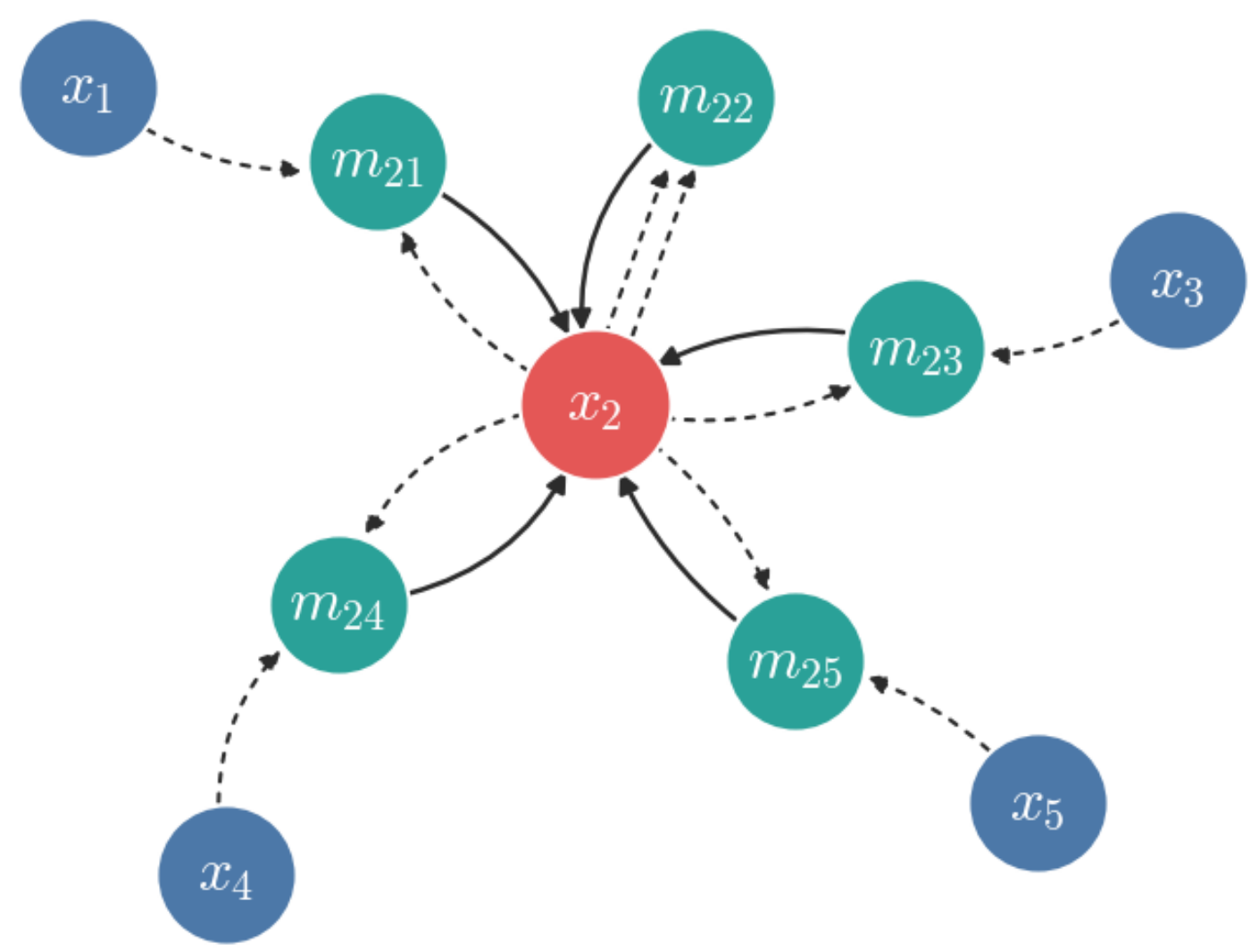
Convolutional

$$h_i = \phi \left(x_i, \bigoplus_{j \in \mathcal{N}_i} c_{ij} \psi(x_j) \right)$$



Attentional

$$h_i = \phi \left(x_i, \bigoplus_{j \in \mathcal{N}_i} \alpha(x_i, x_j) \psi(x_j) \right)$$



Message-passing

$$h_i = \phi \left(x_i, \bigoplus_{j \in \mathcal{N}_i} \psi(x_i, x_j) \right)$$

Gilmer, Justin, Samuel S. Schoenholz, Patrick F. Riley, Oriol Vinyals, and George E. Dahl. Neural Message Passing for Quantum Chemistry. *Proceedings of the 34th International Conference on Machine Learning*, July 17, 2017, 1263–72.

Battaglia, Peter W., Jessica B. Hamrick, Victor Bapst, et al. Relational Inductive Biases, Deep Learning, and Graph Networks. Battaglia, Peter W., Jessica B. Hamrick, Victor Bapst, et al. “Relational Inductive Biases, Deep Learning, and Graph Networks.” arXiv:1806.01261. Preprint, arXiv, October 17, 2018.

MPNNs in practice

Useful libraries to train some GNNs



PyTorch Geometric (PyG)

(pyg.org)

- Built over PyTorch
- Largest collection of implemented GNN architectures (GCN, GAT, GIN, GraphSAGE, and 50+ more)
- Includes common benchmark datasets to test your code



NetworkX

(networkx.org)

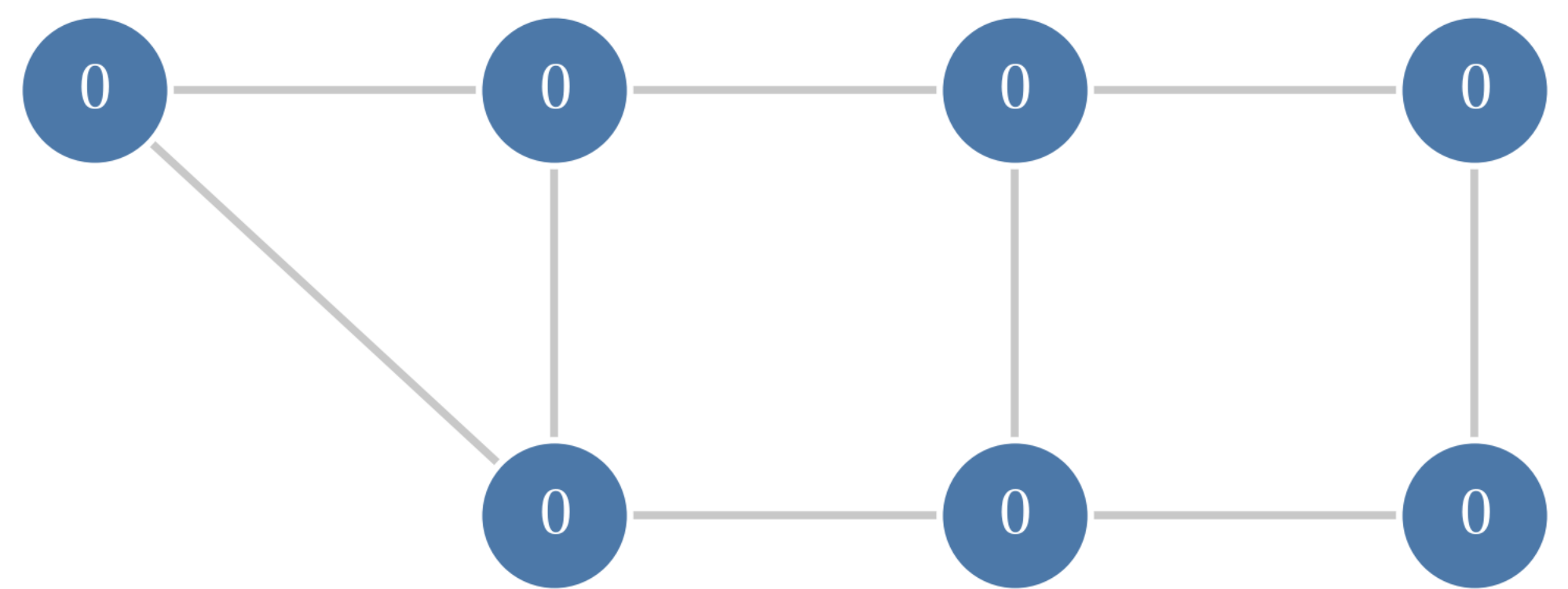
- The standard library for graph analysis and visualization
- Integrates with PyG for graph construction

Expressivity: graph isomorphism testing

Lecture 3 — Learning on Topological Data

Weisfeiler—Leman test

An algorithm for testing graph isomorphism



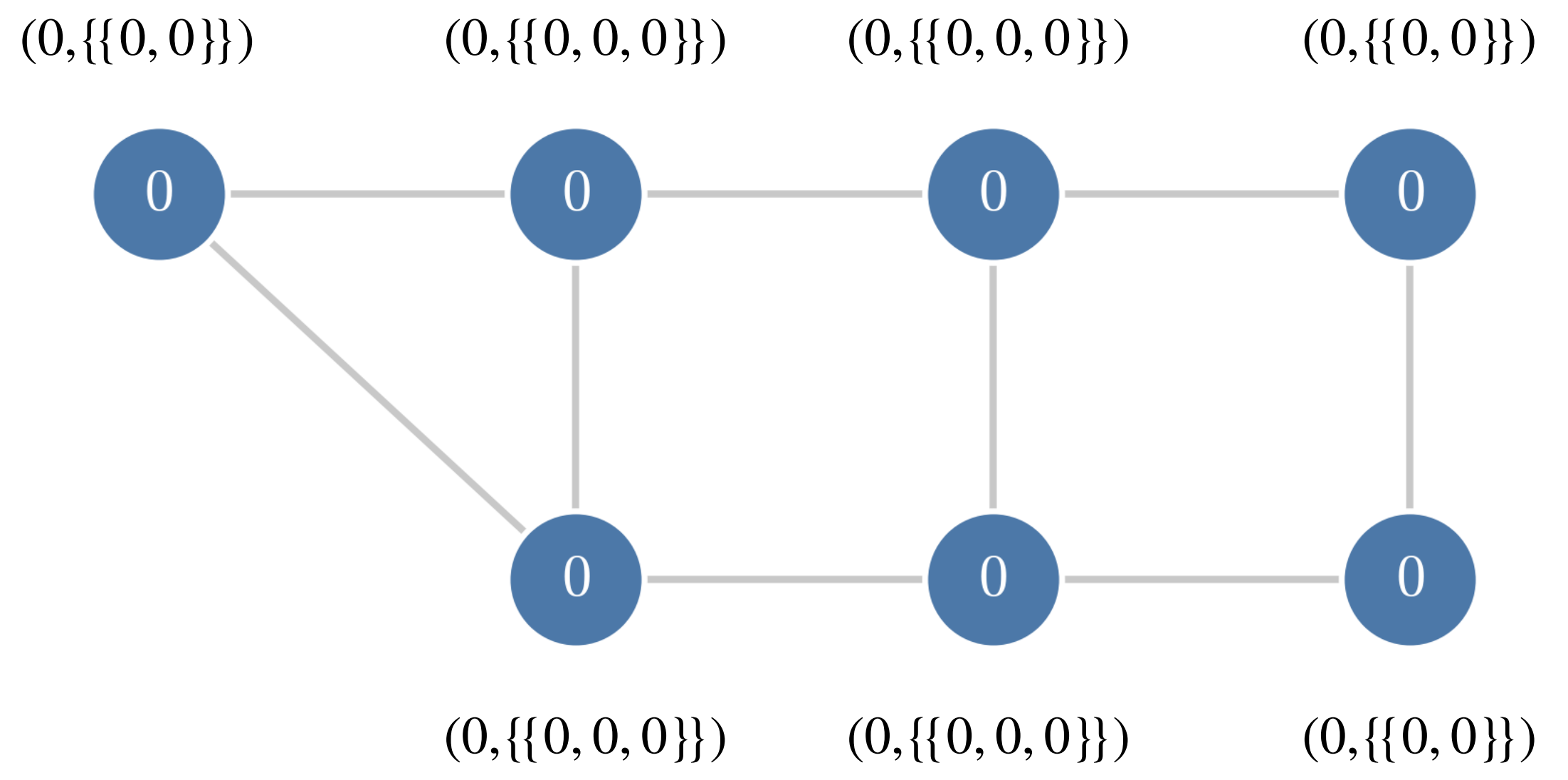
Initialization:

Assign the same label (or color) to all vertices in the graph

Weisfeiler, B. & Leman, A. (1968). The reduction of a graph to canonical form and the algebra which appears therein. *Nauchno-Technicheskaya Informatsia*, Series 2, 9, pp. 12–16.

Weisfeiler—Leman test

An algorithm for testing graph isomorphism



$$c_i^{(t)} = \text{HASH}\left(c_i^{(t-1)}, \{\{c_j^{(t-1)} : j \in \mathcal{N}(i)\}\}\right)$$

Iterative step:

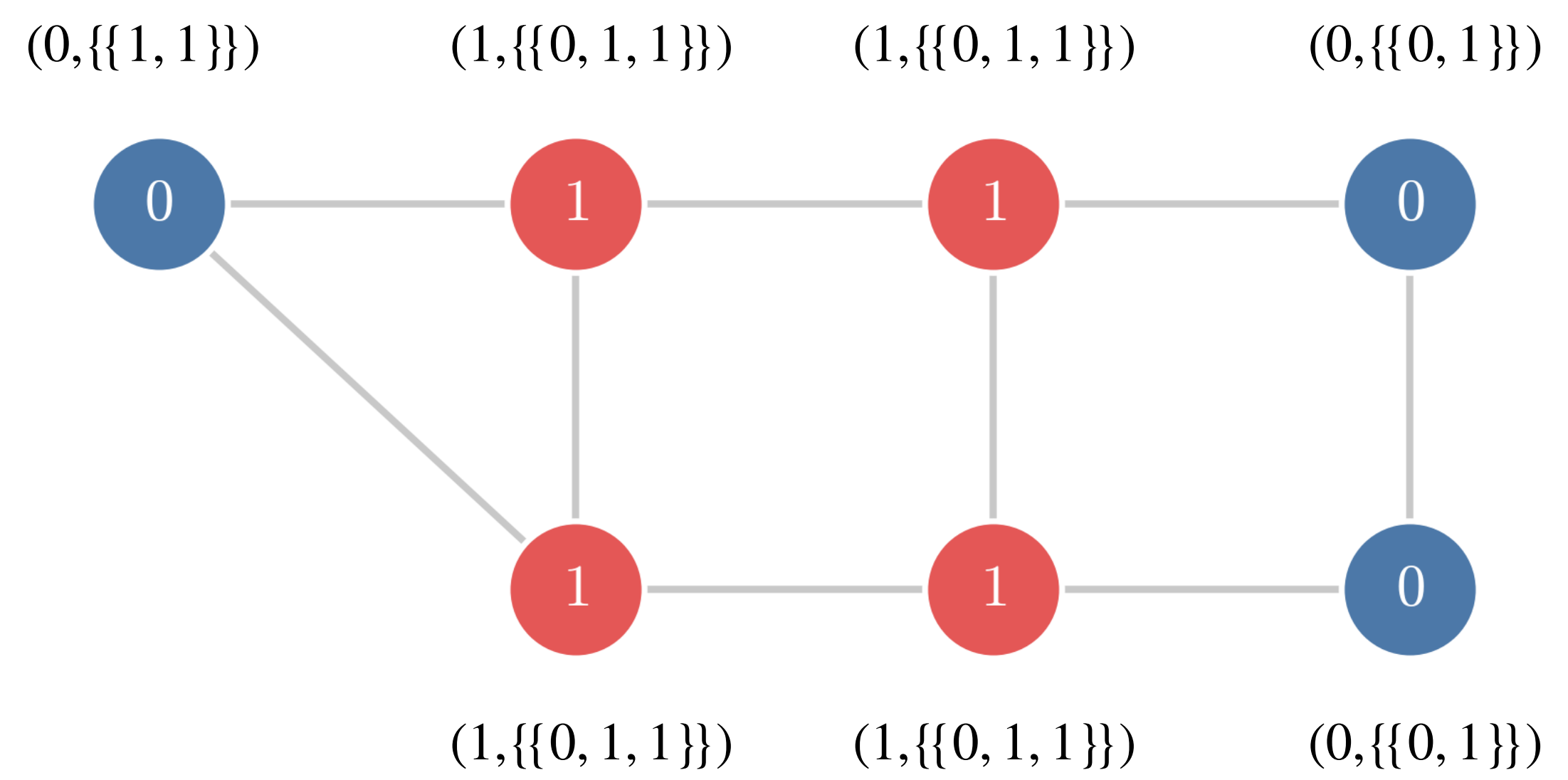
Compute a new label by applying a hash function to the current label and the labels of its neighbors

Weisfeiler, B. & Leman, A. (1968). The reduction of a graph to canonical form and the algebra which appears therein. *Nauchno-Technicheskaya Informatsia*, Series 2, 9, pp. 12–16.

Weisfeiler—Leman test

An algorithm for testing graph isomorphism

Step 1



$$c_i^{(t)} = \text{HASH}\left(c_i^{(t-1)}, \{\{c_j^{(t-1)} : j \in \mathcal{N}(i)\}\}\right)$$

Iterative step:

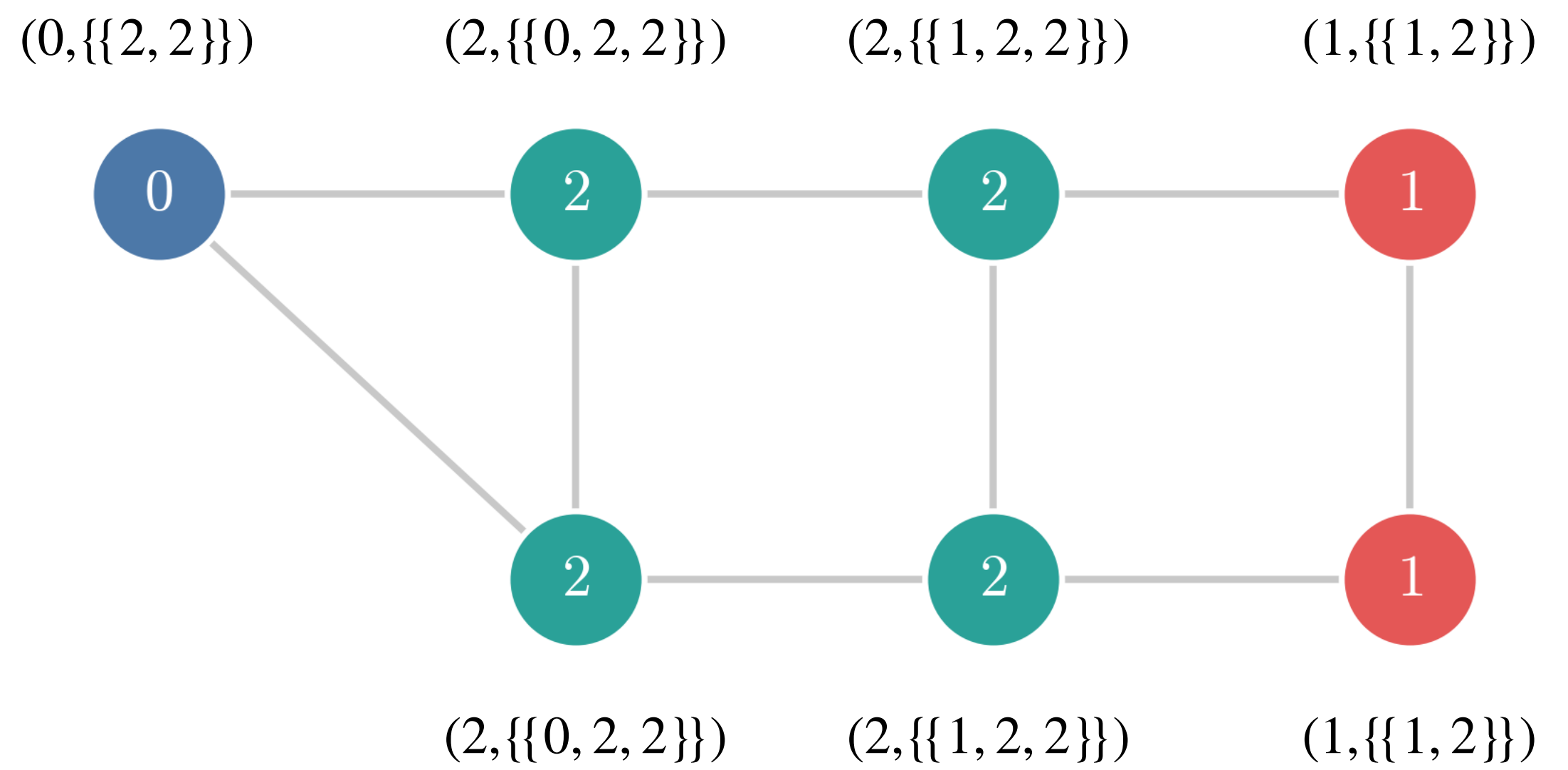
Compute a new label by applying a hash function to the current label and the labels of its neighbors

Weisfeiler, B. & Leman, A. (1968). The reduction of a graph to canonical form and the algebra which appears therein. *Nauchno-Technicheskaya Informatsia*, Series 2, 9, pp. 12–16.

Weisfeiler—Leman test

An algorithm for testing graph isomorphism

Step 2



$$c_i^{(t)} = \text{HASH}\big(c_i^{(t-1)}, \{\{c_j^{(t-1)} : j \in \mathcal{N}(i)\}\}\big)$$

Iterative step:

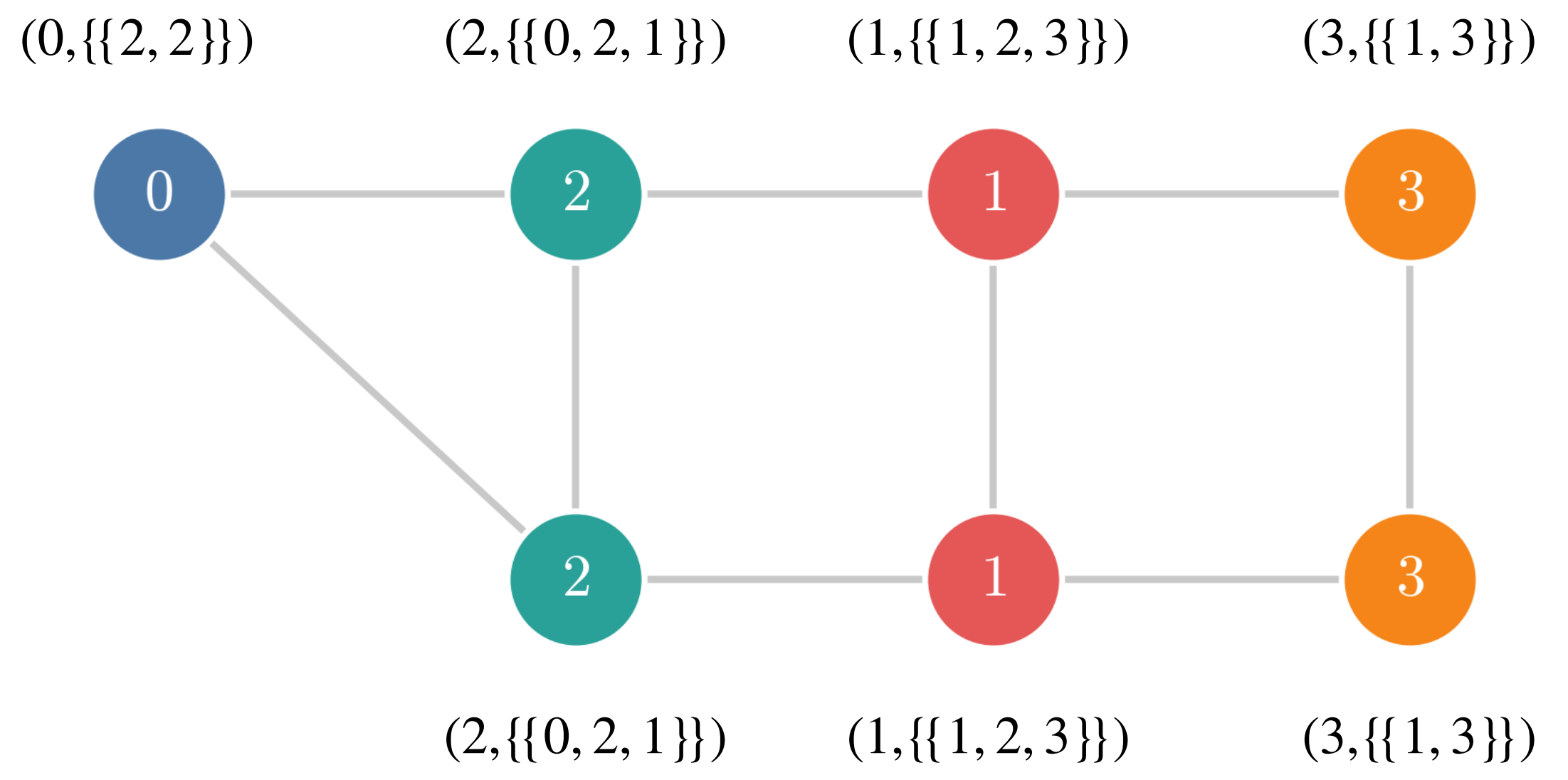
Compute a new label by applying a hash function to the current label and the labels of its neighbors

Weisfeiler, B. & Leman, A. (1968). The reduction of a graph to canonical form and the algebra which appears therein. *Nauchno-Technicheskaya Informatsia*, Series 2, 9, pp. 12–16.

Weisfeiler—Leman test

An algorithm for testing graph isomorphism

Step 3



$$c_i^{(t)} = \text{HASH}\big(c_i^{(t-1)}, \{\{c_j^{(t-1)} : j \in \mathcal{N}(i)\}\}\big)$$

Iterative step:

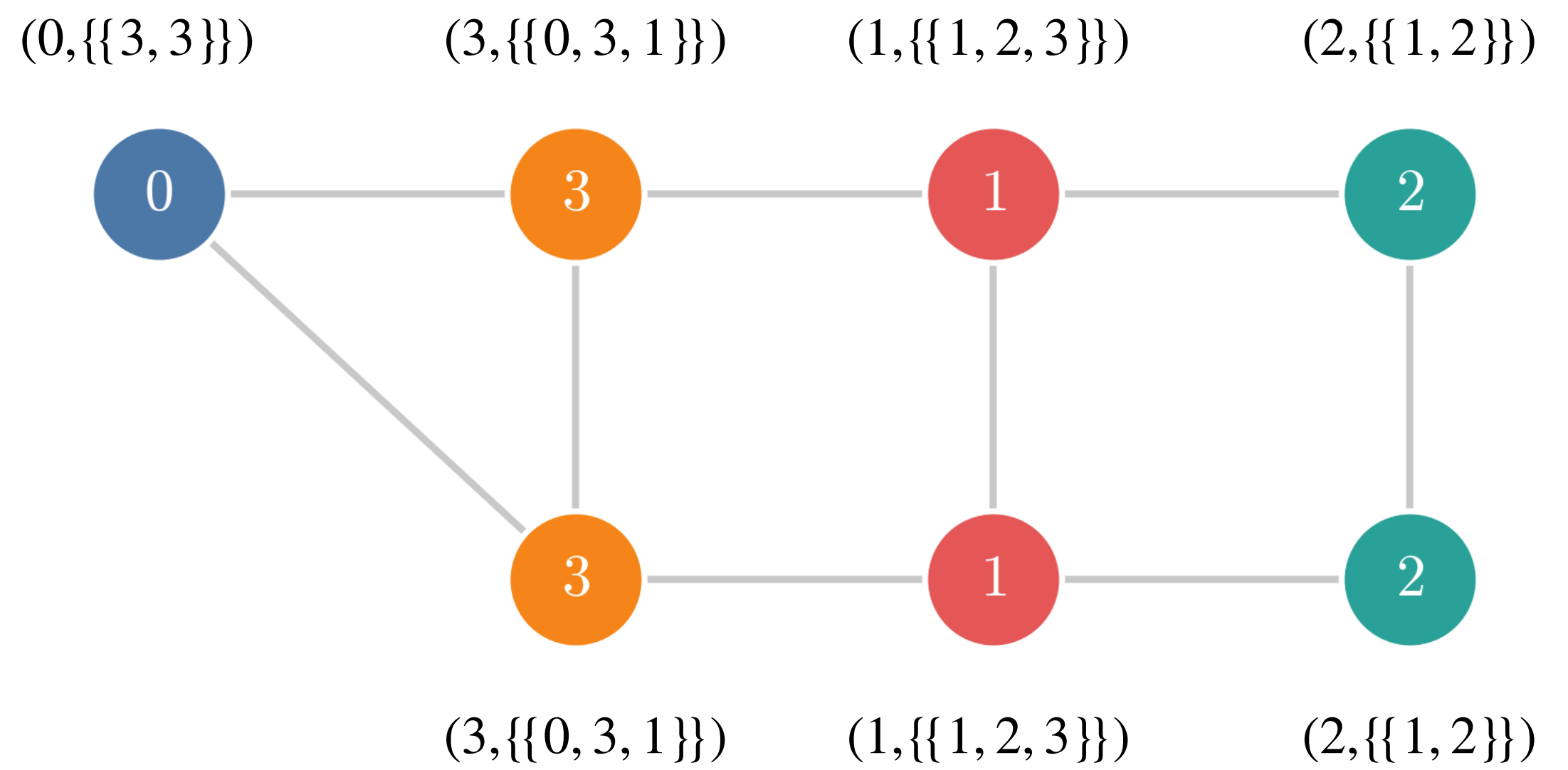
Compute a new label by applying a hash function to the current label and the labels of its neighbors

Weisfeiler, B. & Leman, A. (1968). The reduction of a graph to canonical form and the algebra which appears therein. *Nauchno-Technicheskaya Informatsia*, Series 2, 9, pp. 12–16.

Weisfeiler—Leman test

An algorithm for testing graph isomorphism

Step 4



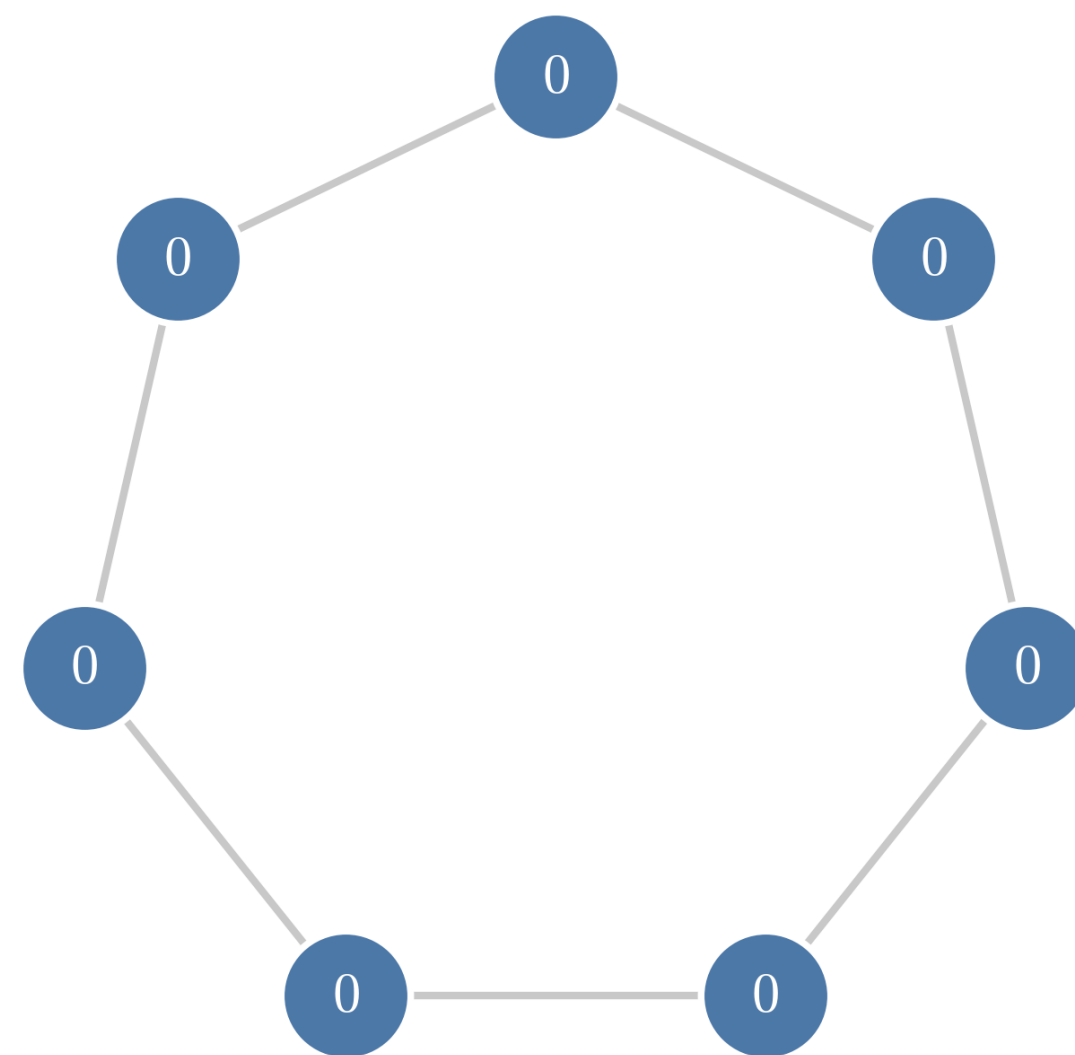
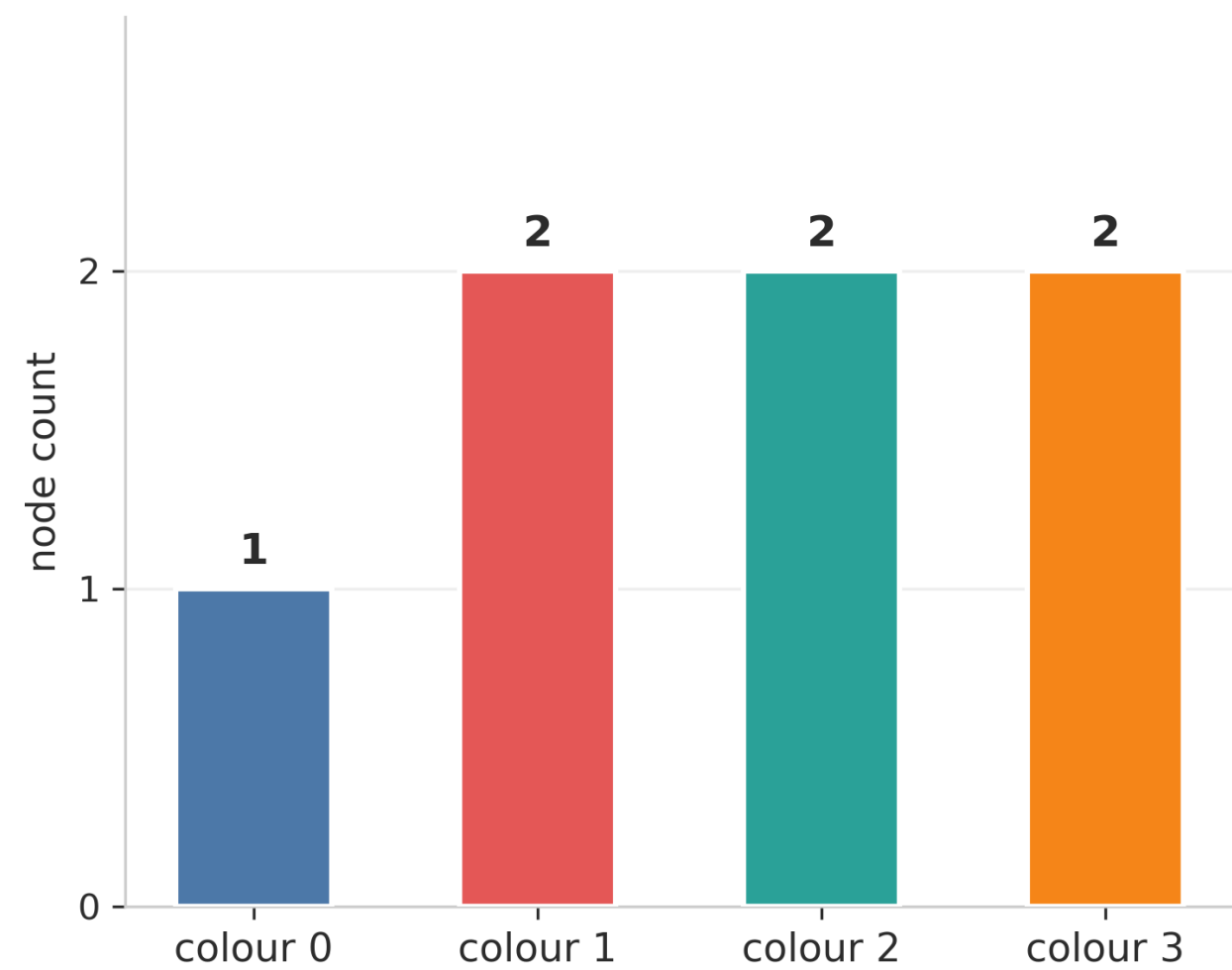
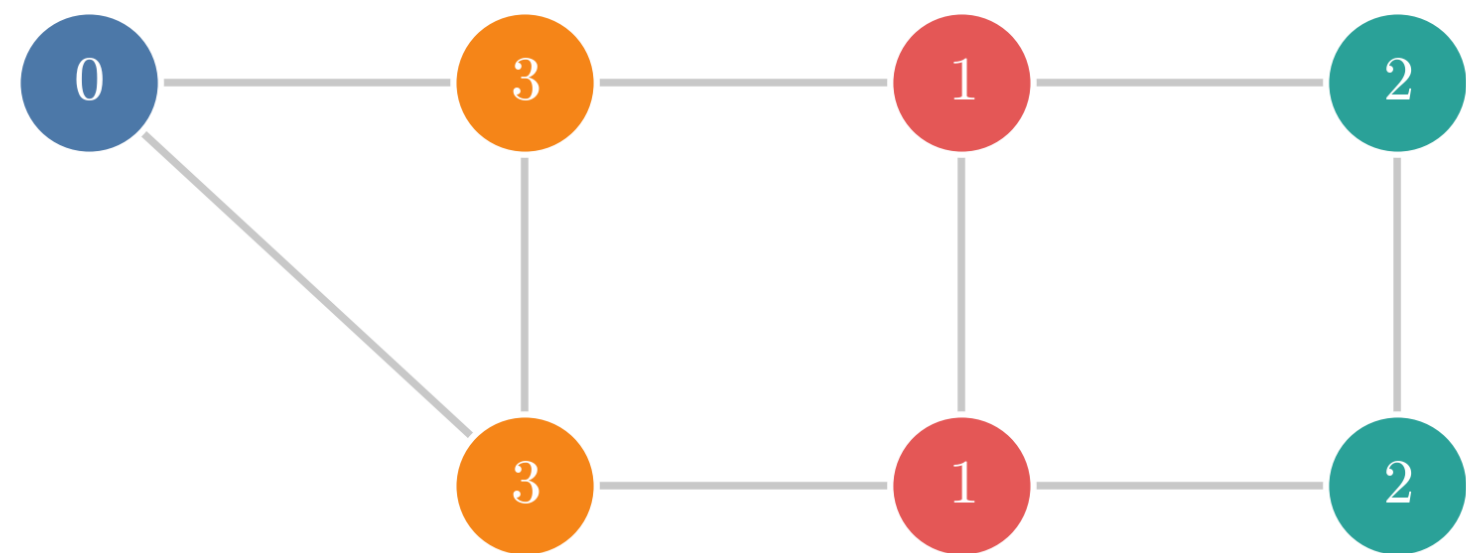
Stopping condition:

Stop the algorithm when we have converged to a partition of the nodes that does not change in an iteration

Weisfeiler, B. & Leman, A. (1968). The reduction of a graph to canonical form and the algebra which appears therein. *Nauchno-Technicheskaya Informatsia*, Series 2, 9, pp. 12–16.

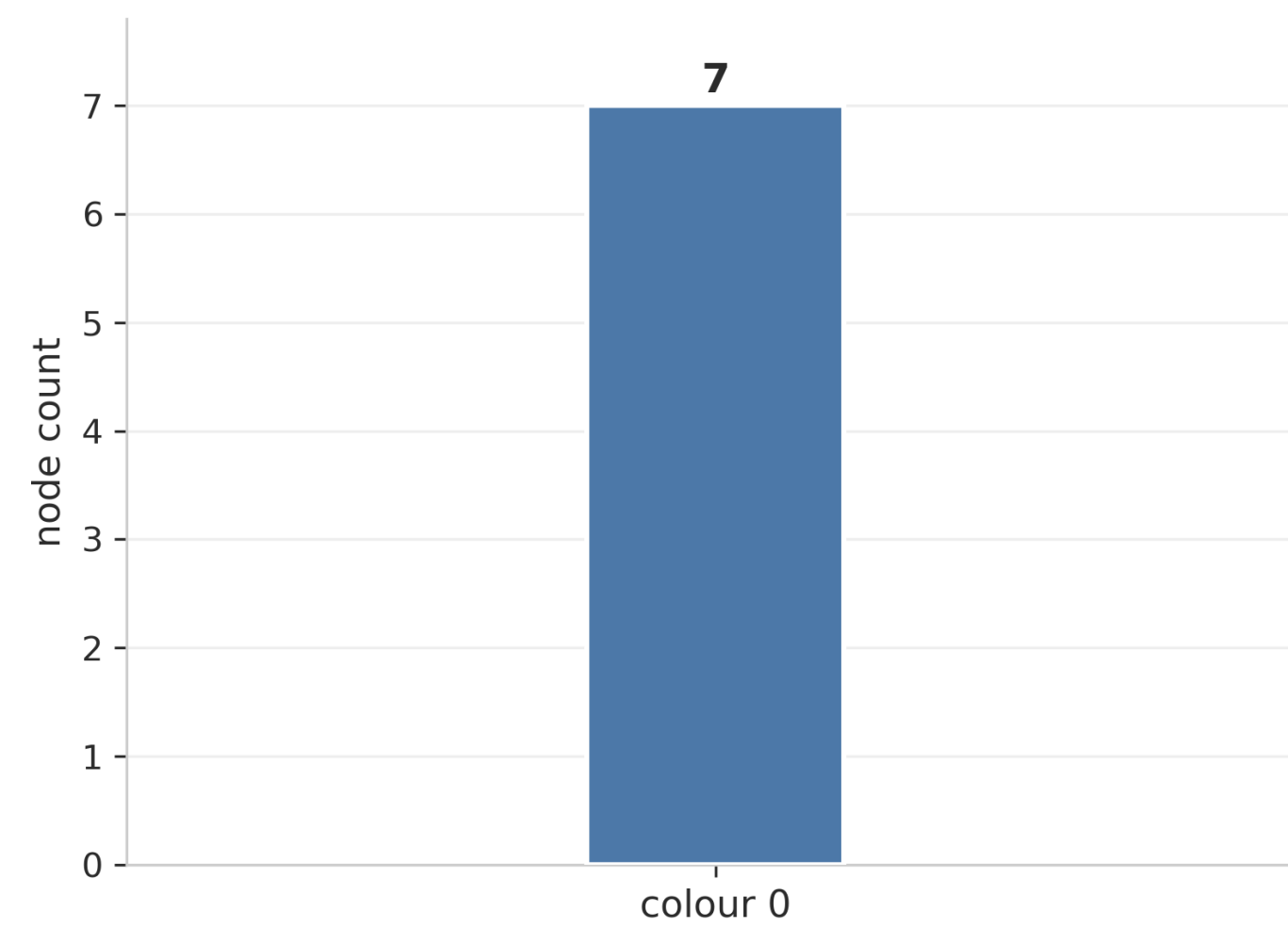
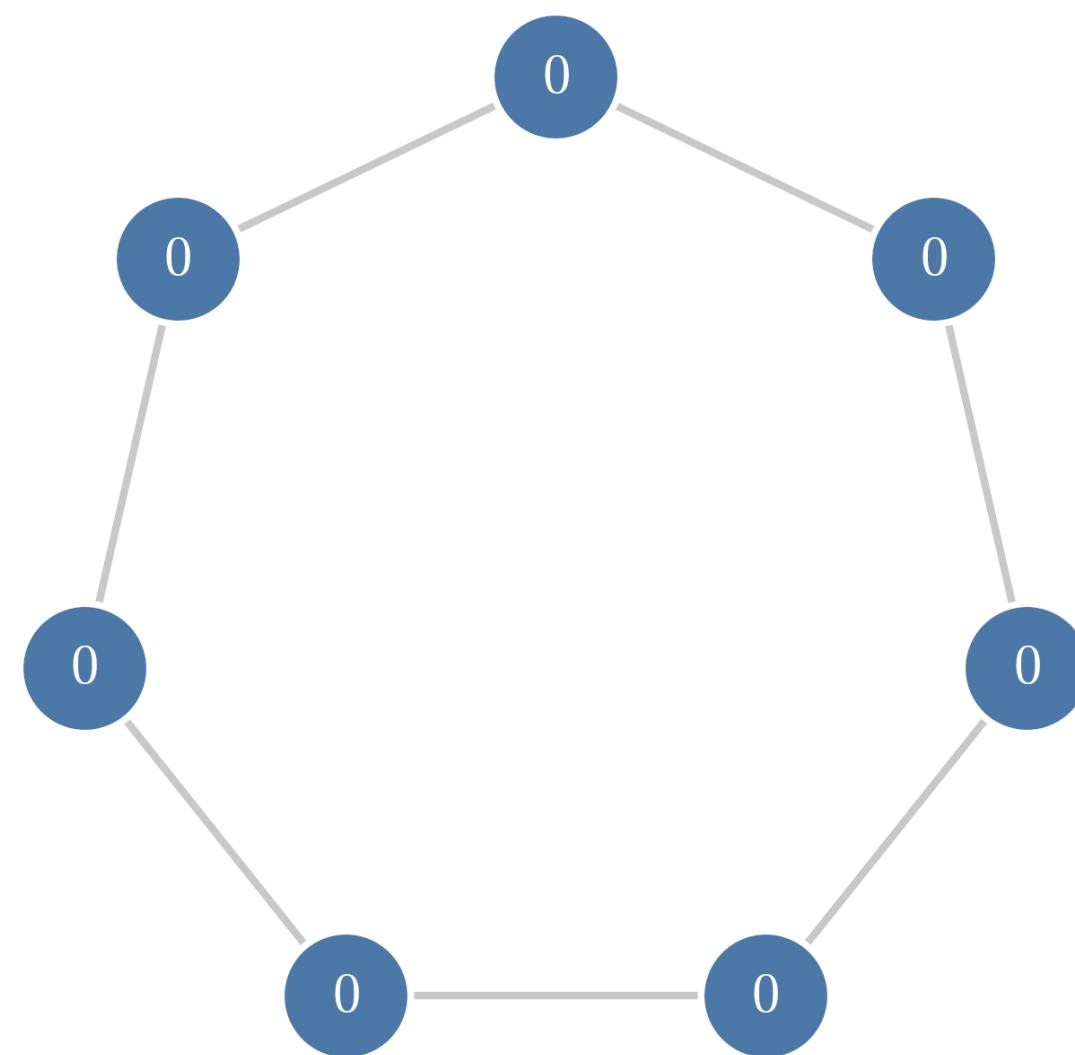
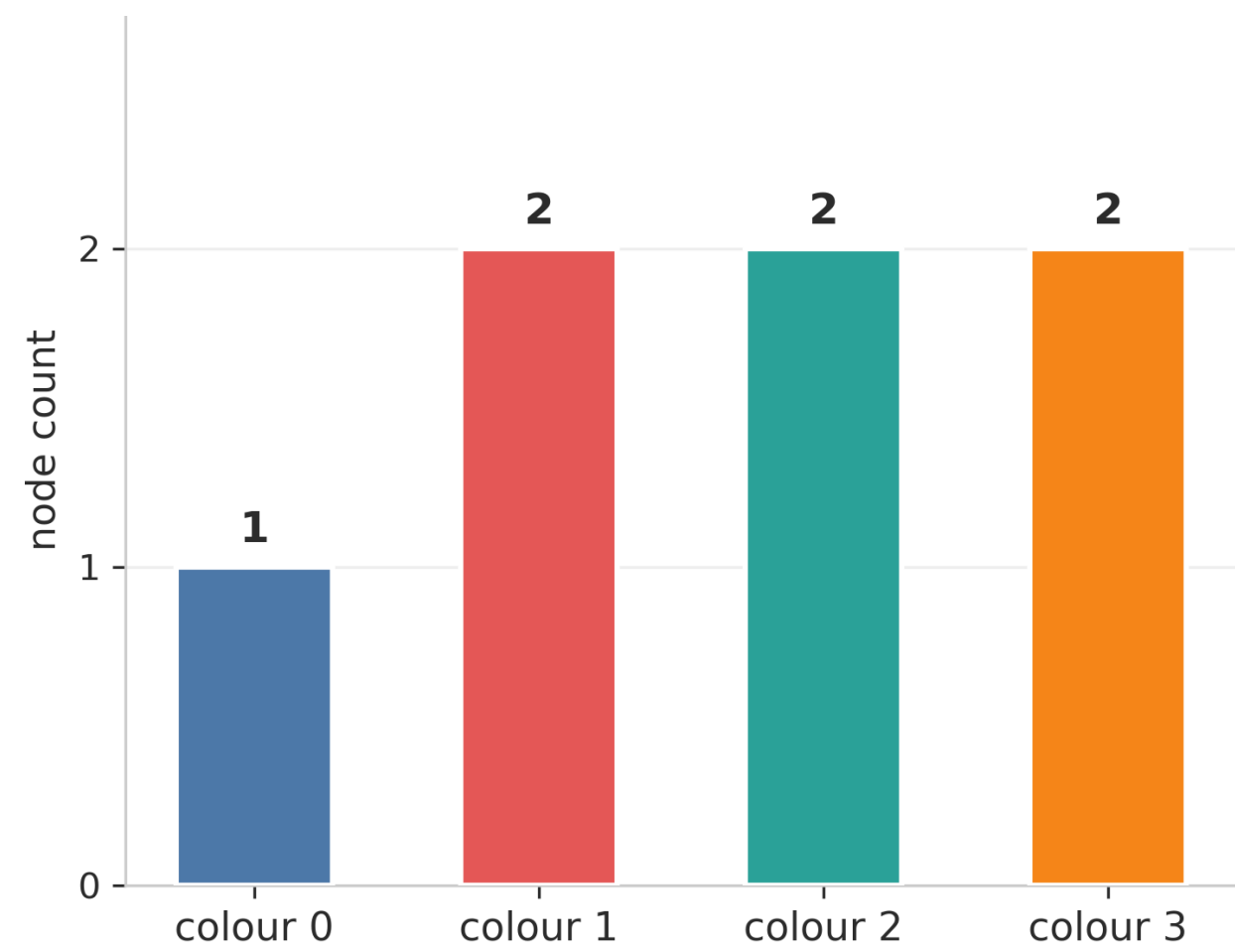
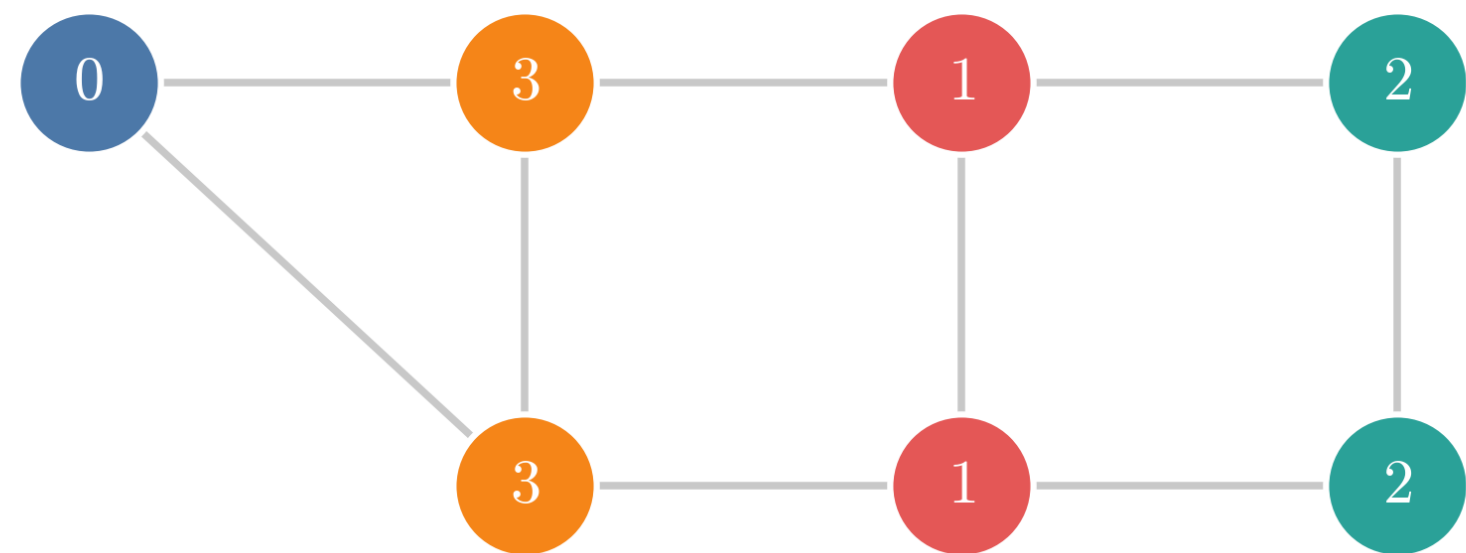
Weisfeiler—Leman test

An algorithm for testing graph isomorphism



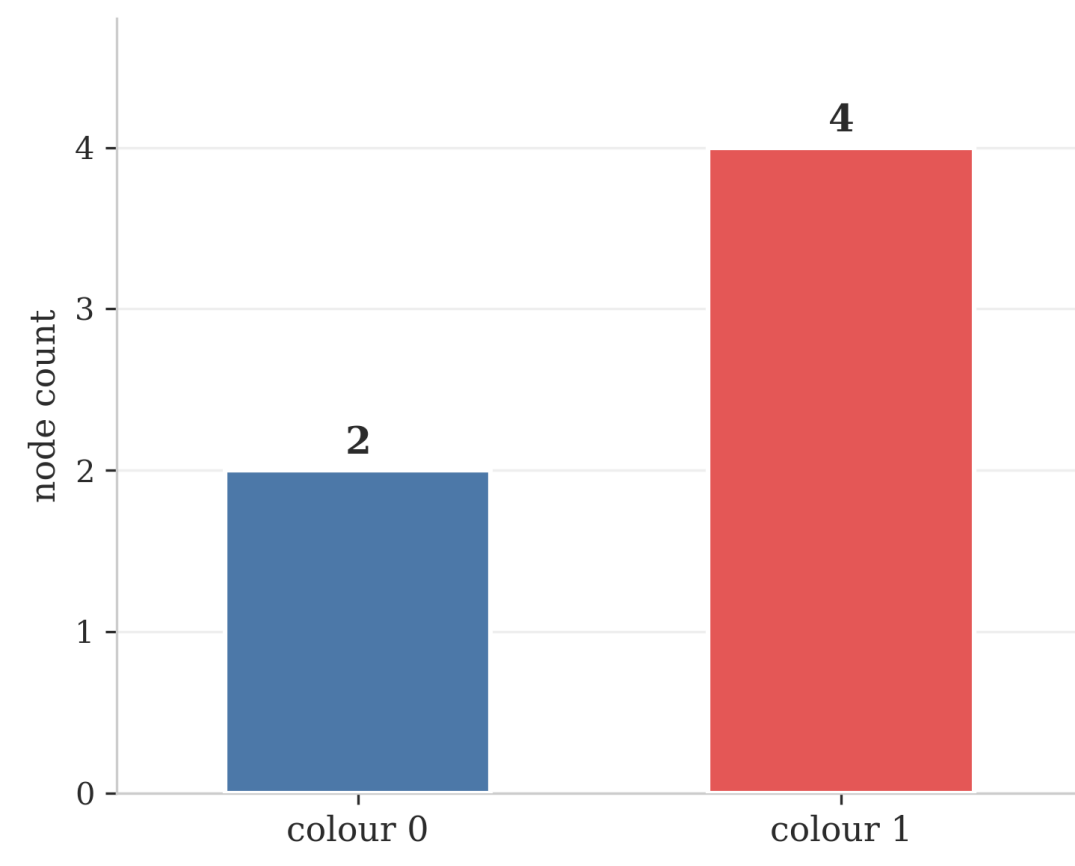
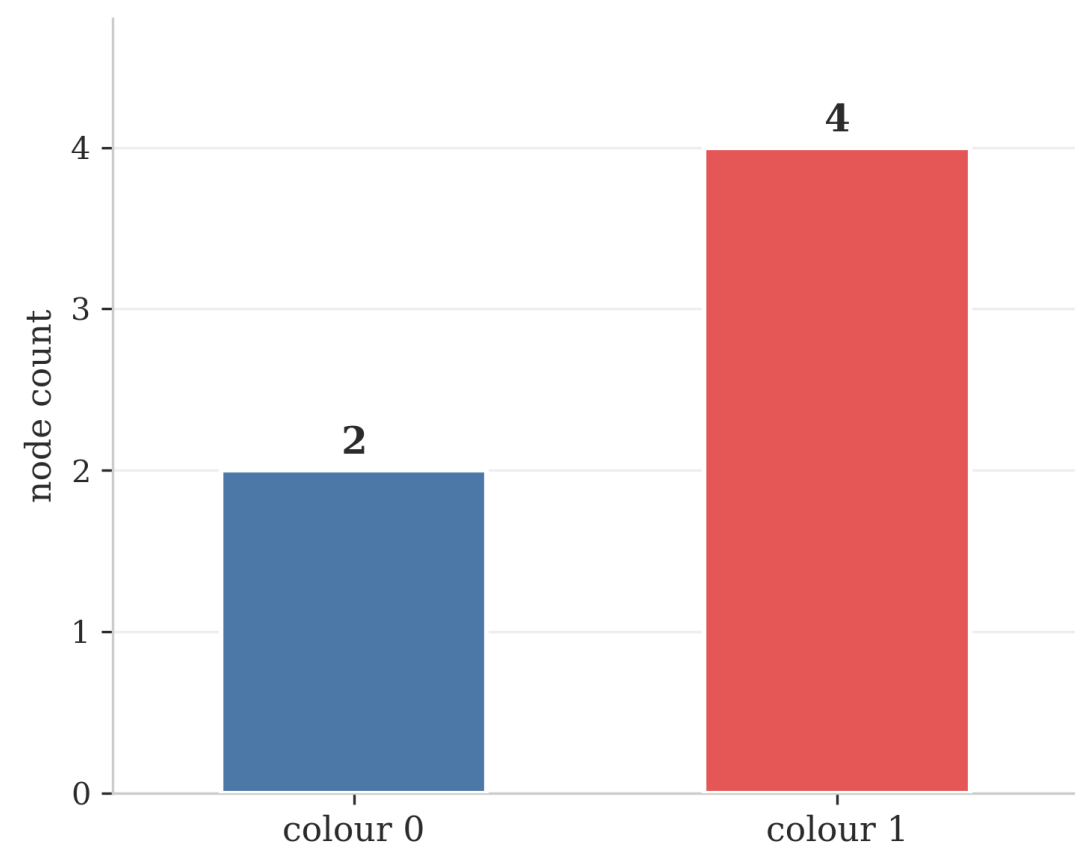
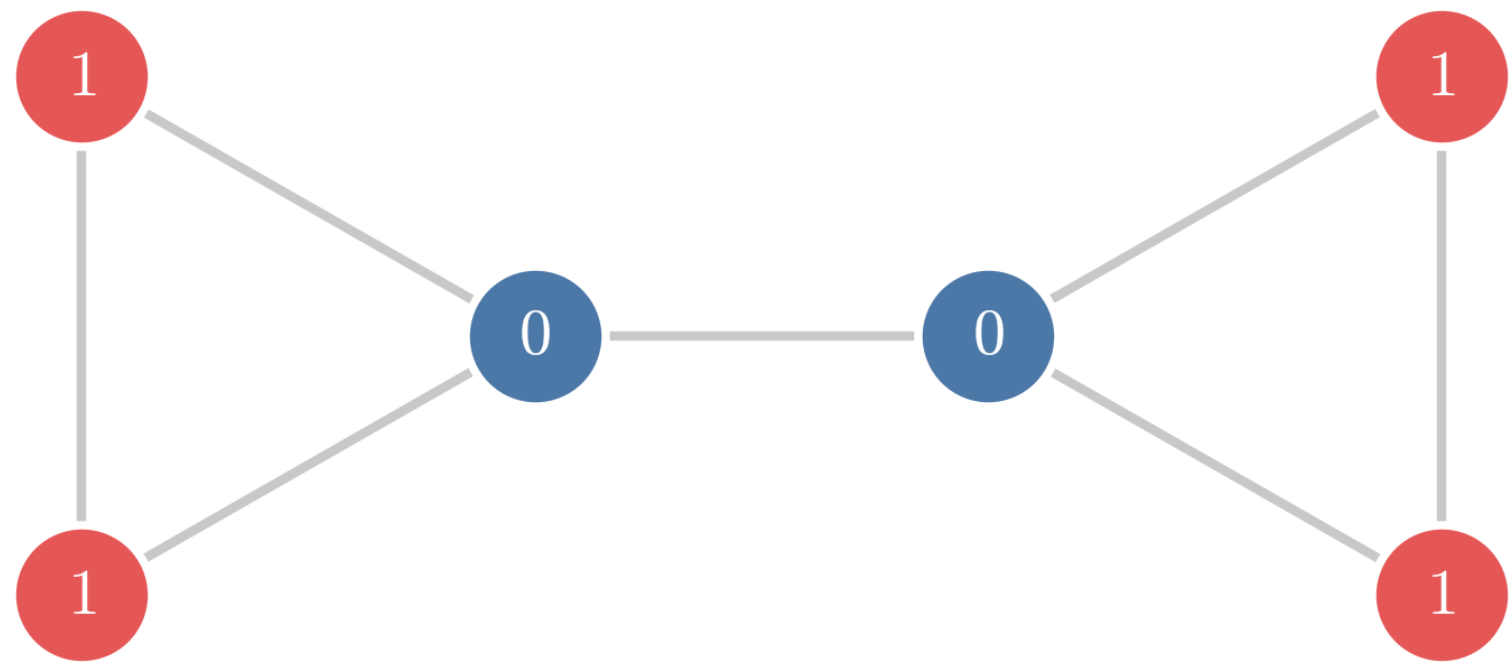
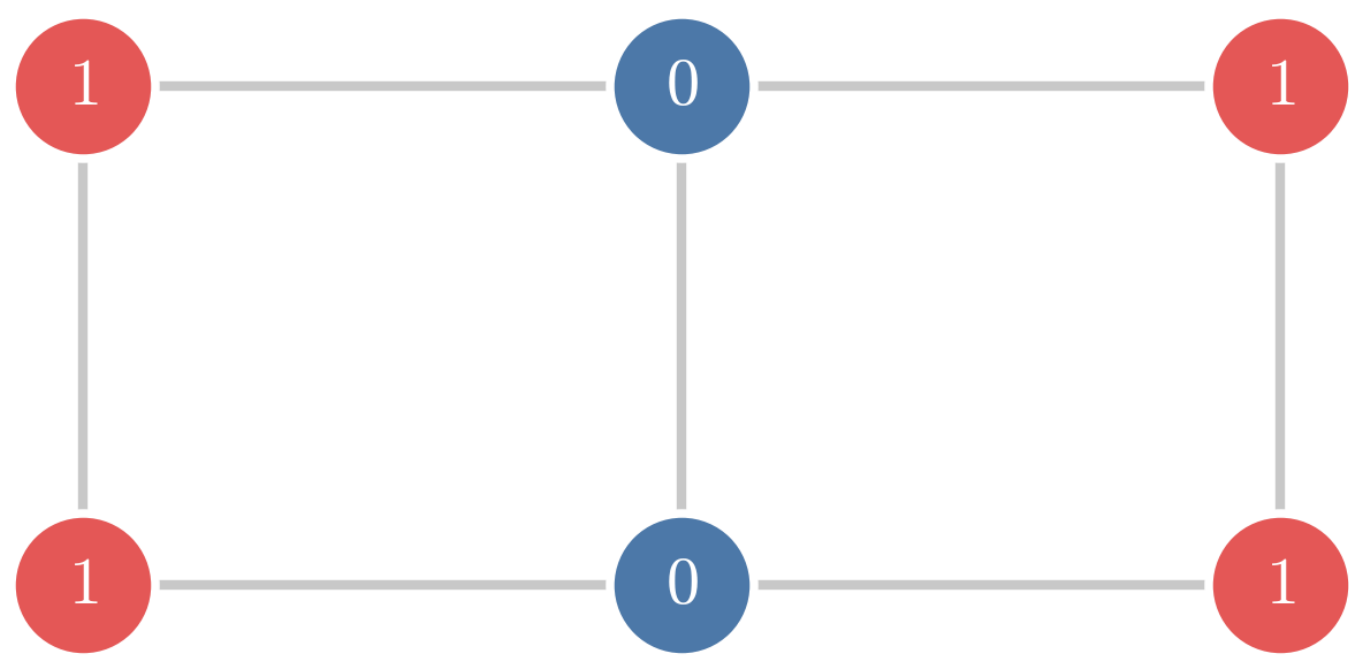
Weisfeiler—Leman test

An algorithm for testing graph isomorphism



Weisfeiler—Leman test

An algorithm for testing graph isomorphism



Expressivity of message-passing GNNs

How powerful are GNNs?

With a GNN, we want that at least is able to distinguish non-isomorphic graphs, that is: that it produces different embeddings $\mathbf{h}^{G_1} \neq \mathbf{h}^{G_2}$, $i \in V$ for two non-isomorphic graphs $G_1 \not\cong G_2$

We can achieve this thanks to the natural connection of message-passing with the WL test (applying an untrained GNN is similar in nature to running the WL algorithm)

Over discrete features, **message-passing GNNs can only be as powerful as the WL test**

You can reach maximum expressivity using an injective aggregator and maps in message-passing

You can try to improve the expressive power of GNNs by looking at the failure modes of the WL test

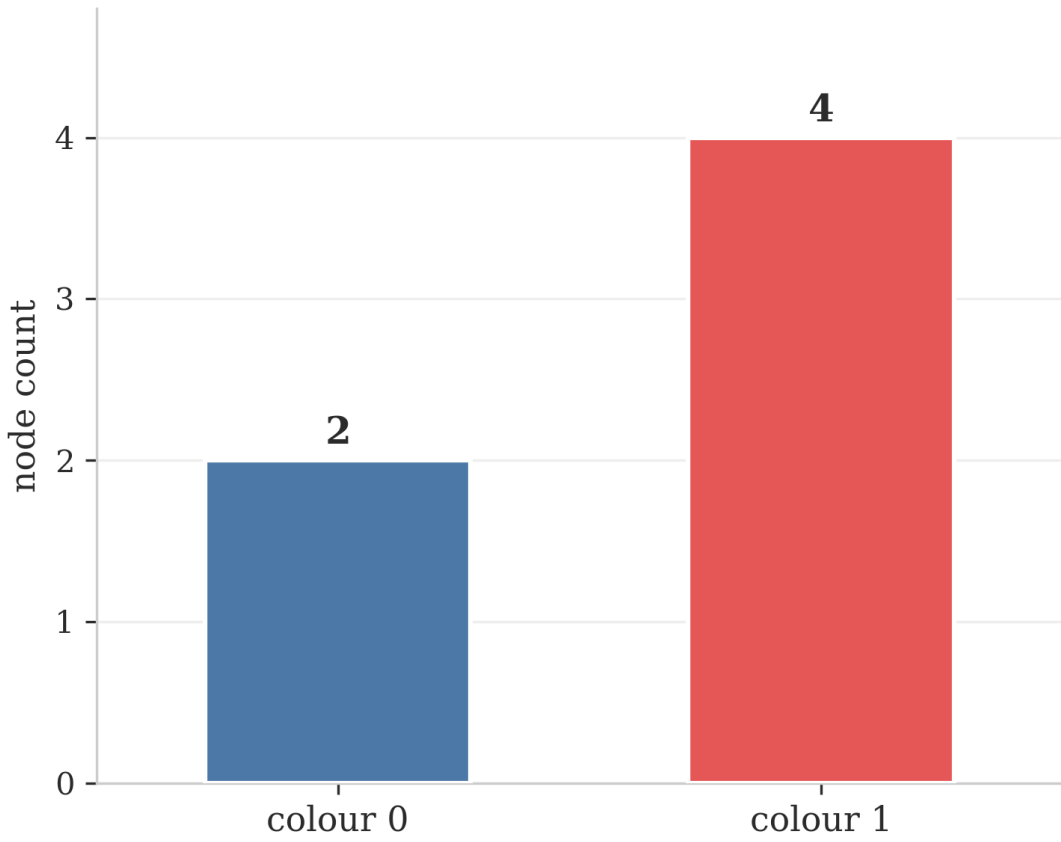
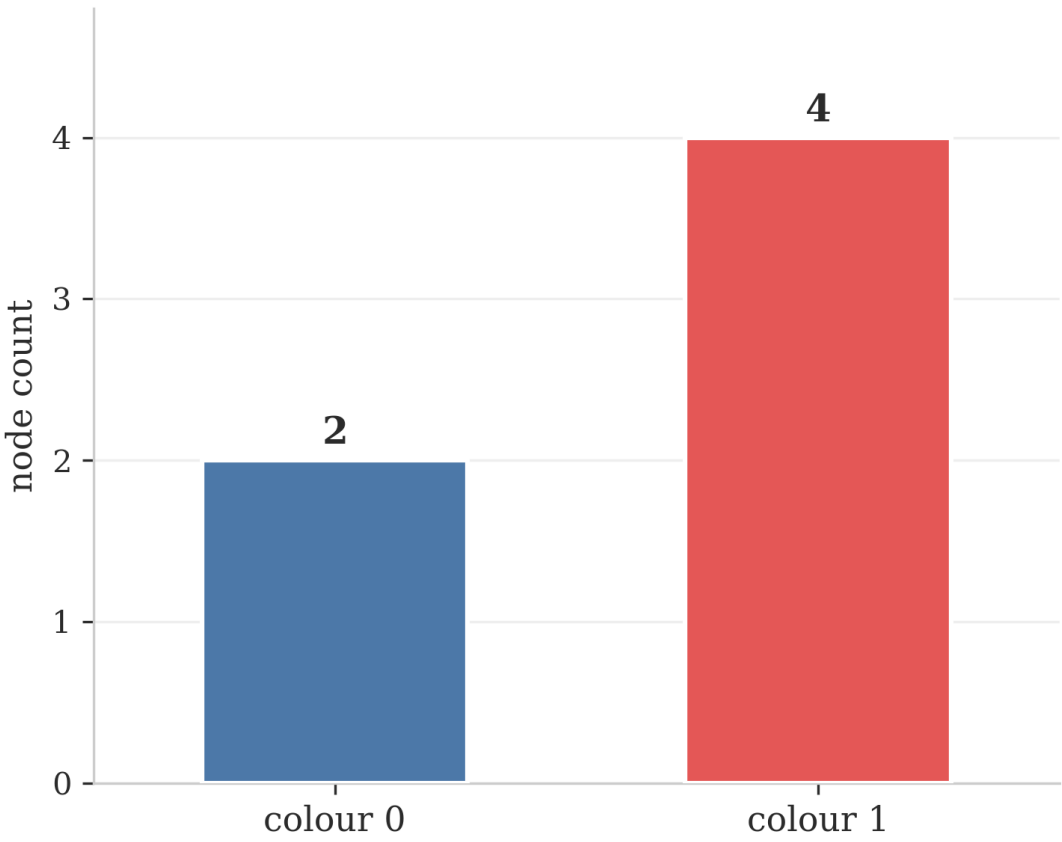
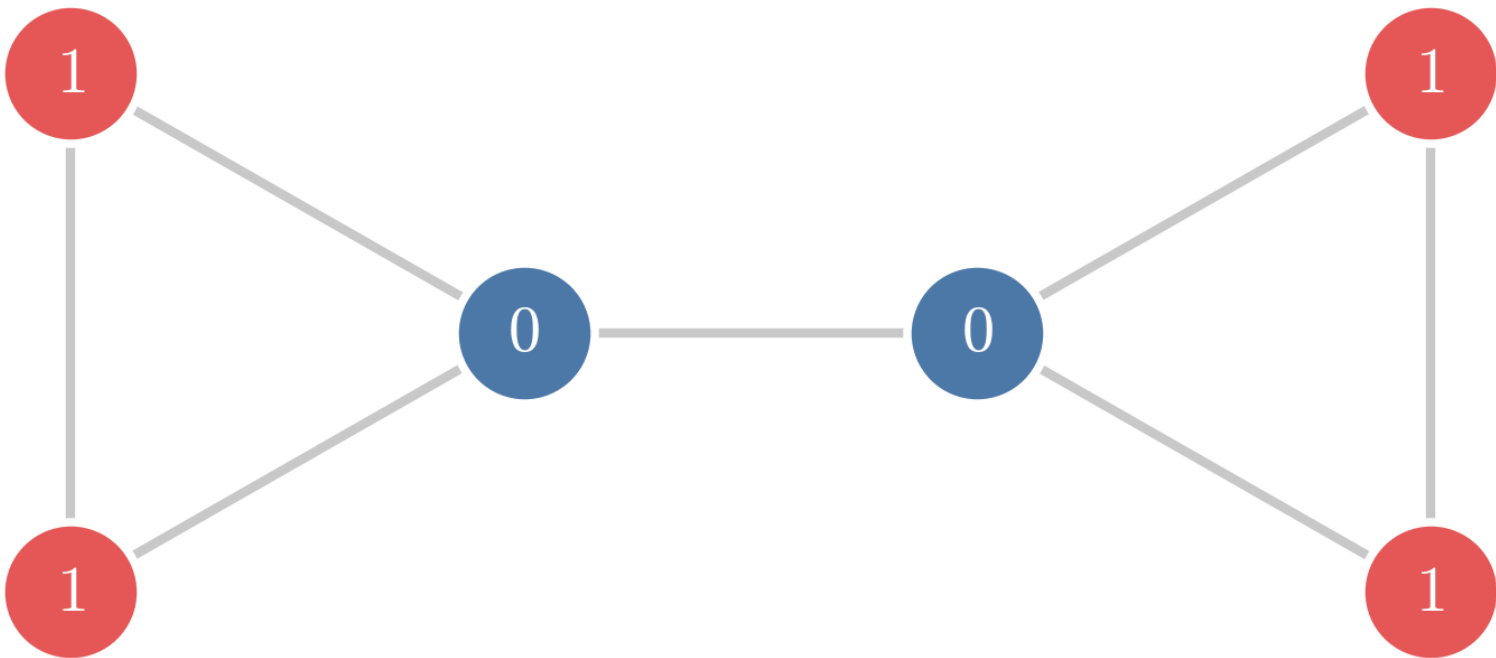
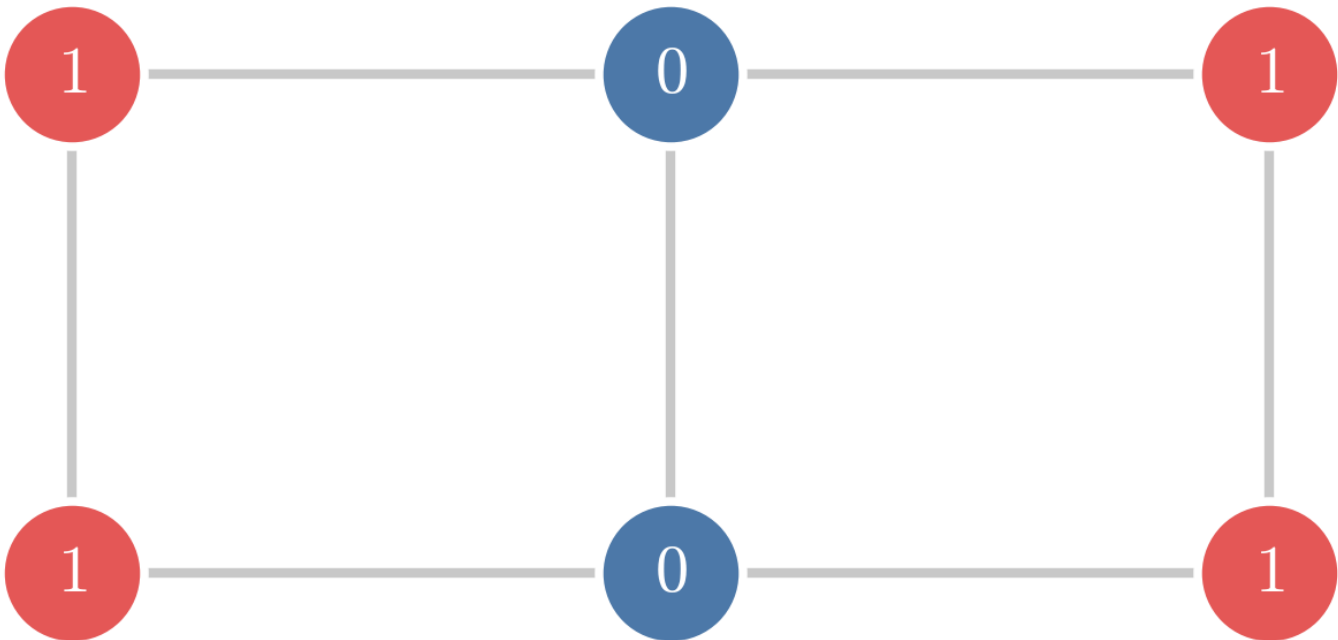
Xu, Keyulu, Weihua Hu, Jure Leskovec, and Stefanie Jegelka. How Powerful Are Graph Neural Networks? *International Conference on Learning Representations* (2018).

Going beyond the WL test

Lecture 3 — Learning on Topological Data

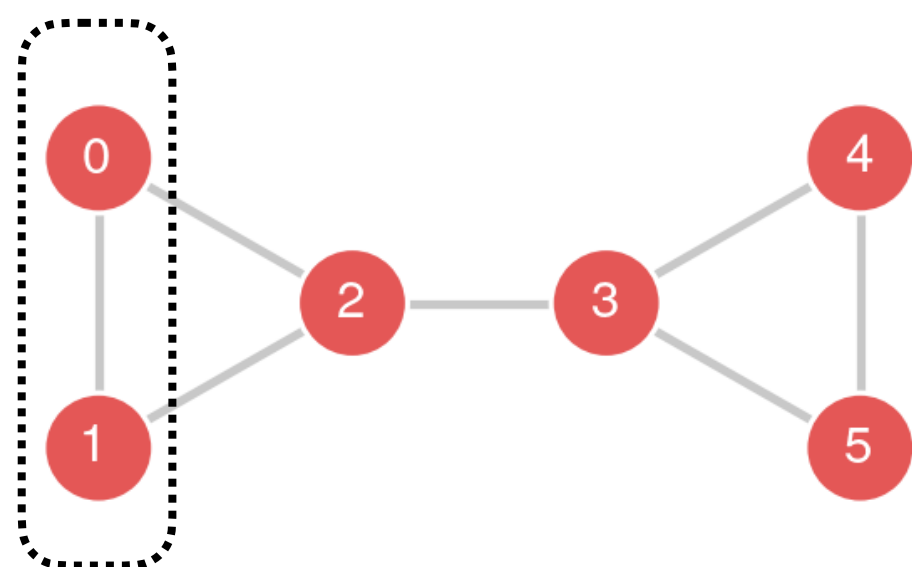
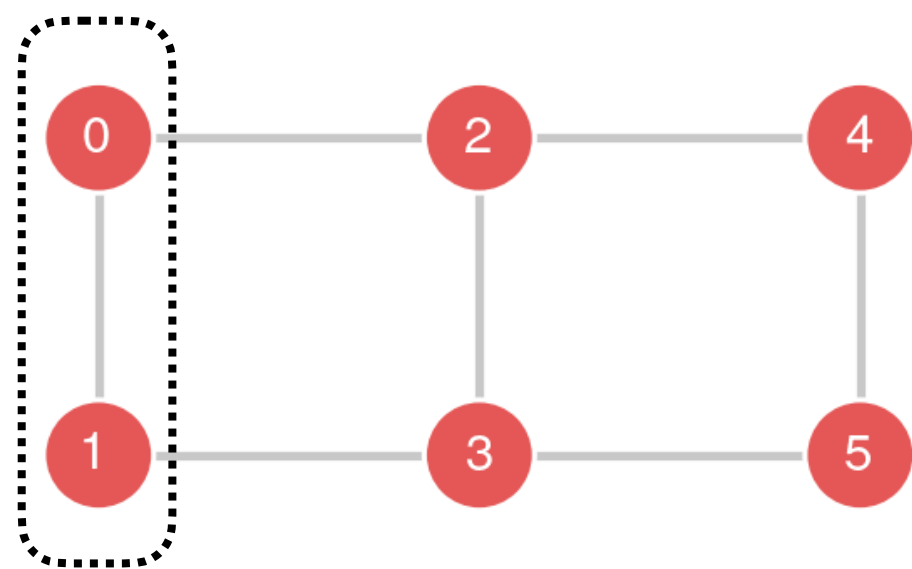
Weisfeiler—Leman test

Recall a failure mode of WL



2-WL test

Assigning colors to subsets of vertices



Initialization:

Initialize with all vertices having the same color as before

Iterative step:

Color pairs of ordered nodes by looking at the possible relations that we find passing through a third vertex

$$c(i, j)^{(t)} = \text{HASH}\big(c(i, j)^{(t-1)}, \{\{ (c(i, k)^{(t-1)}, c(k, j)^{(t-1)}) : k \in V \}\}\big)$$

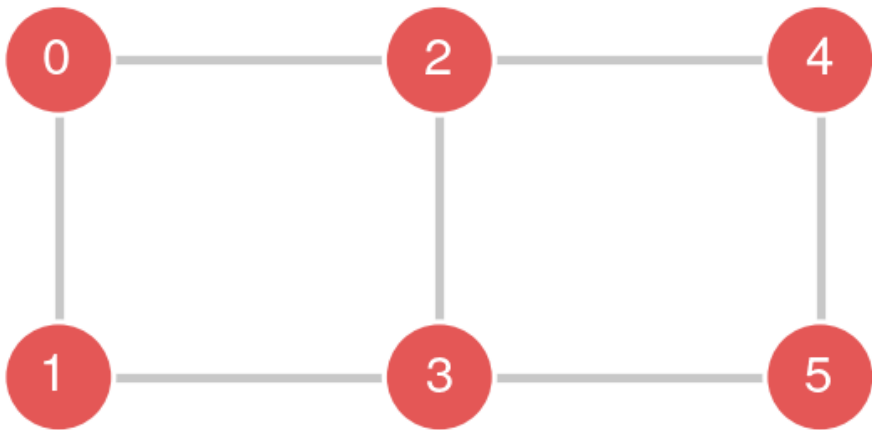
These can be edges, non-edges or diagonal relations

Example:

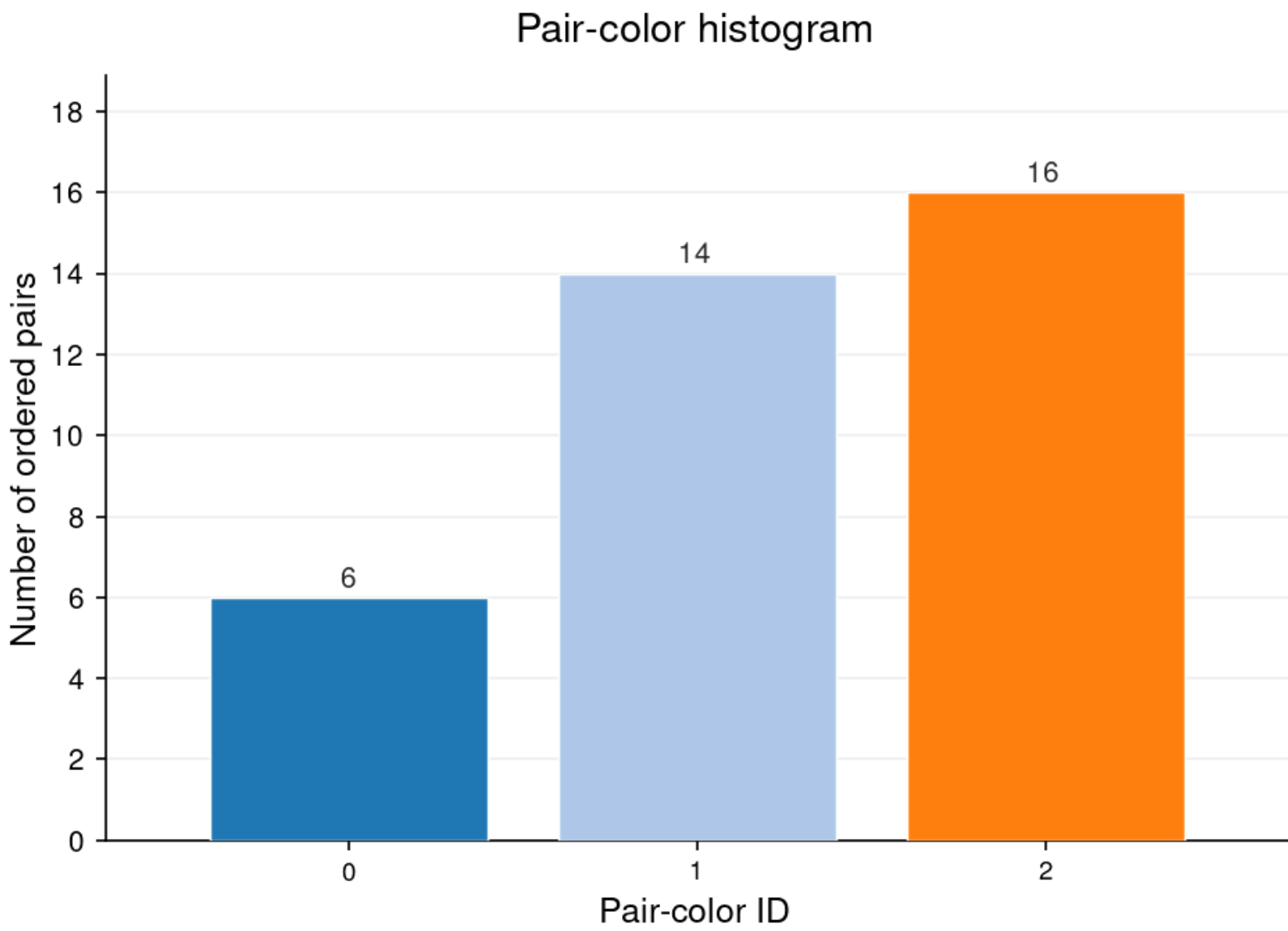
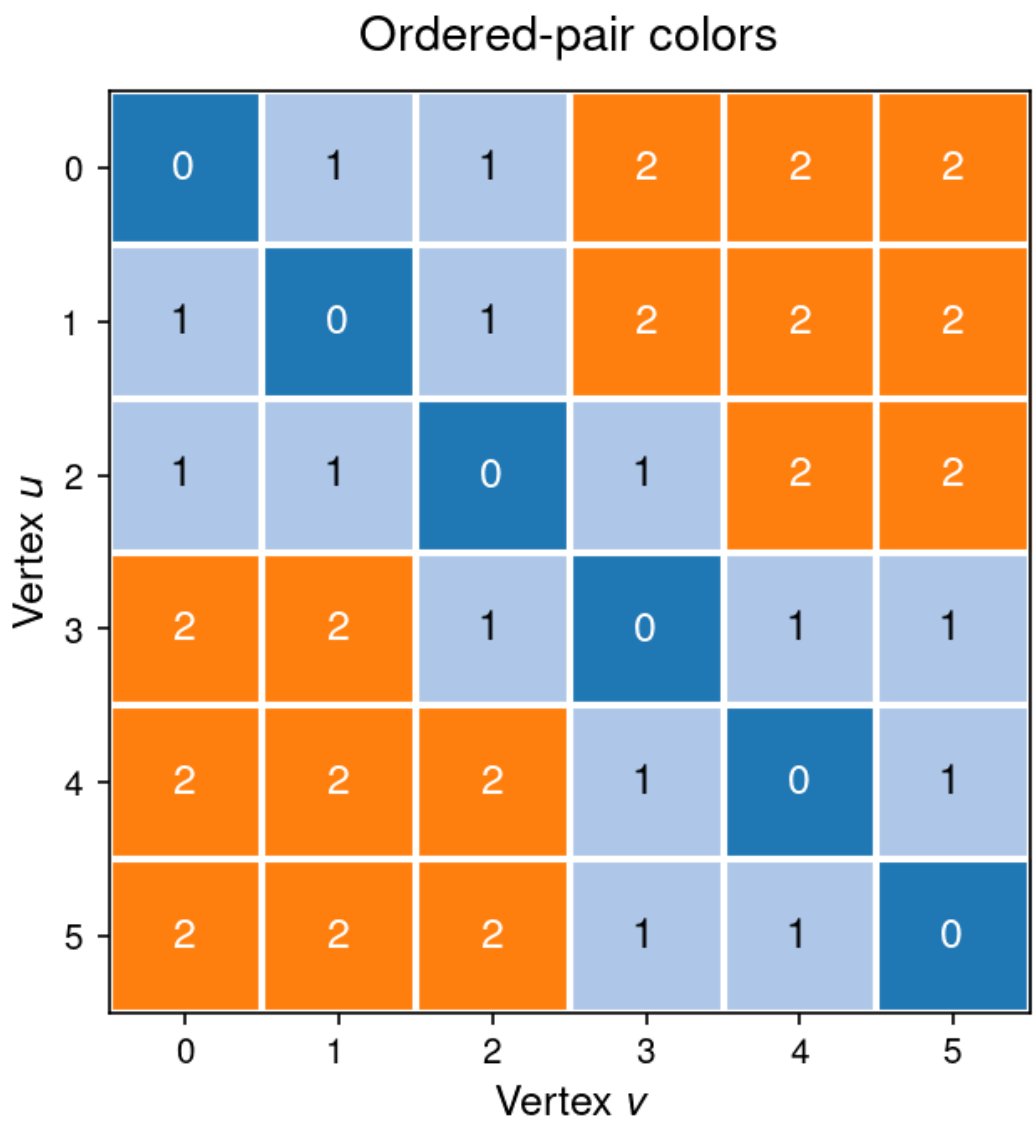
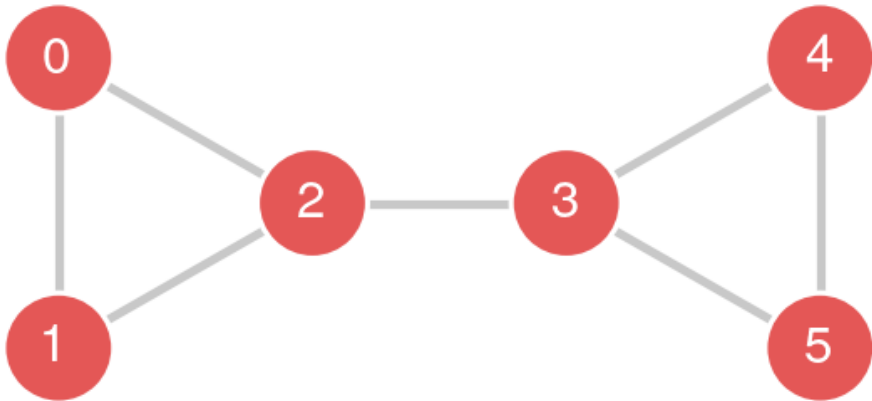
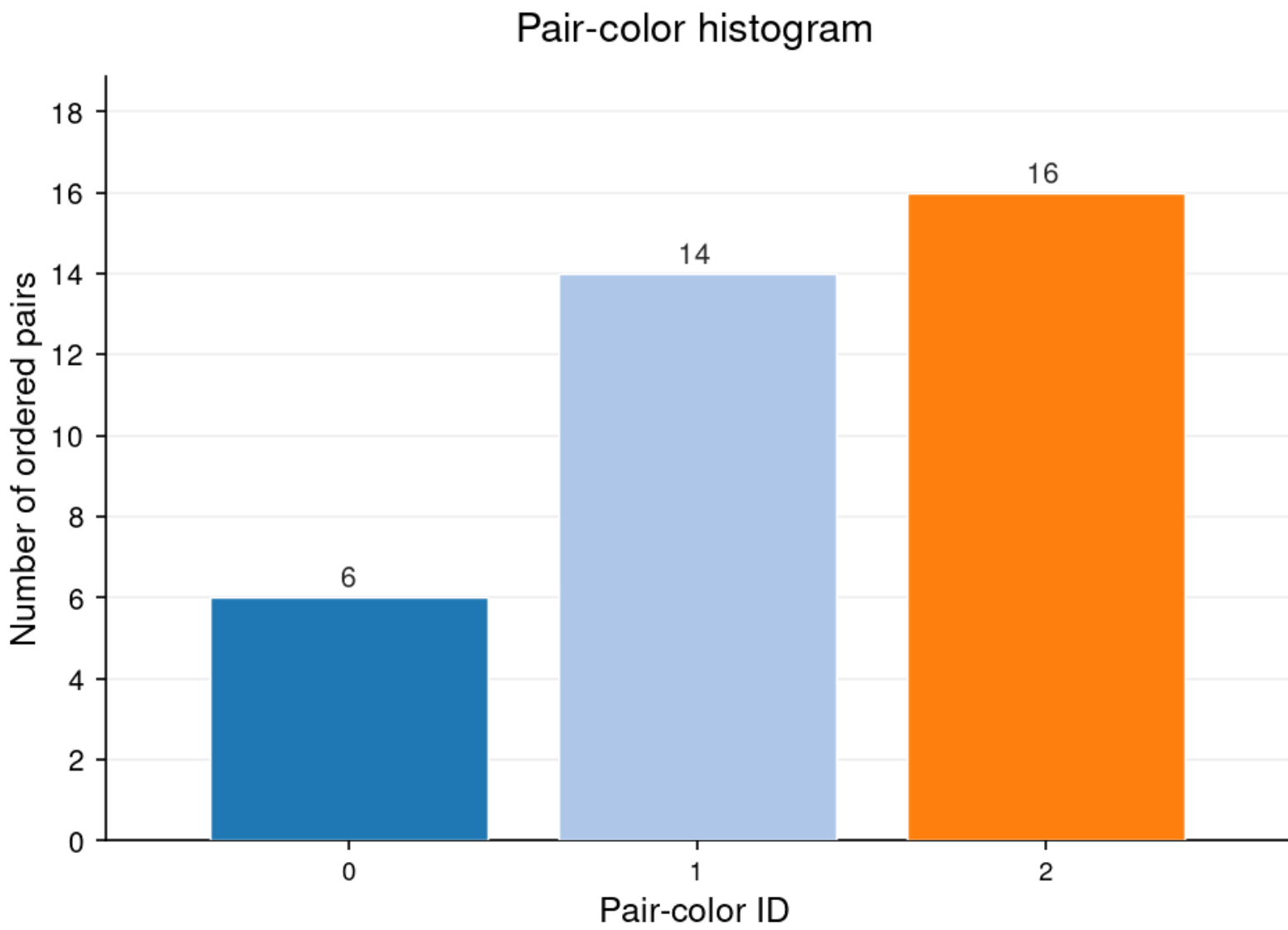
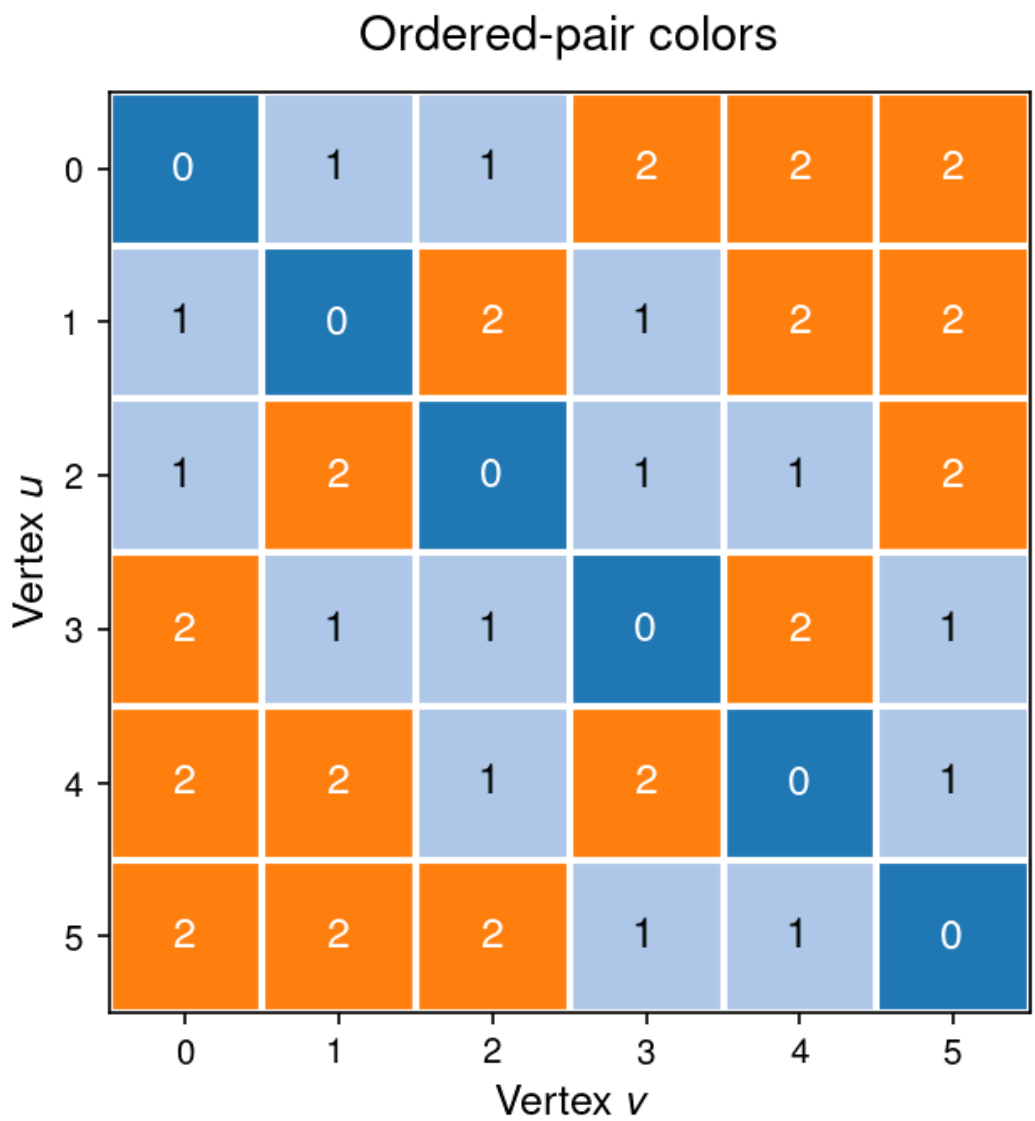
(0,1) in the graph above has an (edge, non-edge) and a (non-edge, non-edge) relation, and then and in the graph above only an (edge, edge) relation, so they get different colors under the HASH function

2-WL test

Assigning colors to subsets of vertices

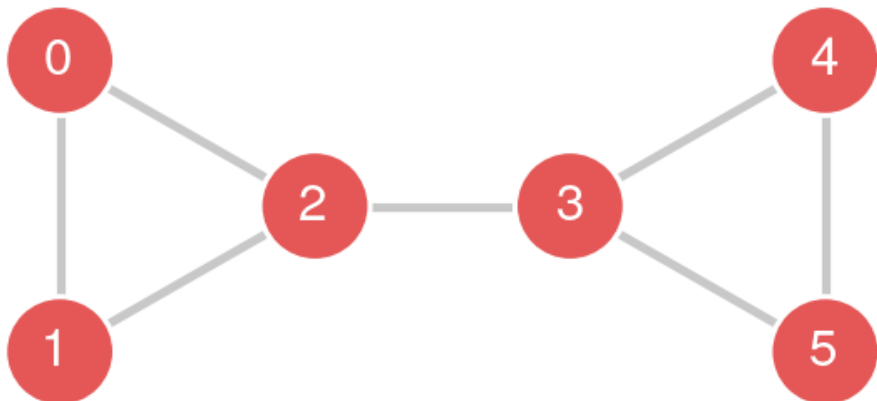
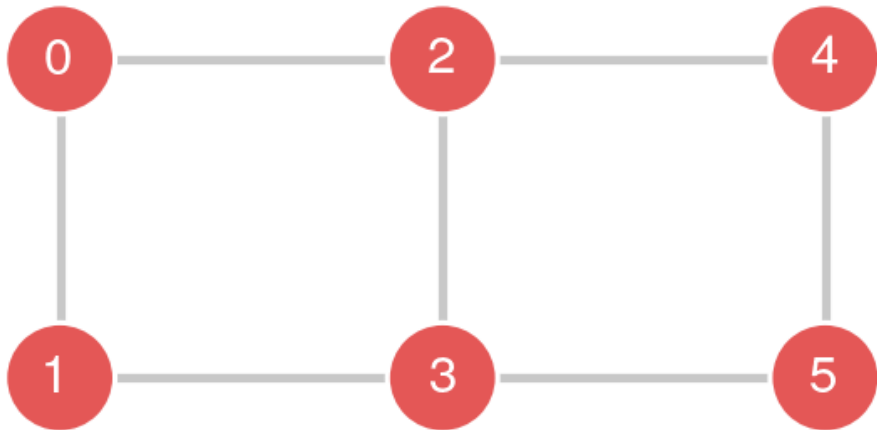


First iteration

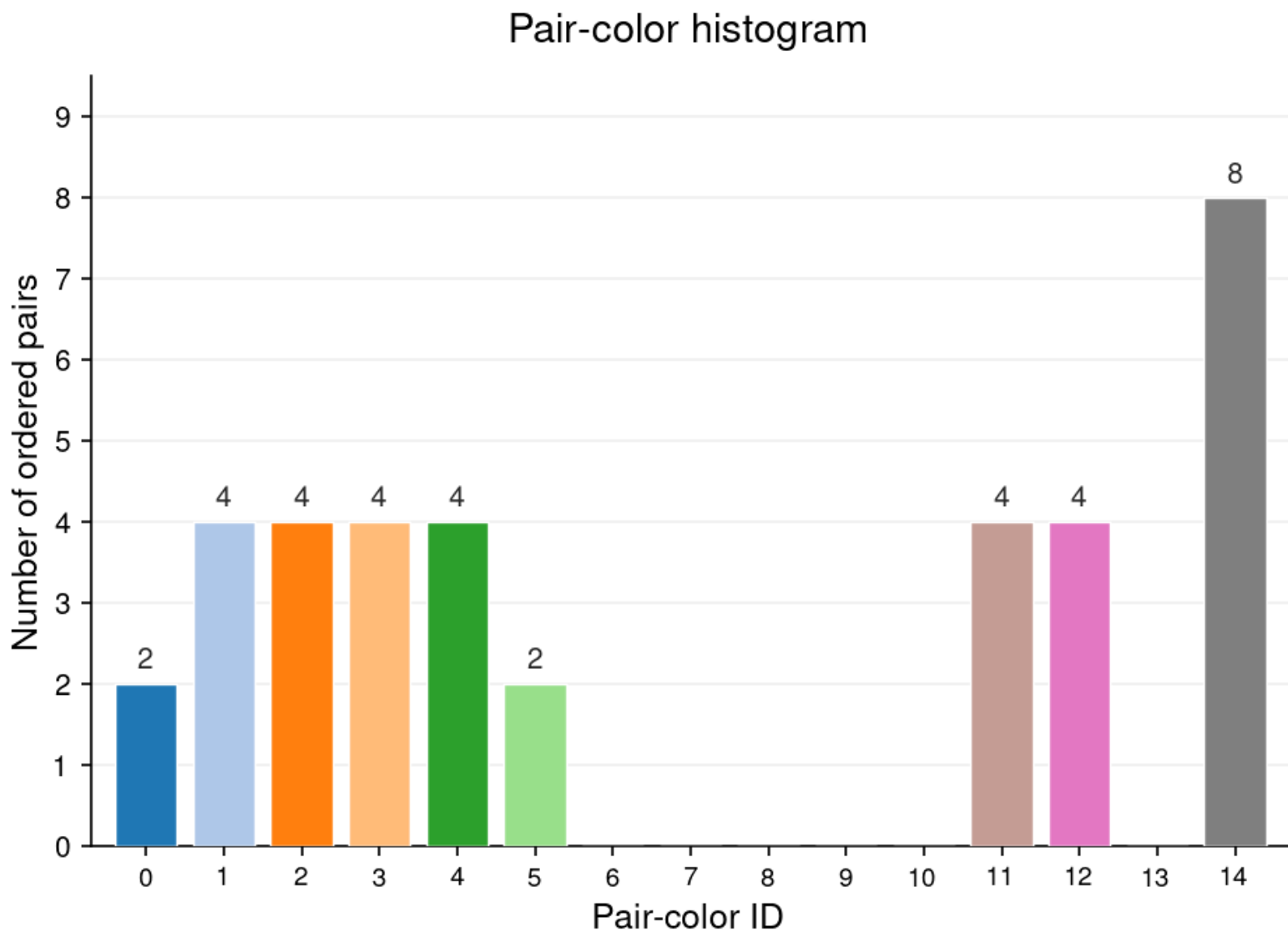
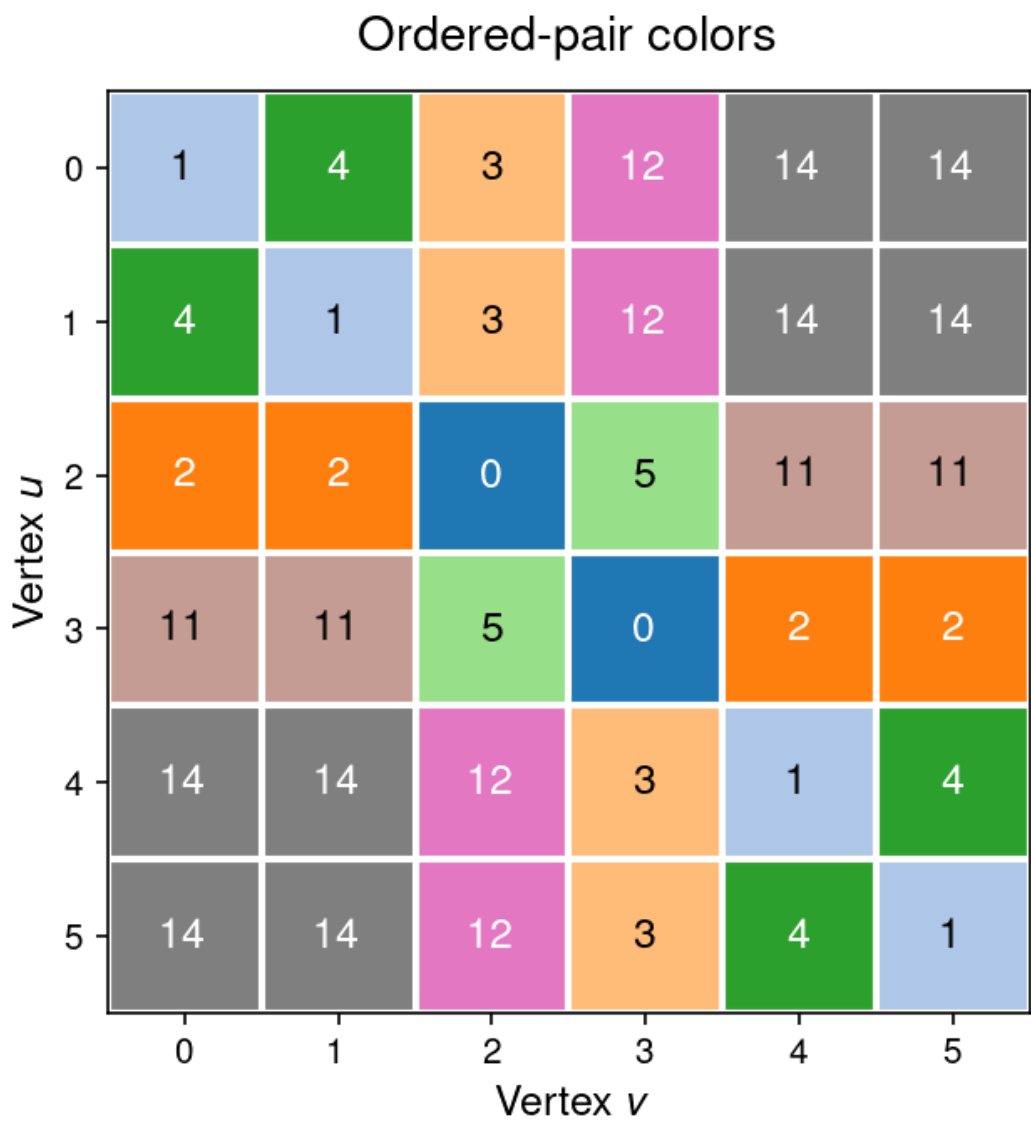
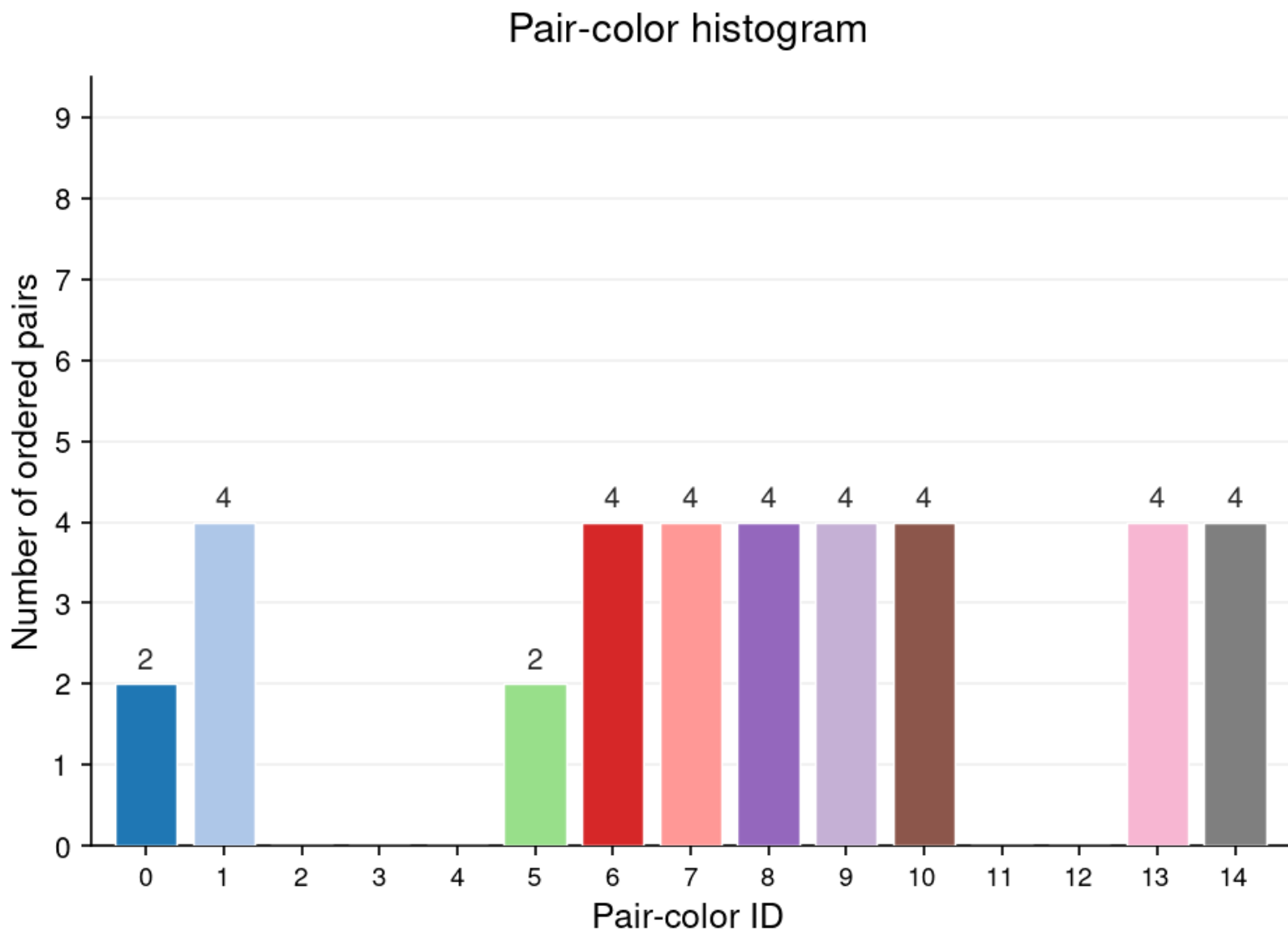
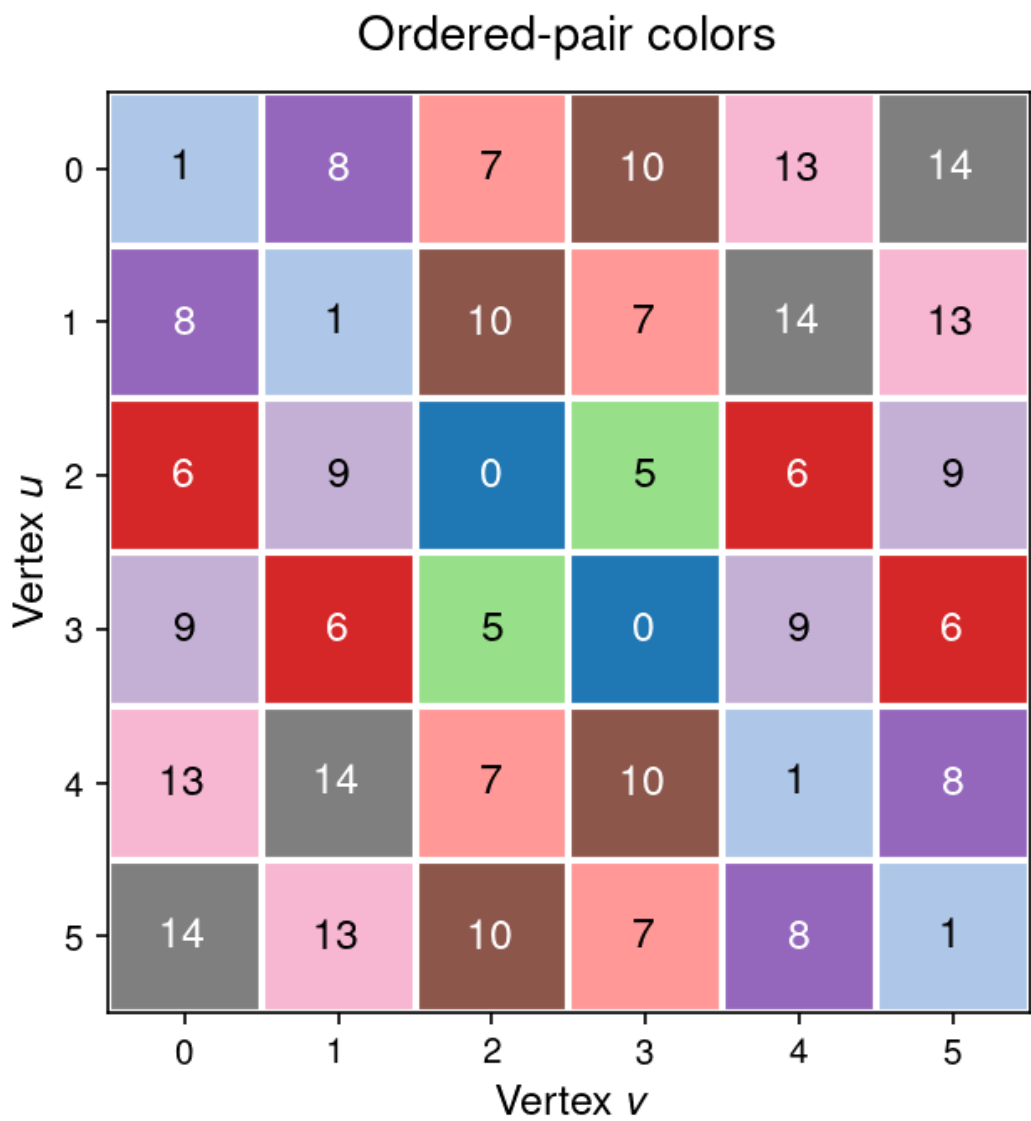


2-WL test

Assigning colors to subsets of vertices



Second iteration



k -WL test

Extending the WL test

This can be generalized to ordered tuples of k elements, giving rise to the k -WL test

Thm.

$(k+1)$ -WL is strictly more expressive than k -WL

But k -WL is also lower bounded in time and space complexity by $\Omega(|V|^k)$, so it scales polynomially on the size of the graph, which limits its applicability in practice

Many GNN extensions are only compared to 2-WL, since it is already highly non-trivial to come up with graphs that are not distinguished by 2-WL

Papp, P. A., and Wattenhofer, R. (2022). A theoretical comparison of graph neural network extensions. In *International Conference on Machine Learning*.

Counting substructures

Lecture 3 — Learning on Topological Data

Differences in substructures

Extending message-passing



If the problem with message-passing is that it is not able to distinguish the triangles on the left from the squares on the right, a possible solution is simply to add this information *ad hoc* as a feature in our GNN

$$\tilde{x}_u = (x_u, \# \text{ triangles with } u, \# \text{ squares with } u, \dots)$$

You can do this for any kind of subgraph, giving rise to Graph Substructure Networks

Bouritsas, G., Frasca, F., Zafeiriou, S., and Bronstein, M. M. (2022). Improving graph neural network expressivity via subgraph isomorphism counting. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 45(1), 657-668.

Chen, Z., Chen, L., Villar, S., and Bruna, J. (2020). Can graph neural networks count substructures?. *Advances in neural information processing systems*, 33, 10383-10395.

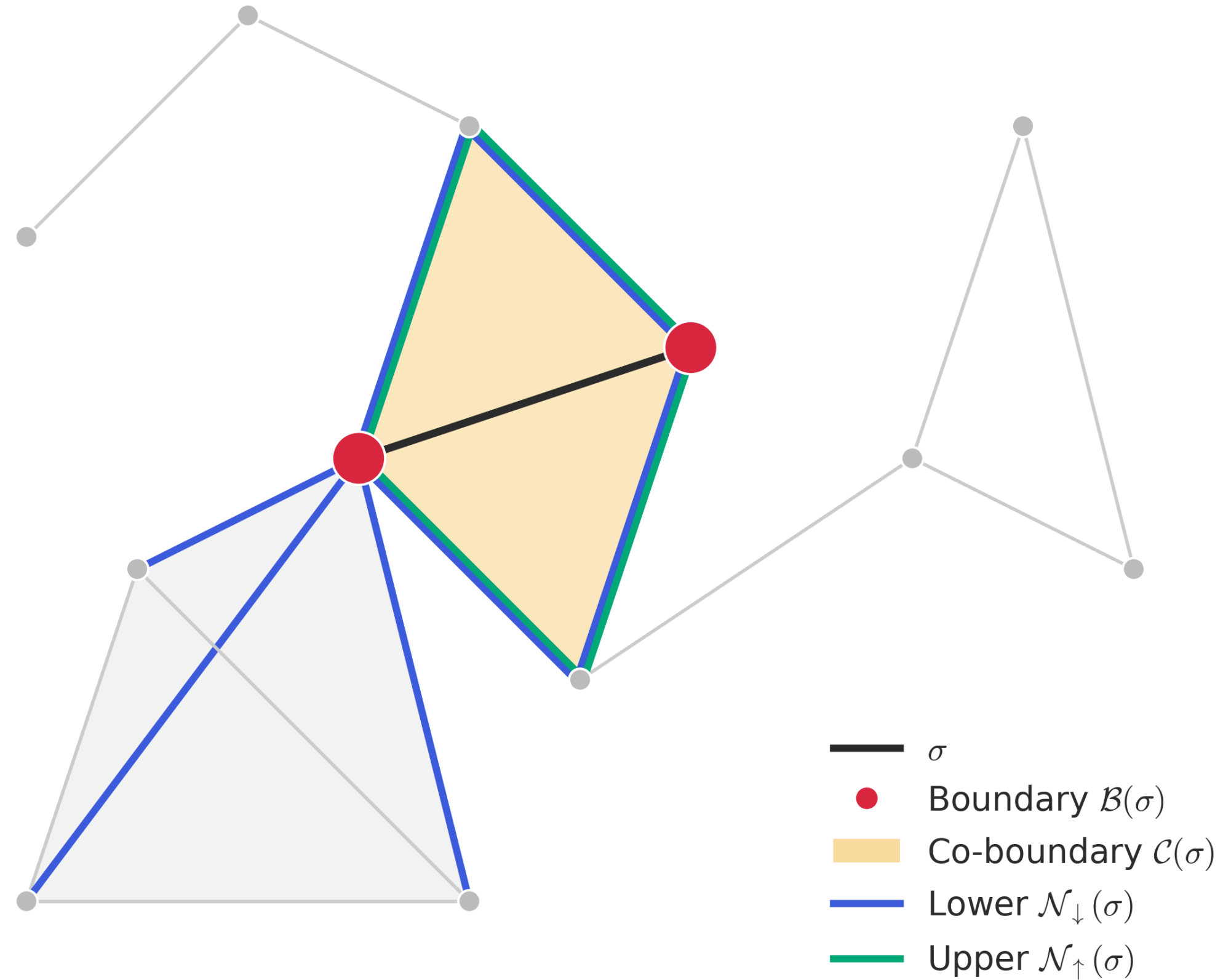
Arvind, V., Fuhlbrück, F., Köbler, J., and Verbitsky, O. (2020). On Weisfeiler—Leman invariance: Subgraph counts and related graph properties. *Journal of Computer and System Sciences*, 113, 42-59.

Higher-order message passing (HOMP)

Lecture 3 — Learning on Topological Data

Weisfeiler and Leman go topological

Including adjacencies



Face relation:

$$\sigma \preceq \tau \iff \sigma \subseteq \tau$$

$$\sigma \prec \tau \iff \sigma \subset \tau \text{ and there is no } \delta \text{ such that } \sigma \subset \delta \subset \tau$$

Adjacencies in a simplicial complex:

- Boundaries: $\mathcal{B}(\sigma) = \{\tau \in X : \tau \prec \sigma\}$

Non-zero rows of the boundary operator B_k

- Co-boundaries: $\mathcal{C}(\sigma) = \{\tau \in X : \sigma \prec \tau\}$

Non-zero rows of $B_{k+1}^\top$

- Lower adjacencies: $\mathcal{N}_{\downarrow}(\sigma) = \{\tau \in X : \exists \delta \in X \text{ s.t. } \delta \prec \tau \text{ and } \delta \prec \sigma\}$

From the down Laplacian $L_k^{\text{down}} = B_k^\top B_k$

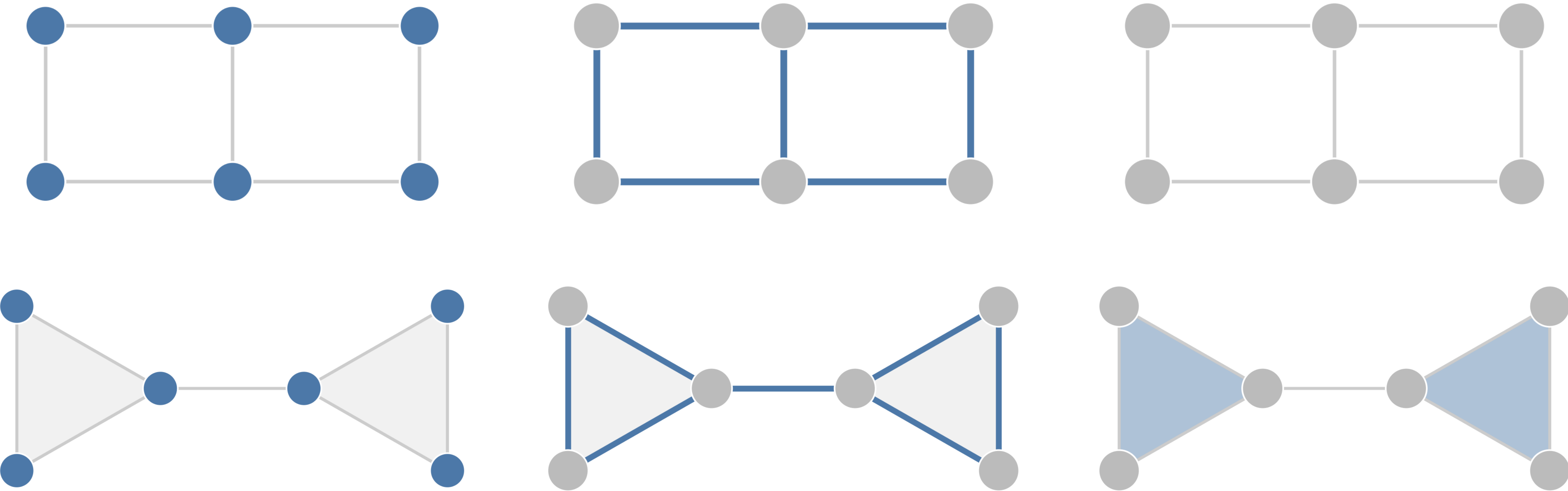
- Upper adjacencies: $\mathcal{N}_{\uparrow}(\sigma) = \{\tau \in X : \exists \delta \in X \text{ s.t. } \tau \prec \delta \text{ and } \sigma \prec \delta\}$

From the up Laplacian $L_k^{\text{up}} = B_{k+1} B_{k+1}^\top$

Weisfeiler and Leman go topological

Simplicial WL algorithm

Initialization



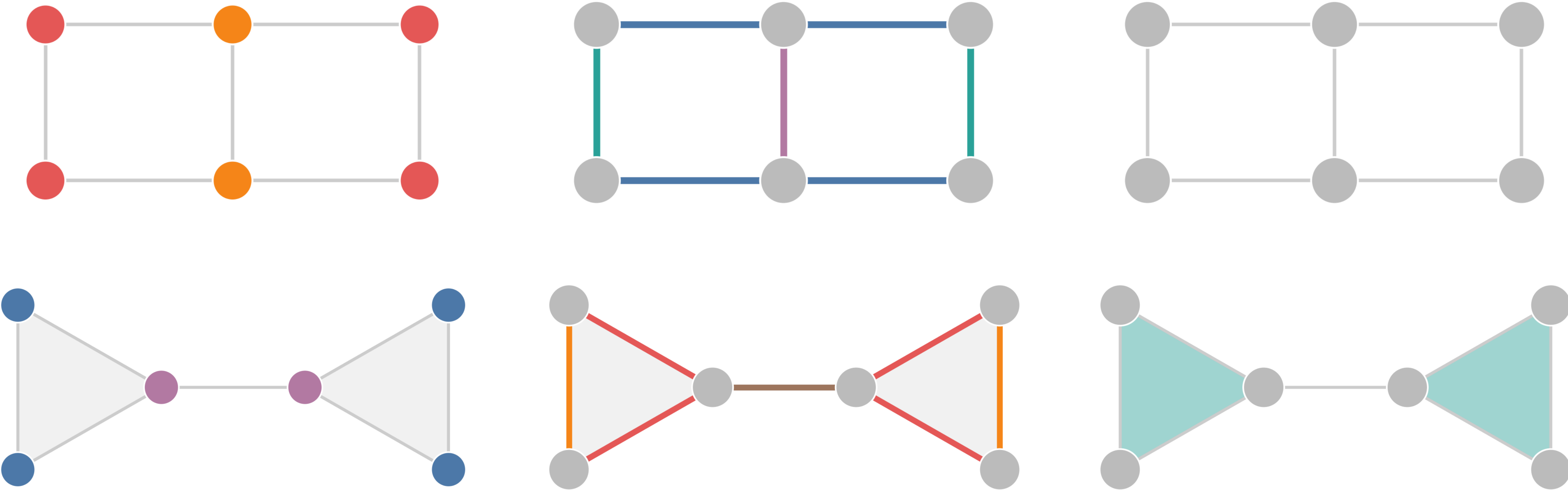
Weisfeiler and Leman go topological

Simplicial WL algorithm

In fact, it is enough to look at the boundary and the upper adjacencies

$$c_{\mathcal{B}}^t(\sigma) = \{\{c_{\tau}^t : \tau \in \mathcal{B}(\sigma)\}\}$$
$$c_{\uparrow}^t(\sigma) = \{\{(c_{\tau}^t, c_{\tau \cup \sigma}^t) : \tau \in \mathcal{N}_{\uparrow}(\sigma)\}\}$$

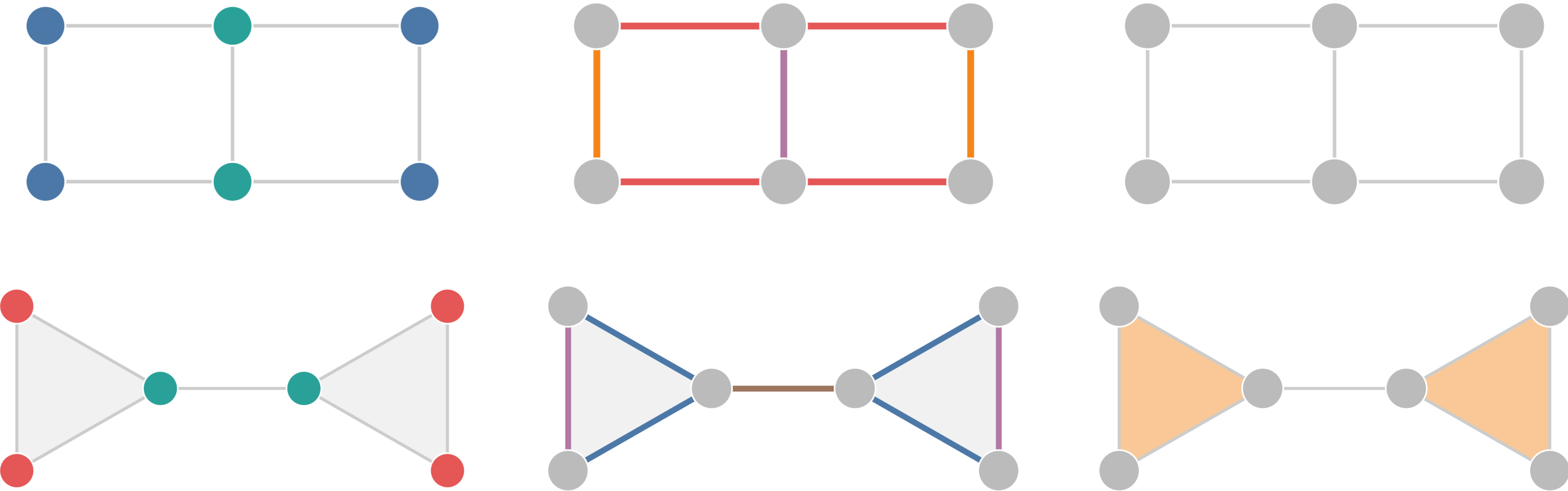
Step 1



Weisfeiler and Leman go topological

Simplicial WL algorithm

Step 2: convergence



Bodnar, Cristian, Fabrizio Frasca, Yuguang Wang, et al. Weisfeiler and Lehman Go Topological: Message Passing Simplicial Networks. *Proceedings of the 38th International Conference on Machine Learning*, July 1, 2021, 1026–37.

Weisfeiler and Leman go topological

Expressiveness of SWL and message-passing simplicial NN

SWL is not less powerful than 3-WL, which is strictly more expressive than WL

Message-passing Simplicial Neural Networks:

$$m_{\mathcal{B}}^{t+1}(\sigma) = \bigoplus_{\tau \in \mathcal{B}(\sigma)} \psi(h_{\sigma}^t, h_{\tau}^t) \quad m_{\mathcal{C}}^{t+1}(\sigma) = \bigoplus_{\tau \in \mathcal{C}(\sigma)} \psi(h_{\sigma}^t, h_{\tau}^t) \quad m_{\uparrow}^{t+1}(\sigma) = \bigoplus_{\tau \in \mathcal{N}_{\uparrow}(\sigma)} \psi(h_{\sigma}^t, h_{\tau}^t, h_{\tau \cup \sigma}^t) \quad m_{\downarrow}^{t+1}(\sigma) = \bigoplus_{\tau \in \mathcal{N}_{\downarrow}(\sigma)} \psi(h_{\sigma}^t, h_{\tau}^t, h_{\tau \cap \sigma}^t)$$

$$h_{\sigma}^{t+1} = \phi \left(h_{\sigma}^t, m_{\mathcal{B}}^{t+1}(\sigma), m_{\mathcal{C}}^{t+1}(\sigma), m_{\uparrow}^{t+1}(\sigma), m_{\downarrow}^{t+1}(\sigma) \right)$$

MPSNs with sufficient layers and injective neighborhood aggregators are as powerful as SWL

Bodnar, Cristian, Fabrizio Frasca, Yuguang Wang, et al. Weisfeiler and Lehman Go Topological: Message Passing Simplicial Networks. *Proceedings of the 38th International Conference on Machine Learning*, July 1, 2021, 1026–37.

SMPNNs in practice

TopoX



TopoNetX (TNX)

TopoNetX is a Python package for computing on topological domains. Topological domains are the natural mathematical structures representing relations between the components of a dataset.

Traditional Discrete Domains

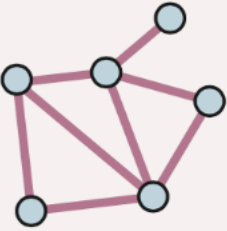
Set

No Relation

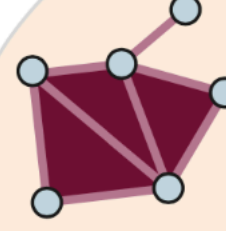


Graph

Pairwise Relations



Simplicial complex

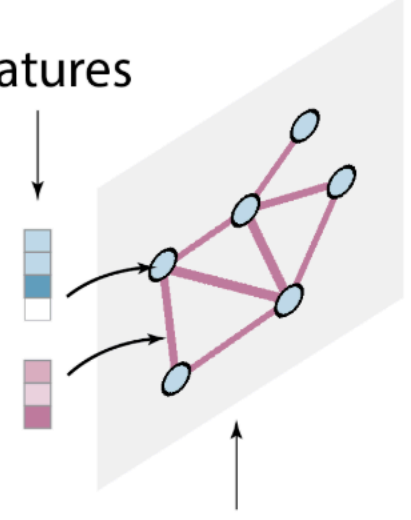




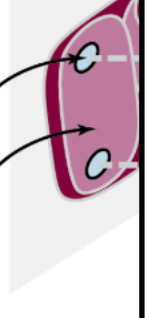
TopoModelX (TMX)

TopoModelX is a Python package for Topological Deep Learning, i.e. Deep Learning on Topological Domains.

Features



Preprocessing



Top

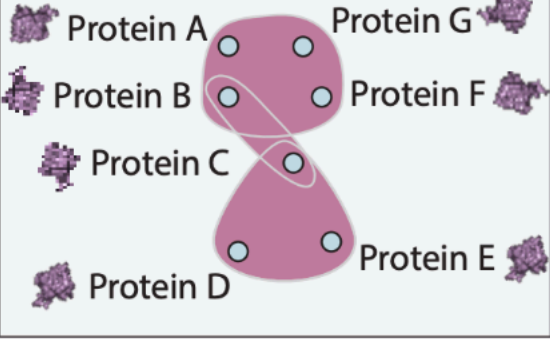


TopoEmbedX (TEX)

TopoEmbedX (TEX) is a Python package for representation Learning on Topological Domains. Topological domains are the natural mathematical structures representing relations between the components of a dataset.

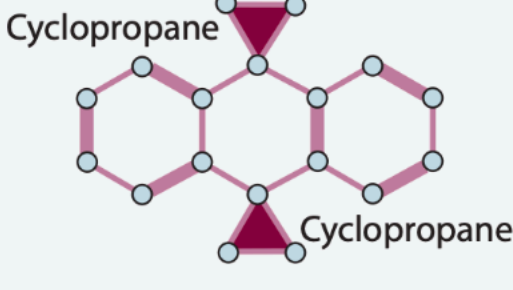
Hypergraph

a.

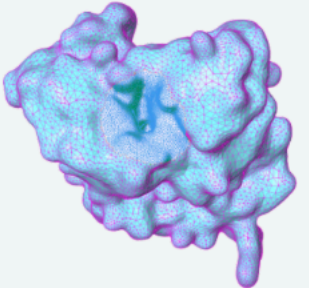


Simplicial Complex

b.

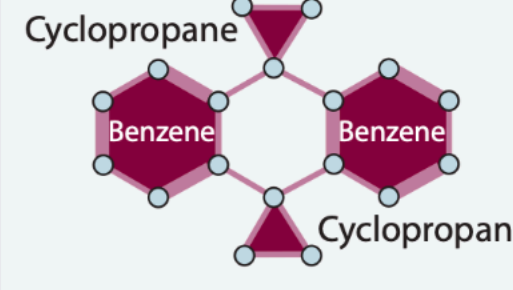


c.

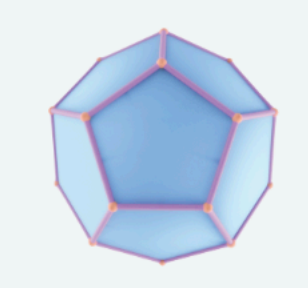


Cellular Complex

d.

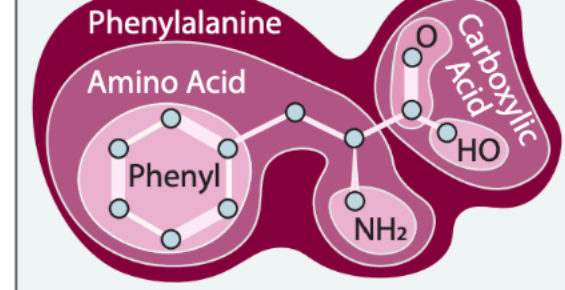


e.

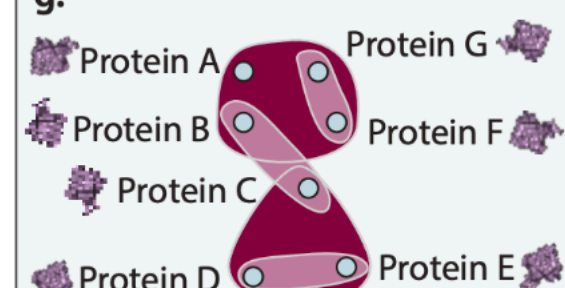


Combinatorial Complex

f.



g.



<https://pyt-team.github.io/index.html>

Papillon, M., Sanborn, S., Hajij, M., & Miolane, N. (2023). Architectures of topological deep learning: A survey of message-passing topological neural networks. *arXiv preprint arXiv:2304.10031*.

Hajij, M., Zamzmi, G., Papamarkou, T., Miolane, N., Guzmán-Sáenz, A., Ramamurthy, K. N., ... & Schaub, M. T. (2022). Topological deep learning: Going beyond graph data. *arXiv preprint arXiv:2206.00606*.

57

Thank you for your attention!



AIDOS LAB
AI FOR DATA-ORIENTED SCIENCE

